

Modellazione - Approccio a Due Stadi

1. Preparazione Dati

```
# 1. PREPARAZIONE DATI E VARIABILI
col_fattori <- c("room_type", "neighbourhood", "host_is_superhost",
               "host_is_local", "instant_bookable", "host_type_classe",
               "host_acceptance_classe", "host_response_classe", "has_reviews",
               "bathrooms_clean")

dati_modelli <- dati_modelli %>%
  # Trasformo in fattori le categoriche
  mutate(across(any_of(col_fattori), as.factor)) %>%
  mutate(
    giorni_da_host = as.numeric(giorni_da_host)
  ) %>%

  # Creazione variabile per effetti casuali
  mutate(zona_tipo = as.factor(paste(neighbourhood, room_type, sep = "_")))

# Pulizia di eventuali NA sul target
dati_modelli <- dati_modelli %>% filter(!is.na(y))

dati_modelli$y = log(dati_modelli$y)

# tab_variabili_pro(dati_modelli)
```

2. Split Stima e Verifica (75% / 25%)

```
set.seed(123)
n <- nrow(dati_modelli)
indici_stima <- sample(1:n, size = round(0.75 * n))

dati_stima <- dati_modelli[indici_stima, ]
dati_verifica <- dati_modelli[-indici_stima, ]

# Funzione per l'errore
get_err <- function(y_hat, y_oss){
  mse <- mean((y_hat - y_oss)^2)
  mae <- mean(abs(y_hat - y_oss))
  return(data.frame(MSE = mse, MAE = mae))
}
```

```

# La risposta dei modelli è in scala logaritmica (y = log(prezzo)). Per metriche
# interpretabili, MSE/MAE vengono riportati sulla SCALA ORIGINALE (€) ri-trasformando
# previsioni e osservato con exp() prima del confronto.
get_err_eur <- function(pred_log, y_log) get_err(exp(pred_log), exp(y_log))

# Tabella riassuntiva errori
errori_test <- data.frame()

# tab_variabili_pro(dati_stima)

```

3. Modelli Lineari + GAM

```

# Per i modelli lineari: Usiamo 'neighbourhood', IGNORIAMO lat, long e zona_tipo
vars_review_all <- c(
  grep("^review_scores", names(dati_stima), value = TRUE),
  "reviews_per_month"
)

# Review scores ed reviews_per_month entrano SOLO come interazione con has_reviews.
# Per chi non ha recensioni (has_reviews=0) il termine è zero per costruzione,
# indipendentemente dal valore imputato; per chi le ha misura la pura qualità.
var_lineari_base <- setdiff(
  names(dati_stima),
  c("y", "listing_id", "latitude", "longitude", "zona_tipo", vars_review_all)
)
interazioni_review <- paste0("has_reviews:", vars_review_all)
var_lineari <- c(var_lineari_base, interazioni_review)

form_lm <- as.formula(paste("y ~", paste(var_lineari, collapse = " + ")))

```

3.1 Lineare Base

```

m_lin <- lm(form_lm, data = dati_stima)
pred_lin <- predict(m_lin, newdata = dati_verifica)
errori_test <- bind_rows(errori_test, get_err_eur(pred_lin, dati_verifica$y) %>%
  ↪ mutate(Modello = "Lineare Base"))

```

3.2 Stepwise Backward

```

m_step <- step(m_lin, direction = "backward", trace = 0)
pred_step <- predict(m_step, newdata = dati_verifica)
errori_test <- bind_rows(errori_test, get_err_eur(pred_step, dati_verifica$y) %>%
  ↪ mutate(Modello = "Lineare Stepwise"))

```

3.3 LMM (Lineare Misto con Stepwise)

```
# L'LMM usa l'intercetta casuale per zona_tipo (neighbourhood x room_type) come
# proxy della posizione. Lat/long continue NON sono incluse qui: sarebbero
# fortemente collineari con zona_tipo (che contiene gia' il quartiere) e
# introdurrebbero instabilita' numerica. Restano invece nei modelli a foresta
# (MERF, Metboost), che sono scale-invarianti. Così l'effetto casuale per zona
# resta interpretabile come "posizione".
var_misti <- setdiff(names(dati_stima), c("y", "listing_id", "latitude", "longitude",
  ↪ "neighbourhood", "zona_tipo"))
form_lmer <- as.formula(paste("y ~", paste(var_misti, collapse = " + "), "+
  ↪ (1|zona_tipo)"))

# Stima e Stepwise su tutto l'insieme di stima
m_lmer_full <- lmer(form_lmer, data = dati_stima)
step_lmer <- step(m_lmer_full, reduce.random = FALSE, trace = 0)
m_lmer_opt <- get_model(step_lmer)

# Random effects:
# Groups      Name      Variance Std.Dev.
# zona_tipo (Intercept) 0.03196  0.1788
# Residual          0.10096  0.3177
# Number of obs: 4332, groups:  zona_tipo, 20

# Verifica
pred_lmer <- predict(m_lmer_opt, newdata = dati_verifica, allow.new.levels = TRUE)
errori_test <- bind_rows(errori_test, get_err_eur(pred_lmer, dati_verifica$y) %>%
  ↪ mutate(Modello = "LMM (Gerarchico)"))
```

3.4 GAM

```
library(gam)

# var_lineari contiene termini formula (es. "has_reviews:review_scores_rating")
# che non sono nomi di colonna: si usano i nomi reali delle colonne sottostanti.
cols_gam <- c("y", var_lineari_base, vars_review_all)
dati_stima_gam <- dati_stima[, cols_gam]
dati_verifica_gam <- dati_verifica[, cols_gam]

# 1. Costruiamo la griglia dei gradi di libertà (df) per i componenti non lineari
# Limitiamo il massimo dei gradi di libertà a 5 per evitare che le curve oscillino troppo
df_list <- paste("df = ", 2:5)

# La funzione gam.scope genera automaticamente l'ambiente di selezione:
gam_list <- gam.scope(
  frame = dati_stima_gam,
  response = which(names(dati_stima_gam) == "y"),
  smoother = "s",
  arg = df_list
)
```

```

# Definiamo il modello nullo di partenza
gam_null <- gam(y ~ 1, data = dati_stima_gam)

# Selezione Stepwise (Forward/Backward) basata sull'AIC sull'insieme di Stima
m_gam_fin <- step.Gam(gam_null, scope = gam_list, direction = "both", trace = FALSE)

# La formula finale del GAM
# print(formula(m_gam_fin))

# y ~ room_type + s(accommodates, df = 5) + s(bedrooms, df = 5) +
#   bathrooms_clean + neighbourhood + host_is_superhost + s(giorni_da_host,
#   df = 5) + host_is_local + instant_bookable + s(calculated_host_listings_count,
#   df = 5) + host_type_classe + host_acceptance_classe + host_response_classe +
#   s(availability_30, df = 5) + s(availability_365, df = 5) +
#   s(number_of_reviews_ltm, df = 5) + s(review_scores_rating,
#   df = 5) + review_scores_cleanliness + review_scores_checkin +
#   review_scores_communication + review_scores_value

# Formula ottima (selezione stepwise già' eseguita una volta). Il modello finale
# viene messo in cache: se esiste lo si carica, altrimenti lo si stima e salva,
# evitando di ri-eseguire la selezione/stima a ogni knit.
form_gam <- y ~ room_type + s(accommodates, df = 5) + s(bedrooms, df = 5) +
  bathrooms_clean + neighbourhood + host_is_superhost + s(giorni_da_host,
  df = 5) + host_is_local + instant_bookable + s(calculated_host_listings_count,
  df = 5) + host_type_classe + host_acceptance_classe + host_response_classe +
  s(availability_30, df = 5) + s(availability_365, df = 5) +
  s(number_of_reviews_ltm, df = 5) + s(review_scores_rating,
  df = 5) + review_scores_cleanliness + review_scores_checkin +
  review_scores_communication + review_scores_value

file_gam <- "../Modelli/modello_gam_finale.rds"
if (file.exists(file_gam)) {
  m_gam_fin <- readRDS(file_gam)
} else {
  m_gam_fin <- gam(form_gam, data = dati_stima_gam)
  saveRDS(m_gam_fin, file_gam)
}

# Previsione finale sull'insieme di Verifica
pred_gam <- predict(m_gam_fin, newdata = dati_verifica_gam)

# Aggiornamento della tabella degli errori
errori_test <- bind_rows(
  errori_test,
  get_err_eur(pred_gam, dati_verifica_gam$y) %>% mutate(Modello = "GAM (Additivo Non
  ↳ Lineare)")
)

```

4. Modelli con Regolazione dei parametri tramite insieme di convalida

```
set.seed(123)
n_stima <- nrow(dati_stima)

# Campiono il 75% del set di Stima per il vero e proprio Train
indici_train <- sample(1:n_stima, size = round(0.75 * n_stima))

dati_train <- dati_stima[indici_train, ]
dati_val <- dati_stima[-indici_train, ]

cat("Dimensioni Set:\n")

## Dimensioni Set:

cat("Train (Stima pura):", nrow(dati_train), "\n")

## Train (Stima pura): 3249

cat("Validation (Convalida tuning):", nrow(dati_val), "\n")

## Validation (Convalida tuning): 1083

cat("Test (Verifica finale):", nrow(dati_verifica), "\n")

## Test (Verifica finale): 1444

# --- Setup Variabili per Alberi ---
var_alberi <- setdiff(names(dati_stima), c("y", "listing_id", "neighbourhood",
  ↪ "zona_tipo"))

# Matrici pronte all'uso per gli alberi
x_train_tree <- dati_train[, var_alberi]
y_train_tree <- dati_train$y

x_val_tree <- dati_val[, var_alberi]
y_val_tree <- dati_val$y

x_stima_tree <- dati_stima[, var_alberi] # Set unito per riaddestramento finale
y_stima_tree <- dati_stima$y

x_verifica_tree <- dati_verifica[, var_alberi]

# Dataset con NA per review scores: per rfsrc e gbm che gestiscono NA natively.
# Il coalesce(., 0) rimane in dati_stima/verifica per lineari e GAM
# (dove has_reviews:review_scores_* azzera già il termine per chi non ha recensioni).
vars_rev_na <- grep("^review_scores|^reviews_per_month", names(dati_stima), value = TRUE)
```

```

ripristina_na <- function(df, vars, flag_var = "has_reviews") {
  df <- as.data.frame(df) # rfsrc richiede data.frame puro, non tibble
  df[df[[flag_var]] == "0", vars] <- NA
  df
}

dati_stima_na <- ripristina_na(dati_stima, vars_rev_na)
dati_verifica_na <- ripristina_na(dati_verifica, vars_rev_na)
dati_train_na <- ripristina_na(dati_train, vars_rev_na)
dati_val_na <- ripristina_na(dati_val, vars_rev_na)

# rfsrc richiede la risposta in double: coercizione esplicita.
dati_stima_na$y <- as.numeric(dati_stima_na$y)
dati_verifica_na$y <- as.numeric(dati_verifica_na$y)
dati_train_na$y <- as.numeric(dati_train_na$y)
dati_val_na$y <- as.numeric(dati_val_na$y)

# Aggiornamento matrici albero con dataset NA
x_train_tree <- dati_train_na[, var_alberi]
x_val_tree <- dati_val_na[, var_alberi]
x_stima_tree <- dati_stima_na[, var_alberi]
x_verifica_tree <- dati_verifica_na[, var_alberi]

```

4.1 Random Forest

Si impiega `randomForestSRC` (`rfsrc`): gestisce nativamente i valori mancanti apprendendo a ogni nodo la direzione di split ottimale per gli NA, così le *review scores* degli alloggi senza recensioni possono restare mancanti senza imputazione arbitraria. Il numero di variabili campionate a ogni split (`mtry`) è stato calibrato per convalida sull'insieme `dati_val`, con ottimo a `mtry = 17`; il modello finale è addestrato sull'intero insieme di stima e salvato su disco per le esecuzioni successive.

```

nn <- 300

# Tuning di mtry già eseguito (ottimo: mtry = 17). Per rieseguirlo, decommentare:
# colnum <- seq(from = 1, to = length(var_alberi), by = 2)
# err_rf_val <- numeric(length(colnum))
# form_rf_tune <- as.formula(paste("y ~", paste(var_alberi, collapse = " + ")))
# for (j in seq_along(colnum)) {
#   m_val <- rfsrc(form_rf_tune,
#     data      = dati_train_na[, c("y", var_alberi)],
#     ntree     = nn, mtry = colnum[j], nodesize = 5,
#     na.action = "na.impute", importance = FALSE, forest = TRUE)
#   pred_val <- predict(m_val, newdata = dati_val_na[, var_alberi], na.action =
#     ↪ "na.impute")$predicted
#   err_rf_val[j] <- mean(abs(y_val_tree - pred_val))
#   cat("Provato mtry:", colnum[j], "| MAE Val:", round(err_rf_val[j], 4), "\n")
# }
# best_mtry <- colnum[which.min(err_rf_val)] # 17

best_mtry <- 17
form_rf <- as.formula(paste("y ~", paste(var_alberi, collapse = " + ")))
file_rf <- "../Modelli/modello_rf_finale_rfsrc.rds"

```

```

# Cache: se il modello esiste lo carico, altrimenti lo addestro e lo salvo.
if (file.exists(file_rf)) {
  m_rf_final <- readRDS(file_rf)
} else {
  m_rf_final <- rfsrc(
    formula   = form_rf,
    data      = dati_stima_na[, c("y", var_alberi)],
    ntree     = 500,
    mtry      = best_mtry,
    nodesize  = 5,
    na.action  = "na.impute",
    importance = TRUE
  )
  saveRDS(m_rf_final, file_rf)
}

pred_rf <- predict(m_rf_final, newdata = dati_verifica_na[, var_alberi], na.action =
  ↪ "na.impute")$predicted

errori_test <- bind_rows(errori_test, get_err_eur(pred_rf, dati_verifica$y) %>%
  ↪ mutate(Modello = "Random Forest"))

```

4.2 Gradient Boosting

```

# Preparazione Dati per GBM
# Filtriamo le colonne per passare solo la 'y' e le variabili ad albero (lat/lon, niente
  ↪ sestieri)
# Uso dataset NA: gbm gestisce NA nativly via surrogate splits

dati_train_gbm <- dati_train_na[, c("y", var_alberi)]
dati_val_gbm <- dati_val_na[, c("y", var_alberi)]
dati_stima_gbm <- dati_stima_na[, c("y", var_alberi)]
dati_verifica_gbm <- dati_verifica_na[, c("y", var_alberi)]

# Calibrazione del GBM (prezzo base) in convalida incrociata: vedi descrizione
# nel testo. Risultati in cache (primo knit calcola, i successivi ricaricano).
file_cache_gbm_base <- "../Modelli/cache_gbm_base_cv_v1_def.rds"

if (file.exists(file_cache_gbm_base)) {
  cache_gbm_base <- readRDS(file_cache_gbm_base)
  tuning_par <- cache_gbm_base$tuning_par
  cv_mae <- cache_gbm_base$cv_mae
  cv_se <- cache_gbm_base$cv_se
  tree_seq_max <- cache_gbm_base$tree_seq_max
  step_trees <- cache_gbm_base$step_trees
  K_cv <- cache_gbm_base$K_cv
  opt_shrinkage <- cache_gbm_base$opt_shrinkage
  opt_depth <- cache_gbm_base$opt_depth
  opt_trees <- cache_gbm_base$opt_trees
  mae_min_gbm <- cache_gbm_base$mae_min_gbm
  se_min_gbm <- cache_gbm_base$se_min_gbm

```

```

sel_shrinkage <- cache_gbm_base$sel_shrinkage
sel_depth <- cache_gbm_base$sel_depth
sel_trees <- cache_gbm_base$sel_trees
mae_sel_gbm <- cache_gbm_base$mae_sel_gbm
sel_trees_final <- cache_gbm_base$sel_trees_final
mod_gbm_final <- cache_gbm_base$modello_finale
} else {

# --- 1. Griglia parametri + tetto di alberi dinamico -----
nus <- c(0.01, 0.05, 0.1, 0.2)
dps <- c(2, 5, 7, 12, 17, 20)
tuning_par <- expand.grid(shrinkage = nus, interaction.depth = dps)
tuning_par$max_trees_run <- dplyr::case_when(
  tuning_par$shrinkage == 0.01 ~ 6000,
  tuning_par$shrinkage == 0.05 ~ 3000,
  tuning_par$shrinkage == 0.10 ~ 1500,
  tuning_par$shrinkage == 0.20 ~ 800,
  TRUE ~ 1500
)

# --- 2. Struttura errori in CV: array [combinazione, n.alberi, fold] -----
step_trees <- 50
tree_seq_max <- seq(step_trees, max(tuning_par$max_trees_run), by = step_trees)
K_cv <- 5
err_cv <- array(NA_real_, dim = c(nrow(tuning_par), length(tree_seq_max), K_cv),
  dimnames = list(NULL, paste0("trees_", tree_seq_max),
    paste0("fold_", 1:K_cv)))

set.seed(123)
folds <- sample(rep(1:K_cv, length.out = nrow(dati_stima_gbm)))
cat("\nAvvio calibrazione GBM in CV (", K_cv, " blocchi, ",
  nrow(tuning_par), " combinazioni)... \n", sep = "")

for (j in seq_len(nrow(tuning_par))) {
  curr_shrinkage <- tuning_par$shrinkage[j]
  curr_depth <- tuning_par$interaction.depth[j]
  curr_max_trees <- tuning_par$max_trees_run[j]
  curr_tree_seq <- seq(step_trees, curr_max_trees, by = step_trees)

  cat(sprintf(" [%2d/%d] shrinkage=%.2f | depth=%d | alberi max=%d\n",
    j, nrow(tuning_par), curr_shrinkage, curr_depth, curr_max_trees))

  for (k in 1:K_cv) {
    idx_val <- which(folds == k)
    train_cv <- dati_stima_gbm[-idx_val, ]
    val_cv <- dati_stima_gbm[ idx_val, ]

    tmp <- gbm(y ~ ., data = train_cv, distribution = "laplace",
      shrinkage = curr_shrinkage, interaction.depth = curr_depth,
      n.trees = curr_max_trees, bag.fraction = 0.5,
      n.minobsinnode = 25, keep.data = FALSE, verbose = FALSE)

    p.tmp <- predict(tmp, newdata = val_cv, n.trees = curr_tree_seq)
    err_cv[j, seq_along(curr_tree_seq), k] <-

```



```

    apply(p.tmp, 2, function(pred) mean(abs(val_cv$y - pred)))
  }
}

# --- 3-5. Ottimo (MAE-CV minimo) e SELEZIONE 1-SE via helper condiviso ----
# seleziona_gbm_1se() (in utils.R) calcola cv_mae/cv_se TRA i fold, trova il
# minimo e applica la regola one-standard-error scegliendo la combinazione piu'
# parsimoniosa (depth * alberi) entro (minimo + 1 SE). Stessa funzione usata
# per il GBM mostra e per la CV annidata: regola identica ovunque.
sel_1se <- seleziona_gbm_1se(err_cv, tuning_par$interaction.depth, tree_seq_max)
cv_mae <- sel_1se$cv_mae
cv_se <- sel_1se$cv_se
dimnames(cv_mae) <- list(NULL, paste0("trees_", tree_seq_max))

opt_shrinkage <- tuning_par$shrinkage[sel_1se$opt_row]
opt_depth <- tuning_par$interaction.depth[sel_1se$opt_row]
opt_trees <- sel_1se$opt_trees
mae_min_gbm <- sel_1se$mae_min
se_min_gbm <- sel_1se$se_min

sel_shrinkage <- tuning_par$shrinkage[sel_1se$sel_row]
sel_depth <- tuning_par$interaction.depth[sel_1se$sel_row]
sel_trees <- sel_1se$sel_trees
mae_sel_gbm <- sel_1se$mae_sel

# --- 6. Modello finale su TUTTO l'insieme di stima (parametri 1-SE) -----
# In CV ogni fold vede (K-1)/K dei dati; sull'intero set l'ottimo cade a un
# numero di alberi leggermente maggiore: riscalo di K/(K-1).
sel_trees_final <- round(sel_trees * K_cv / (K_cv - 1))
mod_gbm_final <- gbm(y ~ ., data = dati_stima_gbm, distribution = "laplace",
  n.trees = sel_trees_final, shrinkage = sel_shrinkage,
  interaction.depth = sel_depth, bag.fraction = 0.5,
  n.minobsinnode = 25, keep.data = FALSE, verbose = FALSE)

saveRDS(list(tuning_par = tuning_par, cv_mae = cv_mae, cv_se = cv_se,
  tree_seq_max = tree_seq_max, step_trees = step_trees, K_cv = K_cv,
  opt_shrinkage = opt_shrinkage, opt_depth = opt_depth, opt_trees =
    ↪ opt_trees,
  mae_min_gbm = mae_min_gbm, se_min_gbm = se_min_gbm,
  sel_shrinkage = sel_shrinkage, sel_depth = sel_depth, sel_trees =
    ↪ sel_trees,
  mae_sel_gbm = mae_sel_gbm, sel_trees_final = sel_trees_final,
  modello_finale = mod_gbm_final),
  file = file_cache_gbm_base)
}

# Parametri/alberi del modello finale (riusati a valle per la stima base)
opt_shrinkage_final <- sel_shrinkage
opt_depth_final <- sel_depth
opt_trees_final <- mod_gbm_final$n.trees

cat(sprintf("\nGBM CV - ottimo (MAE-CV minimo): shrinkage=%.2f | depth=%d | alberi=%d |
  ↪ MAE-CV=%.3f (log) (SE %.3f)\n",
  opt_shrinkage, opt_depth, opt_trees, mae_min_gbm, se_min_gbm))

```

```
##
## GBM CV - ottimo (MAE-CV minimo): shrinkage=0.01 | depth=12 | alberi=5950 | MAE-CV=0.214 (log) (SE 0.0001)
```

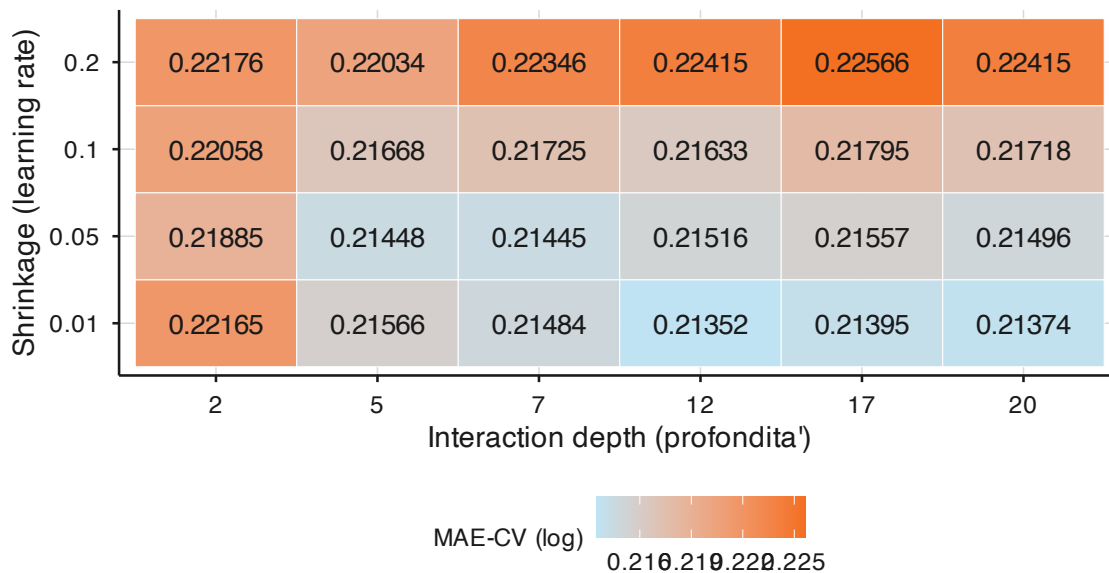
```
cat(sprintf("GBM CV - regola 1-SE (scelto): shrinkage=%.2f | depth=%d | alberi=%d |
  ↳ MAE-CV=%.3f (log) (soglia %.3f)\n",
    sel_shrinkage, sel_depth, sel_trees, mae_sel_gbm, mae_min_gbm + se_min_gbm))
```

```
## GBM CV - regola 1-SE (scelto): shrinkage=0.05 | depth=5 | alberi=1300 | MAE-CV=0.216 (log) (soglia 0.225)
```

```
# --- Heatmap del tuning: MAE-CV minimo per ciascuna combinazione -----
tuning_par$min_error <- apply(cv_mae, 1, min, na.rm = TRUE)
grafico_tuning_gbm <- ggplot(tuning_par,
  aes(x = factor(interaction.depth), y = factor(shrinkage), fill = min_error)) +
  geom_tile(color = col_sfondo) +
  geom_text(aes(label = round(min_error, 5)), colour = "grey10", size = 4) +
  scale_fill_gradient(low = "#BFE3F2", high = col_arancio) +
  labs(title = "Tuning GBM in convalida incrociata: MAE-CV (scala log)",
    subtitle = "MAE medio sui K blocchi per combinazione (risposta in log-prezzo) -
  ↳ selezione con regola 1-SE",
    x = "Interaction depth (profondita')", y = "Shrinkage (learning rate)",
    fill = "MAE-CV (log)") +
  tema_slide
print(grafico_tuning_gbm)
```

Tuning GBM in convalida incrociata: MAE-CV (scala log)

MAE medio sui K blocchi per combinazione (risposta in log-prezzo) - selezione con reg



```
# Previsione finale sull'insieme di Verifica con il modello selezionato (1-SE)
pred_gbm <- predict(mod_gbm_final, newdata = dati_verifica_gbm, n.trees =
  ↳ opt_trees_final)

# Aggiorno la tabella errori
errori_test <- bind_rows(errori_test, get_err_eur(pred_gbm, dati_verifica_gbm$y) %>%
  mutate(Modello = "Gradient Boosting (GBM)"))
```

La calibrazione avviene in **convalida incrociata a 5 blocchi** sull'insieme di stima: per ogni combinazione di iperparametri il MAE è mediato sui fold e se ne ricava l'errore standard *tra* i fold. La griglia assegna inoltre a ogni *learning rate* un tetto di alberi proporzionato (più basso lo shrinkage, più alberi servono per convergere), evitando di addestrare 6000 alberi anche dove ne bastano poche centinaia. Il **minimo del MAE-CV** corrisponde a shrinkage = 0.01, profondità = 12 e 5950 alberi (SE tra i fold 0.002). Applicando la regola *one-standard-error* — che entro una deviazione standard dal minimo preferisce il modello **più semplice** — la scelta finale ricade su shrinkage = 0.05, profondità = 5 e 1300 alberi (riscalati a 1625 per il rifit sull'intero insieme di stima, dato che in CV ogni fold ne vede solo 4 su 5). Si ottiene un modello meno complesso e meno incline all'overfitting a parità sostanziale di errore.

4.3 Mixed Effects Random Forest - MERF

```
# Variabili e Formula
# Escludiamo neighbourhood e zona_tipo dai predittori fissi della foresta
var_merf <- setdiff(names(dati_stima_na), c("y", "listing_id", "neighbourhood",
  ↪ "zona_tipo"))
form_merf <- as.formula(paste("y ~", paste(var_merf, collapse = " + ")))
nn <- 300
# colnum_merf <- seq(from = 1, to = 25, by = 2)
# err_merf_val <- numeric(length(colnum_merf))
# # #
# # #
# # #
# # # -- Tuning MERF via stima-convalida -----
# # # Per ogni valore di mtry: stima MERF su dati_train_na, previsione su dati_val_na.
# # # Usa ntree_iter ridotto (200) per accelerare il tuning
# # #
# set.seed(123)
#
# for (j in seq_along(colnum_merf)) {
#
#   cat(sprintf("\n+=====+\\n"))
#   cat(sprintf("|  MERF Tuning  %d/%d  -  mtry = %d\\n",
#     j, length(colnum_merf), colnum_merf[j]))
#   cat(sprintf("+=====+\\n"))
#
#   m_tune <- MERF_rfsrc_safe(
#     formula      = form_merf,
#     data         = dati_train_na,
#     random       = " + (1/zona_tipo)",
#     ntree_iter   = 200,           # iter leggere nel tuning
#     ntree_final  = nn,
#     mtry_grid    = colnum_merf[j],
#     nodesize     = 15,
#   )
#
#   # Previsione sull'insieme di convalida (parte fissa + parte random)
#   pred_fix <- predict(m_tune$RandomForest,
#     newdata      = dati_val_na,
#     na.action    = "na.impute")$predicted
#   pred_ran <- predict(m_tune$EffectModel,
#     newdata      = dati_val_na,
#     allow.new.levels = TRUE) -
```

```

#           predict(m_tune$EffectModel,
#                   newdata = dati_val_na,
#                   re.form = ~0)
#   pred_tot <- pred_fix + pred_ran
#
#   err_merf_val[j] <- mean(abs(dati_val_na$y - pred_tot))
#   cat(sprintf(" > MAE Validation mtry=%d: %.2f\n", colnum_merf[j], err_merf_val[j]))
# }
# #
# indice_min_merf <- which.min(err_merf_val)
# best_mtry_merf <- colnum_merf[indice_min_merf]
# #
# # cat(sprintf("\n== Risultati Tuning MERF =====\n"))
# for (j in seq_along(colnum_merf)) {
#   marker <- if (j == indice_min_merf) " <- best" else ""
#   cat(sprintf("   mtry = %2d | MAE = %.2f%s\n",
#               colnum_merf[j], err_merf_val[j], marker))
# }
# cat(sprintf(" Best mtry: %d (MAE = %.2f)\n", best_mtry_merf,
#   ↪   err_merf_val[indice_min_merf]))
# #
# # # Grafico tuning MERF
# plot(colnum_merf, err_merf_val, type = "b", pch = 19, col = "steelblue", lwd = 2,
#      xlab = "mtry", ylab = "MAE (Validation)",
#      main = "Tuning MERF: errore sul validation set")
# points(best_mtry_merf, err_merf_val[indice_min_merf], col = "red", pch = 19, cex = 1.5)
#
best_mtry_merf = 13
#
# # -- Stima Modello Finale (su tutto l'insieme di STIMA: Train + Val) -----
# # Usa best_mtry_merf ricavato dal tuning rfsrc su dati con NA nativi.
# cat(sprintf("\n== Stima modello MERF finale (ntree=%d, mtry=%d) ===\n",
#             1500L, best_mtry_merf))
#
# m_merf_final <- MERF_rfsrc_safe(
#   formula      = form_merf,
#   data         = dati_stima_na,
#   random       = " + (1|zona_tipo)",
#   ntree_iter   = 300,
#   ntree_final  = 1500,
#   mtry_grid    = best_mtry_merf,
#   nodesize     = 15,
#   verbose      = TRUE
# )
# Cache del MERF (oggetto pesante, con lat/long tra i predittori della foresta):
# se il file esiste lo carico, altrimenti addestro sull'intero insieme di stima e salvo.
file_merf <- "/Users/riccardoforte/Desktop/Strumenti per dati
  ↪   aziendali/modello_merf_finale_rfsrc.rds"
if (file.exists(file_merf)) {
  m_merf_final <- readRDS(file_merf)
} else {
  m_merf_final <- MERF_rfsrc_safe(
    formula      = form_merf, data = dati_stima_na,
    random       = " + (1|zona_tipo)",

```

```

    ntree_iter = 300, ntree_final = 1500,
    mtry_grid = best_mtry_merf, nodesize = 15, verbose = TRUE
  )
  saveRDS(m_merf_final, file_merf)
}

# Previsione finale sull'insieme di verifica
p_fix_test <- predict(m_merf_final$RandomForest, newdata = dati_verifica_na, na.action =
  ↪ "na.impute")$predicted
p_ran_test <- predict(m_merf_final$EffectModel, newdata = dati_verifica_na,
  ↪ allow.new.levels = TRUE) -
  predict(m_merf_final$EffectModel, newdata = dati_verifica_na, re.form = ~0)

pred_merf_test <- p_fix_test + p_ran_test

# Aggiornamento Tabella Errori
errori_test <- bind_rows(
  errori_test,
  get_err_eur(pred_merf_test, dati_verifica$y) %>% mutate(Modello = "MERF (Gerarchico)")
)

```

4.4 Mixed Effects Gradient Boosting - Metboost

Nota sulla calibrazione. Il tuning del Metboost è volutamente **leggero e non ottimale**: la griglia esplora solo `depth` {1, 2} e `shrinkage` {0.5, 0.7, 1.0} (valori di apprendimento alti, profondità basse), con MAE di validazione molto ravvicinati tra le combinazioni. Una calibrazione più accurata richiederebbe `shrinkage` più piccoli e profondità maggiori, ma il costo computazionale del Metboost (un `rpart` + un `lmer` per ogni albero) è elevato; per gli scopi del confronto il modello viene quindi riportato con questa calibrazione preliminare.

```

metboost_fit_path_manual <- function(train, valid = NULL, y_name = "y", vars_x,
  ↪ group_var,
                                     M_max = 200, eval_rounds = c(250, 500, 750, 1000),
                                     depth = 3, shrinkage = 0.05, bag_fraction = 0.5,
                                     minsplit = 20,
                                     minbucket = 10,
                                     cp = 0.001) {

  train <- as.data.frame(train)
  train[[group_var]] <- factor(train[[group_var]])
  y <- train[[y_name]]
  n <- nrow(train)
  init_value <- mean(y)
  f_hat <- rep(init_value, n)
  g_hat <- rep(0, n)
  r <- y - f_hat

  # Helper locale per l'assegnazione delle foglie (autosufficiente, non dipende
  # da get_leaf_id di utils.R). predict.rpart NON espone type="where", quindi si
  # usa il trick di treeClust: si etichetta ogni nodo con il proprio indice di
  # riga in frame, poi predict(type="vector") restituisce l'indice della foglia
  # di arrivo. rpart usa i surrogate splits nella discesa, quindi ogni riga -
  # anche con NA nelle review scores - riceve sempre un leaf ID valido (mai NA,

```

```

# a differenza di partykit) e funziona anche su dati nuovi.
leaf_assign <- function(tree, df) {
  tree$frame$yval <- seq_len(nrow(tree$frame))
  factor(as.character(predict(tree, newdata = df, type = "vector")))
}

trees <- vector("list", M_max)
lmes <- vector("list", M_max)
leaf_levels_list <- vector("list", M_max)

use_valid <- !is.null(valid)
if (use_valid) {
  valid <- as.data.frame(valid)
  valid[[group_var]] <- factor(valid[[group_var]], levels = levels(train[[group_var]]))
  pred_valid <- rep(init_value, nrow(valid))
  eval_rounds <- sort(unique(eval_rounds))
  mae_valid <- rep(NA_real_, length(eval_rounds))
}

cat("\n Addestramento Metboost (", M_max, " alberi ):\n", sep="")
pb <- txtProgressBar(min = 0, max = M_max, style = 3, width = 50)
# style = 3 mostra la percentuale (es. 45%), width definisce la larghezza della barra

for (m in 1:M_max) {
  idx_sub <- sample(seq_len(n), size = max(2, floor(bag_fraction * n)), replace =
↪ FALSE)
  train_sub <- train[idx_sub, , drop = FALSE]
  r_sub <- r[idx_sub]

  tree_dat <- train_sub[, vars_x, drop = FALSE]
  tree_dat$.r <- r_sub

  tree_formula <- as.formula(paste(".r ~", paste(vars_x, collapse = " + ")))

  tree_m <- rpart(formula = tree_formula, data = tree_dat, method = "anova",
    control = rpart.control(maxdepth = depth, minsplit = minsplit,
↪ minbucket = minbucket, cp = cp))

  leaf_factor_train <- leaf_assign(tree_m, train)
  leaf_levels_list[[m]] <- levels(leaf_factor_train)

  lme_dat <- data.frame(r_prev = r, leaf = leaf_factor_train, group =
↪ train[[group_var]])

  met_ctrl <- lmerControl(
    optimizer = "bobyqa",
    calc.derivs = FALSE,
    check.conv.singular = .makeCC(action = "ignore", tol = 1e-4),
    optCtrl = list(maxfun = 1e5)
  )

  # Guard albero degenerare: con stump (depth=1) + shrinkage alto, dopo molte
  # iterazioni i residui sono minimi e rpart non trova split utili -> un solo

```

```

# nodo -> leaf con 1 livello -> lmer non può costruire i contrasti.
# In quel caso fit intercetta + random intercept per non spezzare il path.
if (nlevels(leaf_factor_train) < 2) {
  lme_m <- lmer(r_prev ~ 1 + (1 | group), data = lme_dat, REML = TRUE,
               control = met_ctrl)
} else {
  lme_m <- lmer(r_prev ~ 0 + leaf + (0 + leaf || group), data = lme_dat,
               REML = TRUE, control = met_ctrl)
}

pred_fixed_m <- predict(lme_m, newdata = lme_dat, re.form = ~0)
pred_total_m <- predict(lme_m, newdata = lme_dat, re.form = NULL)
pred_random_m <- pred_total_m - pred_fixed_m

f_hat <- f_hat + shrinkage * pred_fixed_m
g_hat <- g_hat + shrinkage * pred_random_m
r <- y - f_hat - g_hat

trees[[m]] <- tree_m
lmes[[m]] <- lme_m

if (use_valid) {
  leaf_valid <- factor(as.character(leaf_assign(tree_m, valid)),
                      levels = levels(leaf_factor_train))
  pred_dat_valid <- data.frame(leaf = leaf_valid, group = valid[[group_var]])

  pred_fixed_valid <- predict(lme_m, newdata = pred_dat_valid, re.form = ~0,
↪ allow.new.levels = TRUE)
  pred_total_valid <- predict(lme_m, newdata = pred_dat_valid, re.form = NULL,
↪ allow.new.levels = TRUE)
  pred_random_valid <- pred_total_valid - pred_fixed_valid

  pred_valid <- pred_valid + shrinkage * (pred_fixed_valid + pred_random_valid)

  if (m %in% eval_rounds) {
    j <- which(eval_rounds == m)
    mae_valid[j] <- mean(abs(valid[[y_name]] - pred_valid))
  }
}

setTxtProgressBar(pb, m)
}

close(pb)
cat("\n")

out <- list(trees = trees, lmes = lmes, leaf_levels = leaf_levels_list, vars_x =
↪ vars_x,
           group_var = group_var, y_name = y_name, M = M_max, depth = depth,
           shrinkage = shrinkage, init_value = init_value, fitted = f_hat + g_hat)
if (use_valid) out$valid_path <- data.frame(M = eval_rounds, MAE_val = mae_valid)
out

```

```
}
```

```
# Preparazione Variabili
# rpart gestisce NA nativamente tramite surrogate splits: per ogni nodo primario
# apprende variabili sostituite da usare quando il valore primario è mancante.
# Non è quindi necessario modificare l'algoritmo - basta passare i dataset NA.
var_met <- setdiff(names(dati_stima_na), c("y", "listing_id", "neighbourhood",
  ↪ "zona_tipo"))

# # # 1. Imposto la griglia base
met_grid <- expand.grid(
  depth      = c(1, 2),
  shrinkage  = c(0.5, 0.7, 1),
  min_mae    = NA,
  best_M     = NA,
  M_max_run  = NA
)
#
# # 2. Logica Adattiva per il tetto massimo degli alberi
# # Regola base per lo shrinkage (più è piccolo, più alberi servono)
met_grid$M_max_run[met_grid$shrinkage == 1] <- 200
met_grid$M_max_run[met_grid$shrinkage == 0.1] <- 200
met_grid$M_max_run[met_grid$shrinkage == 0.20] <- 200
met_grid$M_max_run[met_grid$shrinkage == 0.50] <- 200
met_grid$M_max_run[met_grid$shrinkage == 0.70] <- 200
#
# # Raffinatezza estrema: se la profondità è solo 1 (Decision Stump),
# # l'albero è così "stupido" che ha bisogno di un ulteriore 30% di alberi per imparare
met_grid$M_max_run[met_grid$depth == 1] <- met_grid$M_max_run[met_grid$depth == 1] * 1.3

# Cache del tuning Metboost: la griglia di calibrazione è onerosa; se il risultato
# esiste lo si carica, altrimenti si esegue e si salva (nessun codice da commentare).
file_met_tuning <- "../Modelli/cache_metboost_tuning.rds"
if (file.exists(file_met_tuning)) {
  met_grid <- readRDS(file_met_tuning)
} else {
  set.seed(123)

  cat("\nInizio Regolazione Metboost (Totale combinazioni:", nrow(met_grid), ")\n")
  tempo_inizio <- Sys.time()
  #
  # # 3. Ciclo di Tuning sul Validation Set
  for(i in 1:nrow(met_grid)) {

    # CREAZIONE DINAMICA: Definisco gli step di valutazione in base al tetto di questa
    ↪ riga
    # Calcoliamo l'errore ogni 50 alberi fino ad arrivare al M_max_run specifico
    step_valutazione <- seq(50, met_grid$M_max_run[i], by = 50)

    cat("\n--- Test", i, "di", nrow(met_grid),
      "\n--- (depth:", met_grid$depth[i],
      "\n| shrinkage:", met_grid$shrinkage[i],
      "\n| Alberi massimi consentiti:", met_grid$M_max_run[i], ")\n")
  }
}
```



```

# Stima del modello
tmp_model <- suppressWarnings({
  metboost_fit_path_manual(
    train = dati_train_na,
    valid = dati_val_na,
    y_name = "y",
    vars_x = var_met,
    group_var = "zona_tipo",
    M_max = met_grid$M_max_run[i], # PASSIAMO IL VALORE DINAMICO QUI
    eval_rounds = step_valutazione,
    shrinkage = met_grid$shrinkage[i],
    depth = met_grid$depth[i],
    minbucket = 25
  )
})

best_idx <- which.min(tmp_model$valid_path$MAE_val)

met_grid$min_mae[i] <- tmp_model$valid_path$MAE_val[best_idx]
met_grid$best_M[i] <- tmp_model$valid_path$M[best_idx]

cat("* Miglior M trovato:", met_grid$best_M[i], "| MAE Val:",
    ↪ round(met_grid$min_mae[i], 2), "\n")

# Salvataggio di sicurezza (se salta la corrente alle 4 di notte, non perdi i dati!)
saveRDS(met_grid, "backup_met_grid_notturmo.rds")
}
#
tempo_fine <- Sys.time()
cat("\nTempo totale di Tuning:", round(difftime(tempo_fine, tempo_inizio, units =
    ↪ "mins"), 2), "minuti.\n")
saveRDS(met_grid, file_met_tuning)
}

# Estrazione parametri ottimali dalla griglia (caricata o appena calcolata)
best_params_met <- met_grid[which.min(met_grid$min_mae), ]

cat("\n Regolazione Completata. Parametri Ottimali Metboost:\n")

```

```

##
## Regolazione Completata. Parametri Ottimali Metboost:

```

```

cat("- Profondità (depth):", best_params_met$depth, "\n")

```

```

## - Profondità (depth): 2

```

```

cat("- Learning Rate (shrinkage):", best_params_met$shrinkage, "\n")

```

```

## - Learning Rate (shrinkage): 0.5

```

```

cat("- Numero Alberi Ottimale (M):", best_params_met$best_M, "\n\n")

## - Numero Alberi Ottimale (M): 200

# Addestramento Modello Finale su tutto l'insieme di stima (con salvataggio automatico)
nome_file_metboost <- "/Users/riccardoforte/Desktop/Strumenti per dati
  ↪  aziendali/modello_metboost_finale_na_def1.rds"

if (file.exists(nome_file_metboost)) {
  cat("Modello Metboost finale già esistente! Caricamento...\n")
  met_model_final <- readRDS(nome_file_metboost)
} else {
  cat("Addestramento Metboost finale sull'intero set di Stima...\n")
  met_model_final <- metboost_fit_path_manual(
    train      = dati_stima_na,    # NA nativi: rpart gestisce via surrogate splits
    y_name     = "y",
    vars_x     = var_met,
    group_var  = "zona_tipo",
    M_max      = 500,
    shrinkage  = best_params_met$shrinkage,
    depth      = best_params_met$depth
  )
  saveRDS(met_model_final, nome_file_metboost)
}

## Modello Metboost finale già esistente! Caricamento...

# Previsione sull'insieme di Verifica (NA nativi: rpart usa surrogate splits)
pred_met <- metboost_predict_manual(met_model_final, newdata = dati_verifica_na)

# Aggiorna tabella errori
errori_test <- bind_rows(
  errori_test,
  get_err_eur(pred_met$pred, dati_verifica$y) %>% mutate(Modello = "Metboost
    ↪  (Gerarchico)")
)

```

5. Risultato finale

```

errori_test %>%
  select(Modello, MSE, MAE) %>%
  arrange(MAE) %>%
  kable(digits = 2)

```

Modello	MSE	MAE
Gradient Boosting (GBM)	4195.18	42.35
Random Forest	4458.22	44.39

Modello	MSE	MAE
MERF (Gerarchico)	4843.76	46.47
GAM (Additivo Non Lineare)	5126.13	48.05
LMM (Gerarchico)	5392.56	49.53
Lineare Base	5434.73	49.68
Lineare Stepwise	5440.15	49.70
Metboost (Gerarchico)	6145.17	52.21

```
errori_test$RMSE = sqrt(errori_test$MSE)
```

```
errori_test %>%
  select(Modello, MAE, RMSE) %>%
  arrange(MAE) %>%
  kable(digits = 2)
```

Modello	MAE	RMSE
Gradient Boosting (GBM)	42.35	64.77
Random Forest	44.39	66.77
MERF (Gerarchico)	46.47	69.60
GAM (Additivo Non Lineare)	48.05	71.60
LMM (Gerarchico)	49.53	73.43
Lineare Base	49.68	73.72
Lineare Stepwise	49.70	73.76
Metboost (Gerarchico)	52.21	78.39

```
# MAE (in €) per i riferimenti dinamici nel testo della sezione interpretabilità
mae_di <- setNames(round(errori_test$MAE, 1), errori_test$Modello)
```

5.1 Metriche sul sotto-campione competitor (Hotel: Lido/Castello, 2 posti letto)

Le stesse metriche di selezione, calcolate **solo** sugli appartamenti concorrenti diretti dell'Hotel — sestieri **Lido** e **Castello** con **2 posti letto** — presenti nell'insieme di verifica. È il segmento davvero rilevante per il posizionamento della struttura.

Table 3: Metriche sul sotto-campione competitor (Lido/Castello, 2 posti letto): 124 annunci di verifica. Con pochi annunci le stime sono indicative.

Modello	MAE	RMSE
Random Forest	33.15	54.21
MERF (Gerarchico)	34.25	56.27
Gradient Boosting (GBM)	35.02	58.17
LMM (Gerarchico)	36.58	59.65
GAM (Additivo Non Lineare)	36.58	59.71
Lineare Stepwise	36.87	59.79
Lineare Base	36.95	60.02
Metboost (Gerarchico)	37.97	60.40

Su questo sotto-campione (124 annunci) le metriche vanno lette con cautela per la bassa numerosità, ma indicano quale modello generalizza meglio proprio sul segmento di interesse dell'Hotel.

6. Interpretabilità dei modelli e ruolo della posizione

Avendo incluso **latitudine e longitudine continue anche nei modelli a effetti misti** (tranne LMM), il set di variabili è ora **allineato** a quello dei modelli ad albero: la posizione non è più una leva esclusiva di GBM e Random Forest. Di conseguenza i modelli gerarchici si avvicinano: l'**LMM** scende a 50 € di MAE, in linea col **GAM** (48 €) e meglio dei lineari semplici.

Resta comunque un vantaggio dei modelli ad albero — **GBM** (42 €) e **Random Forest** (44 €) — ma a parità di variabili dipende ora dalla **forma funzionale**, non dall'informazione disponibile: gli alberi catturano interazioni non lineari arbitrarie fra posizione e caratteristiche dell'alloggio, mentre LMM e MERF combinano una parte lineare/additiva con un'intercetta di gruppo. Le analisi seguenti mostrano *dove* pesa la posizione e *come* i diversi modelli la sfruttano.

```
# Funzione di previsione del GBM base. Il modello e' in scala log: si ri-trasforma
# con exp() così i PDP sono espressi sulla scala € del prezzo.
pred_gbm_base <- function(d) exp(predict(mod_gbm_final, newdata = d, n.trees =
  ↪ opt_trees_final))

# MAE (in €) dei modelli, per i riferimenti dinamici nel testo
mae_di <- setNames(round(errori_test$MAE, 1), errori_test$Modello)

# (a) Influenza relativa: quanto pesa la posizione nel GBM base
ri_gbm <- summary(mod_gbm_final, n.trees = opt_trees_final, plotit = FALSE)
ri_gbm$var <- as.character(ri_gbm$var)
peso_posizione_gbm <- sum(ri_gbm$rel.inf[ri_gbm$var %in% c("latitude", "longitude")])
ri_top <- head(ri_gbm[order(-ri_gbm$rel.inf), ], 12)

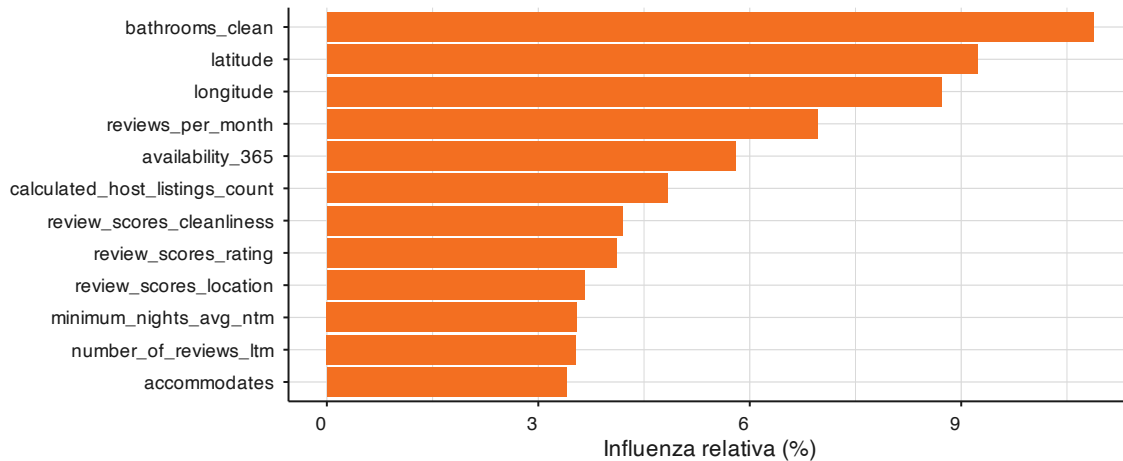
# =====
# 1. Variabili più importanti
# =====

g_ri <- ggplot(ri_top, aes(reorder(var, rel.inf), rel.inf)) +
  geom_col(fill = col_arancio) +
  coord_flip() +
  labs(title = "GBM: variabili più importanti (influenza relativa)",
       subtitle = sprintf("Latitudine + longitudine pesano insieme circa %.0f%% del
  ↪ totale",
                           peso_posizione_gbm),
       x = NULL, y = "Influenza relativa (%)") +
  tema_slide +
  theme(
    axis.text.y = element_text(size = 10, hjust = 1),
    axis.text.x = element_text(size = 10, hjust = 1),
    plot.margin = margin(t = 10, r = 15, b = 10, l = 30, unit = "pt"),
    plot.title = element_text(size = 14),
    plot.subtitle = element_text(size = 11)
  )

print(g_ri)
```

GBM: variabili più importanti (influenza relativa)

Latitudine + longitudine pesano insieme circa 18% del totale



```
# =====
# 2. Dipendenza parziale: POSIZIONE (Latitudine e Longitudine)
# =====
if (modalita_output == "report") {
  pdp_pos <- rbind(
    pdp_modello(pred_gbm_base, dati_stima_gbm, "latitude"),
    pdp_modello(pred_gbm_base, dati_stima_gbm, "longitude")
  )
  g_pdp_pos <- ggplot(pdp_pos, aes(x, y)) +
    geom_line(colour = col_arancio, linewidth = 1.1) +
    facet_wrap(~ variabile, scales = "free_x") +
    labs(title = "GBM: dipendenza parziale dalla posizione",
         subtitle = "Il prezzo base stimato varia con continuita' lungo latitudine e
           ↳ longitudine",
         x = "Coordinata", y = "Effetto sul prezzo (EUR)") +
    tema_slide
  print(g_pdp_pos)
} else if (modalita_output == "slide") {
  # Slide: Latitudine singola
  pdp_lat <- pdp_modello(pred_gbm_base, dati_stima_gbm, "latitude")
  g_lat <- ggplot(pdp_lat, aes(x, y)) +
    geom_line(colour = col_arancio, linewidth = 1.1) +
    labs(title = "PDP - Latitudine",
         x = "Latitudine", y = "Effetto sul prezzo (EUR)") +
    tema_slide
  print(g_lat)

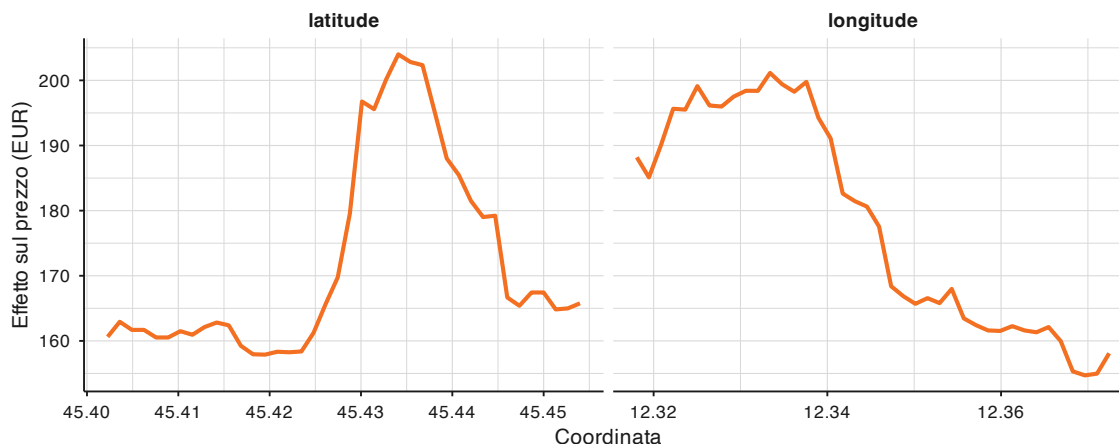
  # Slide: Longitudine singola
  pdp_lon <- pdp_modello(pred_gbm_base, dati_stima_gbm, "longitude")
  g_lon <- ggplot(pdp_lon, aes(x, y)) +
    geom_line(colour = col_arancio, linewidth = 1.1) +
    labs(title = "PDP - Longitudine",
         x = "Longitudine", y = "Effetto sul prezzo (EUR)") +
    tema_slide

  print(g_lon)
}
```

```
}
```

GBM: dipendenza parziale dalla posizione

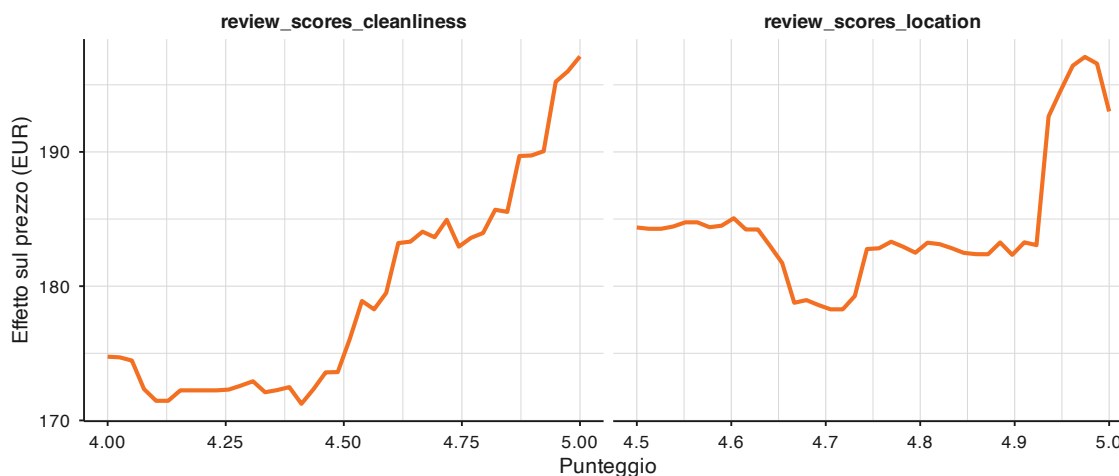
Il prezzo base stimato varia con continuit  lungo latitudine e longitudine



```
# =====  
# 3. Dipendenza parziale: RECENSIONI (Pulizia e Posizione)  
# =====  
if (modalita_output == "report") {  
  pdp_rec <- rbind(  
    pdp_modello(pred_gbm_base, dati_stima_gbm, "review_scores_cleanliness"),  
    pdp_modello(pred_gbm_base, dati_stima_gbm, "review_scores_location")  
  )  
  g_pdp_rec <- ggplot(pdp_rec, aes(x, y)) +  
    geom_line(colour = col_arancio, linewidth = 1.1) +  
    facet_wrap(~ variabile, scales = "free_x") +  
    labs(title = "PDP - Score Recensione: Categorie Pulizia e Posizione",  
         x = "Punteggio", y = "Effetto sul prezzo (EUR)") +  
    tema_slide  
  print(g_pdp_rec)  
}  
else if (modalita_output == "slide") {  
  # Slide: Pulizia singola  
  pdp_clean <- pdp_modello(pred_gbm_base, dati_stima_gbm, "review_scores_cleanliness")  
  g_clean <- ggplot(pdp_clean, aes(x, y)) +  
    geom_line(colour = col_arancio, linewidth = 1.1) +  
    labs(title = "PDP - Punteggio Pulizia",  
         x = "Score Pulizia", y = "Effetto sul prezzo (EUR)") +  
    tema_slide  
  print(g_clean)  
  
  # Slide: Posizione (Score) singola  
  pdp_loc_score <- pdp_modello(pred_gbm_base, dati_stima_gbm, "review_scores_location")  
  g_loc_score <- ggplot(pdp_loc_score, aes(x, y)) +  
    geom_line(colour = col_arancio, linewidth = 1.1) +  
    labs(title = "PDP - Punteggio Posizione",  
         x = "Score Posizione", y = "Effetto sul prezzo (EUR)") +  
    tema_slide  
  print(g_loc_score)
```

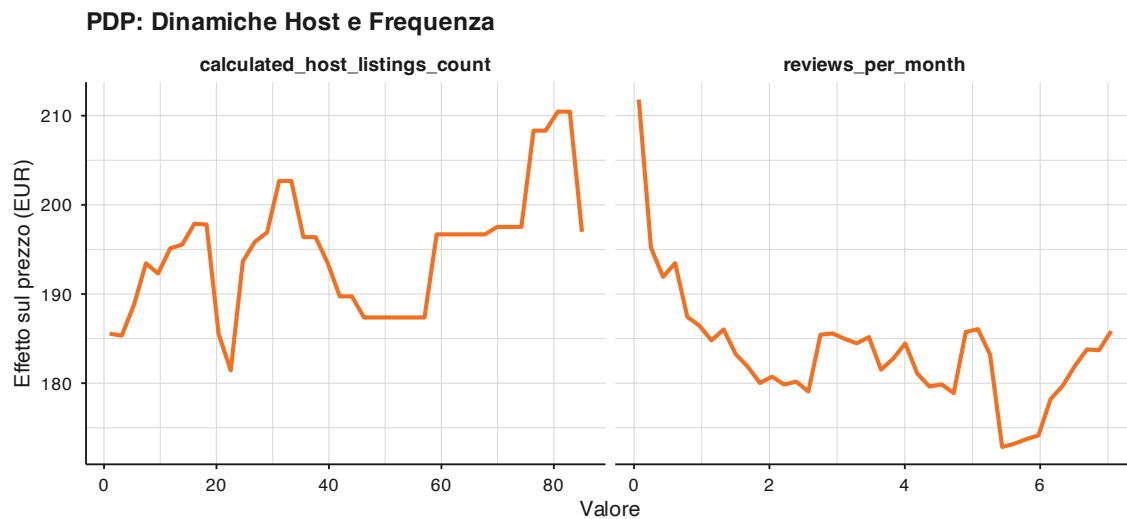
```
}
```

PDP - Score Recensione: Categorie Pulizia e Posizione



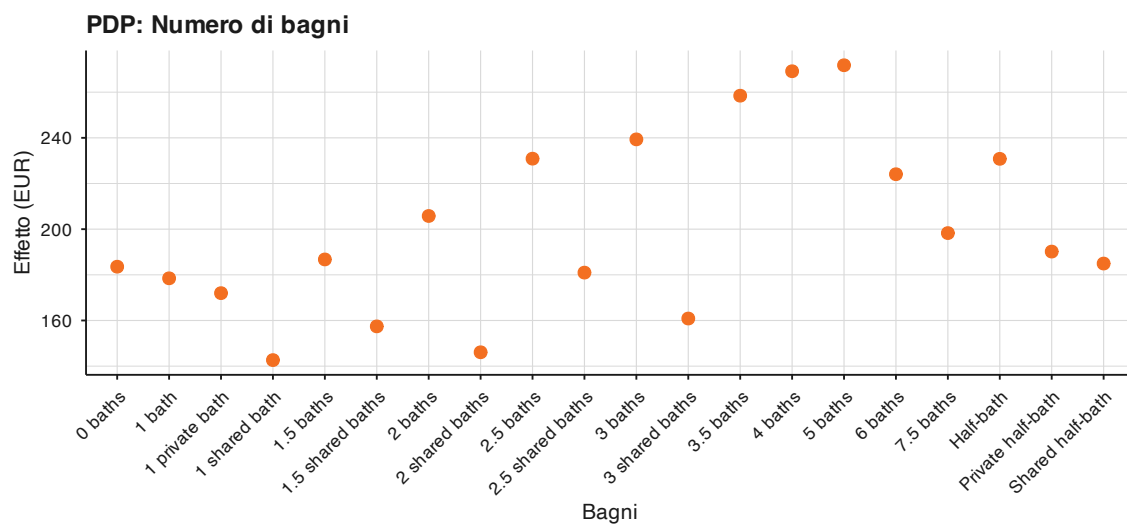
```
# =====  
# 4. Dipendenza parziale: FREQUENZA E DISPONIBILITÀ (Host count & Recensioni/mese)  
# =====  
if (modalita_output == "report") {  
  pdp_disp <- rbind(  
    pdp_modello(pred_gbm_base, dati_stima_gbm, "reviews_per_month"),  
    pdp_modello(pred_gbm_base, dati_stima_gbm, "calculated_host_listings_count")  
  )  
  g_pdp_disp <- ggplot(pdp_disp, aes(x, y)) +  
    geom_line(colour = col_arancio, linewidth = 1.1) +  
    facet_wrap(~ variabile, scales = "free_x") +  
    labs(title = "PDP: Dinamiche Host e Frequenza",  
         x = "Valore", y = "Effetto sul prezzo (EUR)") +  
    tema_slide  
  print(g_pdp_disp)  
}  
else if (modalita_output == "slide") {  
  # Slide: Recensioni per mese singola  
  pdp_rev_month <- pdp_modello(pred_gbm_base, dati_stima_gbm, "reviews_per_month")  
  g_rev_month <- ggplot(pdp_rev_month, aes(x, y)) +  
    geom_line(colour = col_arancio, linewidth = 1.1) +  
    labs(title = "PDP - Recensioni per mese",  
         x = "Recensioni / mese", y = "Effetto sul prezzo (EUR)") +  
    tema_slide  
  print(g_rev_month)  
  
  # Slide: Annunci dell'Host singola  
  pdp_host_list <- pdp_modello(pred_gbm_base, dati_stima_gbm,  
    ↪ "calculated_host_listings_count")  
  g_host_list <- ggplot(pdp_host_list, aes(x, y)) +  
    geom_line(colour = col_arancio, linewidth = 1.1) +  
    labs(title = "PDP - Numero di annunci dell'Host",  
         x = "Annunci totali Host", y = "Effetto sul prezzo (EUR)") +  
    tema_slide
```

```
print(g_host_list)
}
```



```
# =====
# 5. Dipendenza parziale: BAGNI (Rimane singolo in entrambe le modalità)
# =====
pdp_bath <- pdp_modello(pred_gbm_base, dati_stima_gbm, "bathrooms_clean")
g_pdp_bath <- ggplot(pdp_bath, aes(x, y)) +
  geom_point(colour = col_arancio, size = 3) + # geom_point per le categorie!
  labs(title = "PDP: Numero di bagni",
       x = "Bagni", y = "Effetto (EUR)") +
  tema_slide +
  theme(axis.text.x = element_text(angle = 45, hjust = 1))

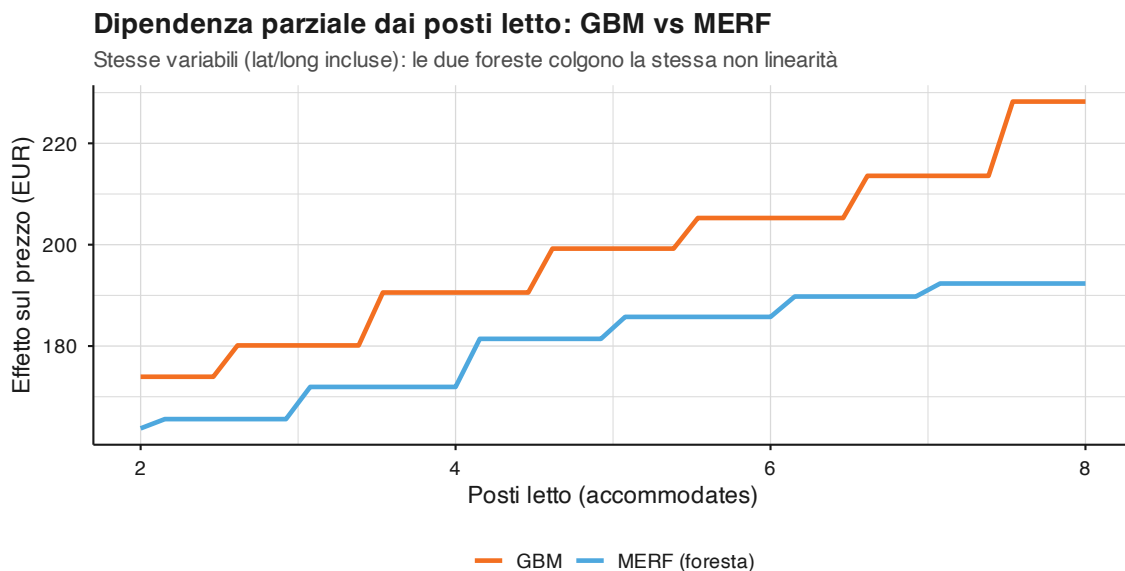
print(g_pdp_bath)
```



Nel GBM la **posizione domina**: latitudine e longitudine pesano insieme circa 18% dell'influenza relativa totale, più di qualunque altra variabile presa singolarmente. La dipendenza parziale spiega perché: il prezzo

stimato segue un **gradiente spaziale continuo** lungo le due coordinate. Gli alberi approssimano questa superficie di prezzo con una sequenza di split su lat/long — qualcosa che un'intercetta per poche zone non può riprodurre.

```
# Posti letto: PDP a confronto tra GBM e parte forest del MERF (entrambi ora con
# lat/long tra i predittori). La foresta del MERF predice in log -> exp() per i €.
pred_merf_fix <- function(d) {
  exp(predict(m_merf_final$RandomForest, newdata = d, na.action = "na.impute")$predicted)
}
pdp_conf <- rbind(
  cbind(pdp_modello(pred_gbm_base, dati_stima_gbm, "accommodates"), modello = "GBM"),
  cbind(pdp_modello(pred_merf_fix, dati_stima_na, "accommodates"), modello = "MERF
    ↪ (foresta)")
)
g_pdp_conf <- ggplot(pdp_conf, aes(x, y, colour = modello)) +
  geom_line(linewidth = 1.1) +
  scale_colour_manual(values = c("GBM" = col_arancio, "MERF (foresta)" = col_azzurro)) +
  labs(title = "Dipendenza parziale dai posti letto: GBM vs MERF",
    subtitle = "Stesse variabili (lat/long incluse): le due foreste colgono la stessa
    ↪ non linearità",
    x = "Posti letto (accommodates)", y = "Effetto sul prezzo (EUR)", colour = NULL) +
  tema_slide
print(g_pdp_conf)
```



Sui posti letto — variabile strutturale — GBM e foresta del MERF producono curve di dipendenza parziale molto simili: colgono le stesse non linearità. Anche il MERF include lat/long, il divario residuo coi modelli ad albero non dipende dalla posizione mancante, ma dal modo in cui la posizione viene combinata con le altre variabili: interazioni piene negli alberi, struttura additiva più intercetta di gruppo nei misti.

```
# Componenti di varianza dell'effetto casuale per zona_tipo: tra-zone vs residua
componenti_var <- function(modello_lmer) {
  vc <- as.data.frame(VarCorr(modello_lmer))
  v_grp <- vc$vcov[vc$grp != "Residual"][1]
  v_res <- vc$vcov[vc$grp == "Residual"]
}
```

```

data.frame(var_gruppo = v_grp, var_resid = v_res,
           sd_gruppo = sqrt(v_grp), icc = v_grp / (v_grp + v_res))
}
cv_lmm <- componenti_var(m_lmer_opt)
cv_merf <- tryCatch(componenti_var(m_merf_final$EffectModel), error = function(e) NULL)

tab_var <- data.frame(Modello = "LMM (gerarchico)", cv_lmm)
if (!is.null(cv_merf)) tab_var <- rbind(tab_var, data.frame(Modello = "MERF (parte
  ↳ mista)", cv_merf))

icc_lmm_val <- cv_lmm$icc

# La tabella viene stampata in entrambe le modalità
tab_var %>%
  transmute(Modello,
            `Var. tra zone (log)` = round(var_gruppo, 3),
            `Var. residua (log)` = round(var_resid, 3),
            `SD tra zone (log)` = round(sd_gruppo, 3),
            `Effetto +/- % prezzo` = round((exp(sd_gruppo) - 1) * 100, 1),
            ICC = round(icc, 3)) %>%
  knitr::kable(caption = "Componenti di varianza dell'effetto casuale per zona_tipo
  ↳ (risposta in log-prezzo). 'Effetto % prezzo' = traduzione moltiplicativa della SD tra
  ↳ zone, exp(SD)-1. ICC = quota di variabilit  (log) dovuta alle differenze TRA zone.")

```

Table 4: Componenti di varianza dell'effetto casuale per zona_tipo (risposta in log-prezzo). 'Effetto % prezzo' = traduzione moltiplicativa della SD tra zone, exp(SD)-1. ICC = quota di variabilit  (log) dovuta alle differenze TRA zone.

Modello	Var. tra zone (log)	Var. residua (log)	SD tra zone (log)	Effetto +/- % prezzo	ICC
LMM (gerarchico)	0.032	0.101	0.179	19.6	0.240
MERF (parte mista)	0.017	0.056	0.130	13.8	0.232

```

# Caterpillar plot: intercette casuali per zona (LMM) con intervalli a +/- 1.96 SE
re_lmm <- ranef(m_lmer_opt, condVar = TRUE)$zona_tipo
se_lmm <- sqrt(as.numeric(attr(re_lmm, "postVar"))))
re_df <- data.frame(zona = rownames(re_lmm), eff = re_lmm[, 1], se = se_lmm)
re_df <- re_df[order(re_df$eff), ]
re_df$zona <- factor(re_df$zona, levels = re_df$zona)
n_zone <- nrow(re_df)

# Creazione del grafico base
g_cat <- ggplot(re_df, aes(eff, zona)) +
  geom_vline(xintercept = 0, linetype = 2, colour = col_grid) +
  geom_errorbarh(aes(xmin = eff - 1.96 * se, xmax = eff + 1.96 * se),
                height = 0.25, colour = col_azzurro) +
  geom_point(colour = col_arancio, size = 2.6) +
  labs(title = "LMM: effetti casuali (intercette) per zona",
       subtitle = sprintf("%d zone | SD tra zone = %.2f (log) ~ +/-%.0f%% prezzo | ICC =
  ↳ %.2f",

```

```

        n_zone, cv_lmm$sd_gruppo, (exp(cv_lmm$sd_gruppo) - 1) * 100,
  ↪   cv_lmm$icc),
      x = "Scostamento (log-prezzo)", y = NULL) +
  tema_slide

# =====
# Controllo Condizionale Output
# =====
if (modalita_output == "report") {

  print(g_cat)

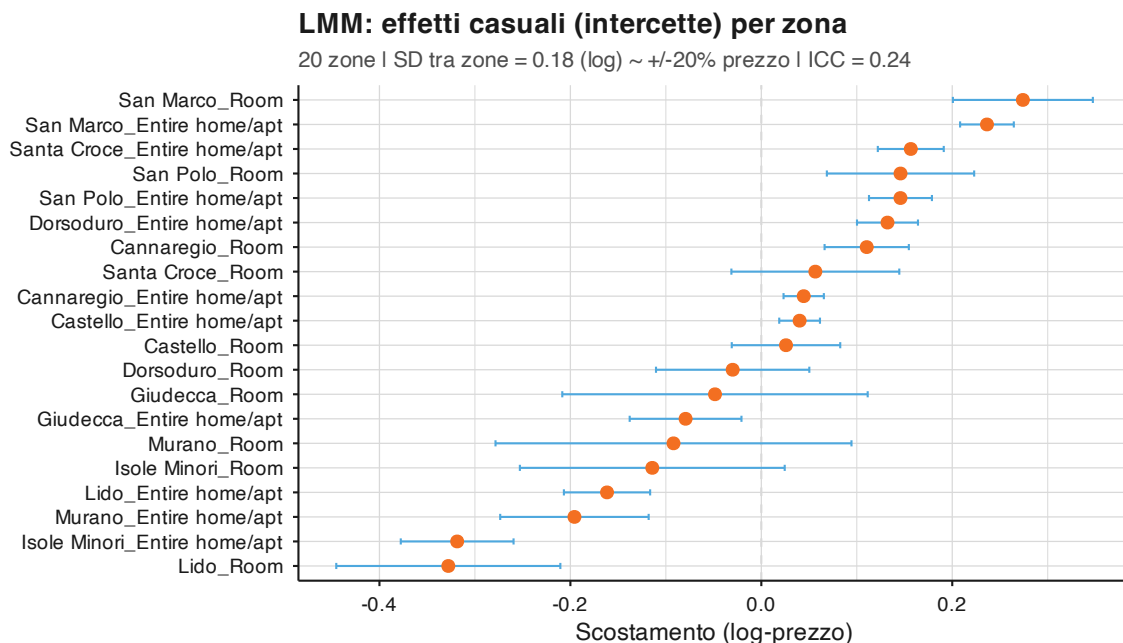
} else if (modalita_output == "slide") {

  # Aggiungiamo i margini maggiorati a sinistra e riduciamo il testo per far
  # entrare comodamente tutti i nomi delle zone veneziane e il titolo nella slide.
  g_cat_slide <- g_cat +
    theme(
      axis.text.y = element_text(size = 10, hjust = 1),
      plot.margin = margin(t = 10, r = 15, b = 10, l = 35, unit = "pt"), # 'l' maggiorato
      plot.title = element_text(size = 14),
      plot.subtitle = element_text(size = 11)
    )

  print(g_cat_slide)

}

```



L'effetto casuale per `zona_tipo` mantiene un ICC pari a circa 0.24 nel LMM.

ESTRAZIONE PESI STAGIONALITÀ (Dataset 300 Appartamenti Storici)

Come si ricavano i pesi di stagionalità (La Teoria)

Il nostro obiettivo è estrarre l'effetto “puro” del mese sul prezzo, isolandolo dalla qualità dell'appartamento. Per farlo, usiamo il dataset dei 300 appartamenti storici stimando un **Modello Lineare Misto (LMM)** senza intercetta globale.

L'equazione che il modello stimerà è:

$$Prezzo_{i,t} = \beta_{mese_t} + u_i + \epsilon_{i,t}$$

Dove:

- β_{mese_t} : è l'effetto fisso specifico del mese t (stimato come media assoluta per quel mese, grazie alla soppressione dell'intercetta globale).
- u_i : è l'effetto casuale (random intercept) dell'appartamento i (`listing_id`). Questo “assorbe” tutta la qualità fisica, la dimensione e la posizione geografica della singola casa, mantenendola costante lungo i 12 mesi.
- $\epsilon_{i,t}$: è l'errore residuo (variazione non spiegata per l'appartamento i nel mese t).

I 3 Passaggi per calcolare il Moltiplicatore:

1. **Stima delle Medie Marginali:** Grazie al pacchetto `emmeans`, estraiamo direttamente i coefficienti β_{mese_t} dal modello, che rappresentano il prezzo medio atteso per ogni mese dell'anno *depurato dall'effetto specifico della casa*.
2. **Creazione dell'Indice (Base = Settembre):** Poiché il dataset principale ha 6000 appartamenti fa riferimento al 15 Settembre, Settembre diventa il nostro mese “1.0”. Dividiamo il prezzo medio marginale di ogni mese per quello di Settembre.

$$Indice_t = \frac{\beta_{mese_t}}{\beta_{Settembre}}$$

3. **Applicazione:** Se l'indice di Agosto è 1.25 e il modello base cross-sectional stima per un appartamento un prezzo di 100€ a Settembre, il suo prezzo destagionalizzato per Agosto sarà $100 \times 1.25 = 125\text{€}$.

```
# CARICA QUI IL DATASET DA 300 (con 12 mesi)
# dati_storici <- read_parquet("../dati/storico_300.parquet")

airroi_pulito_merge = readRDS("../dati/airroi_pulito_merge.rds")

# [Simulazione per far girare lo script]
dati_storici <- airroi_pulito_merge %>%
  rename(y = rate_avg) %>%
  mutate(month = as.factor(month))

# Stimo il modello misto: Il prezzo dipende dal mese, ma isolo l'effetto
#   dell'appartamento
mod_stagionale <- lmer(y ~ -1+month + (1|listing_id), data = dati_storici)
```

```

# Calcolo le Medie Marginali Stimate per ogni mese
medie_mensili <- emmeans(mod_stagionale, "month")
df_mensile <- as.data.frame(medie_mensili)

# Setto il mese di Settembre (es. month == 9) come Base = 1.0
prezzo_base_settembre <- df_mensile$emmean[df_mensile$month == 9]

df_pesi <- df_mensile %>%
  mutate(Moltiplicatore = emmean / prezzo_base_settembre) %>%
  select(month, Moltiplicatore)

kable(df_pesi, digits = 3, caption = "Indici di Stagionalità")

```

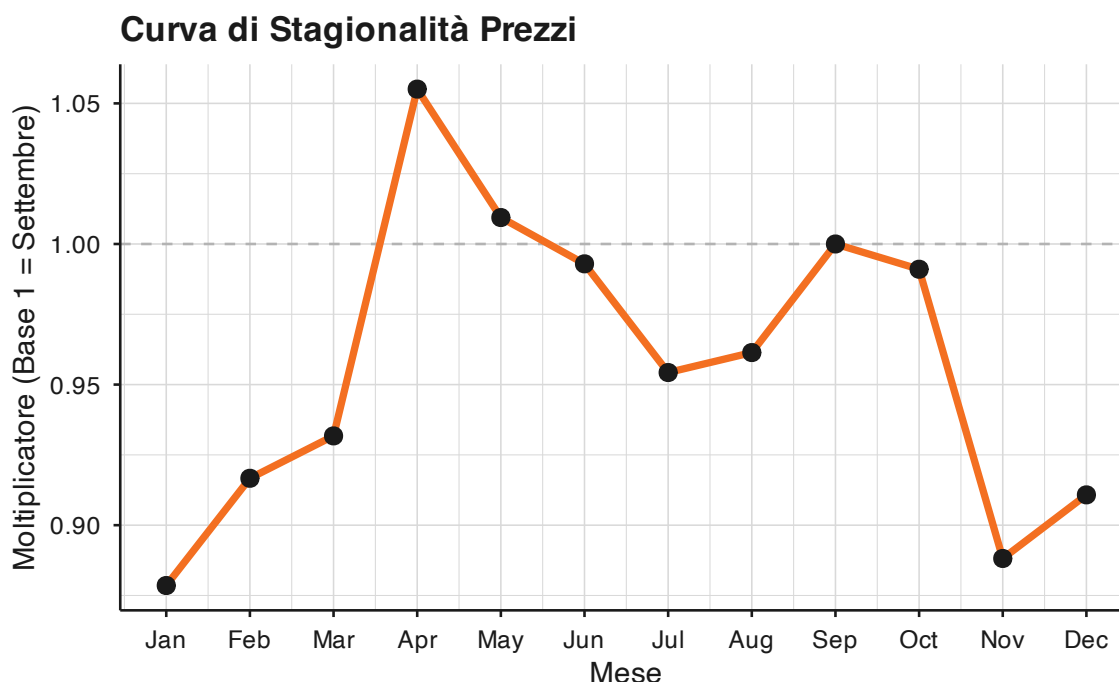
Table 5: Indici di Stagionalità

month	Moltiplicatore
1	0.879
2	0.917
3	0.932
4	1.055
5	1.009
6	0.993
7	0.954
8	0.961
9	1.000
10	0.991
11	0.888
12	0.911

```

# Grafico della Stagionalità
ggplot(df_pesi, aes(x = as.numeric(as.character(month)), y = Moltiplicatore)) +
  geom_hline(yintercept = 1, linetype = "dashed", color = "grey70") +
  geom_line(color = col_arancio, linewidth = 1.4) +
  geom_point(color = col_bianco, size = 3) +
  scale_x_continuous(breaks = 1:12, labels = month.abb) +
  labs(title = "Curva di Stagionalità Prezzi",
       x = "Mese", y = "Moltiplicatore (Base 1 = Settembre)") +
  tema_slide

```



Il lato Business (Perché i numeri “raccontano” la giusta storia)

I moltiplicatori che abbiamo ottenuto disegnano una curva di stagionalità che per una città d’arte come Venezia è estremamente realistica:

- **Il picco di Aprile (1.055):** È il mese più forte (+5.5% rispetto a settembre). Ha perfettamente senso: Pasqua, i ponti del 25 aprile e 1 maggio, e il clima primaverile rendono Venezia ambitissima, spingendo i prezzi al massimo.
- **L’estate “calante” (Luglio 0.954, Agosto 0.961):** Potrebbe sembrare controintuitivo che agosto costi meno di aprile, ma per Venezia è normale. L’estate veneziana è caldissima, umida e piena di zanzare. Attrae un turismo più “di massa” e pendolare (che non pernotta), mentre chi ha alto budget spesso preferisce altre mete o la primavera/autunno.
- **Il mese base forte (Settembre 1.000, Maggio 1.009):** Maggio e settembre sono mesi d’oro. Clima eccellente e, a settembre, il traino del Festival del Cinema e spesso della Biennale mantengono i prezzi alti.
- **La bassa stagione (Novembre 0.888, Gennaio 0.879):** Sono i mesi più freddi e soggetti all’acqua alta, ed è corretto che i prezzi scendano dell’11-12% rispetto a settembre.
- **I “rimbalzi” invernali (Febbraio 0.917, Dicembre 0.911):** Si nota benissimo l’effetto del Carnevale a febbraio (che alza i prezzi rispetto a gennaio) e l’effetto di Natale/Capodanno a dicembre (che alza i prezzi rispetto al morto novembre).

Segmentazione

Selezione delle Variabili e Standardizzazione

Il primo passo per la segmentazione del mercato consiste nell’isolare le variabili chiave che definiscono l’offerta: la **posizione geografica** (latitudine e longitudine), il **prezzo** e la **capacità ricettiva** (numero di ospiti).

Un passaggio matematico fondamentale in questa fase è la **standardizzazione dei dati**. Poiché il prezzo varia su una scala di centinaia di euro mentre gli ospiti si misurano in singole unità, omettere questo passaggio farebbe sì che il calcolo della distanza euclidea venga dominato quasi esclusivamente dalla varianza del prezzo, oscurando l'impatto delle altre variabili.

```
dati_cluster <- airbnb_pulito %>%
  select(latitude, longitude, price_num, accommodates, id) %>%
  na.omit()

dati_scalati <- scale(dati_cluster[, c(1:4)])
```

Ricerca del Numero Ottimale di Gruppi (Metodo del Gomito)

Per individuare la corretta partizione del mercato, il modello stima diverse soluzioni (da 2 a 8 gruppi), eseguendo 20 inizializzazioni casuali per ciascuna così da garantire la stabilità del risultato.

Il grafico a gomito (Elbow Plot) sottostante traccia la somma delle distanze interne ai cluster: cerchiamo il punto in cui l'aggiunta di un ulteriore gruppo smette di ridurre la varianza interna in modo significativo.

```
set.seed(1234)
modello_step <- stepcclust(dati_scalati, k = 2:8, nrep = 20)
```

```
## 2 : * * * * *
## 3 : * * * * *
## 4 : * * * * *
## 5 : * * * * *
## 6 : * * * * *
## 7 : * * * * *
## 8 : * * * * *
```

Analisi e Scelta della Soluzione Ottimale (K = 3)

L'estrazione della soluzione a 3 gruppi è supportata da evidenze matematiche, di robustezza e interpretative:

1. **Robustezza.** La partizione a 3 gruppi è confermata in modo indipendente dal clustering gerarchico con metodo di Ward (cfr. dendrogramma e matrice di consistenza più sotto): due algoritmi con logiche diverse concordano, segno che i segmenti sono pattern reali di mercato e non un artefatto del K-means.
2. **Evidenza interpretativa.** Soprattutto, la soluzione a 3 gruppi **isola con nettezza il segmento geografico Lido–Castello–Isole**, cioè proprio l'arena competitiva della struttura e l'epicentro della Mostra. Per gli scopi di questo lavoro è il taglio più efficace: una suddivisione più fine del nucleo storico aumenterebbe la complessità senza modificare l'analisi del rincaro, che ruota attorno al segmento dell'evento.

Poiché la funzione originaria restituisce un oggetto tecnico, i profili vengono ricostruiti partendo dai centri standardizzati. In questo grafico, **lo zero rappresenta la media cittadina**: un valore positivo indica che quel segmento si posiziona sopra la media di Venezia per quella specifica caratteristica, mentre un valore negativo ne indica un posizionamento inferiore.

```
modello_scelto <- getModel(modello_step, "3")
centri <- modello_scelto@centers
colnames(centri) <- c("latitudine", "longitudine", "prezzo", "ospiti")
```

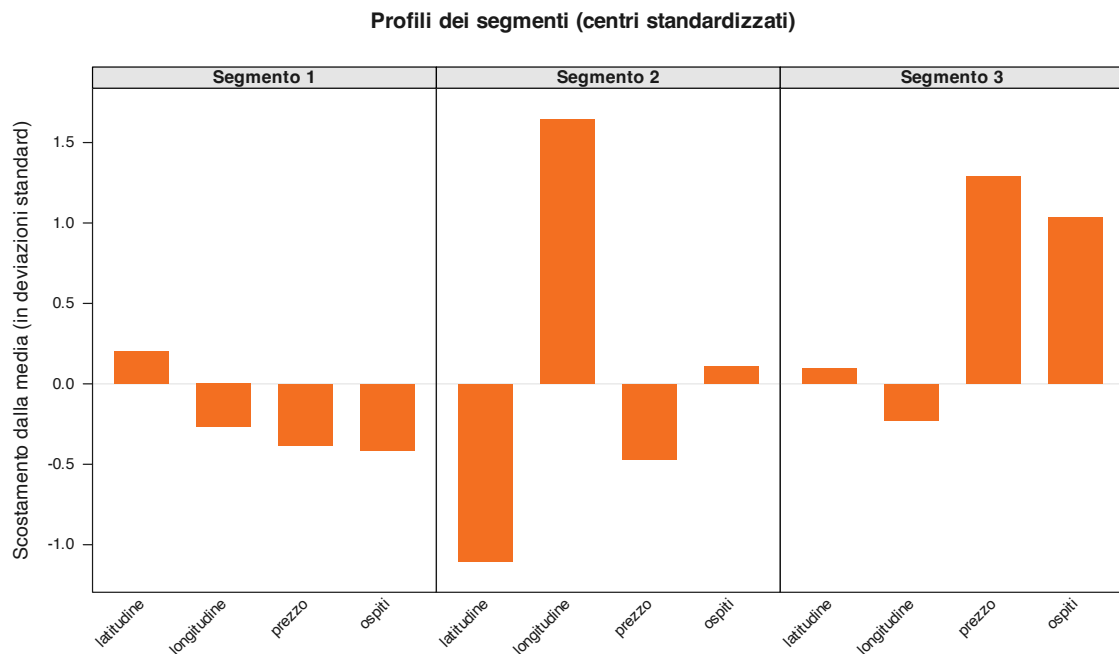
```

rownames(centri) <- paste("Segmento", seq_len(nrow(centri)))

centri_df <- as.data.frame(as.table(as.matrix(centri)))
names(centri_df) <- c("Segmento", "Variabile", "Valore")

barchart(Valore ~ Variabile | Segmento, data = centri_df,
  origin = 0, col = col_arancio, border = "transparent", horizontal = FALSE,
  main = "Profili dei segmenti (centri standardizzati)",
  ylab = "Scostamento dalla media (in deviazioni standard)",
  scales = list(x = list(rot = 45), col = col_testo),
  par.settings = pset_slide,
  par.strip.text = list(col = col_testo, font = 2, cex = if (is_report) 0.9 else
    ↪ 1.4),
  layout = c(nrow(centri), 1))

```



Ricadute Economiche e Distribuzione Geografica

Riportando le assegnazioni di cluster sul dataset originale, è possibile calcolare le statistiche descrittive in scala reale (Euro e numero effettivo di ospiti) e osservare come i tre segmenti si distribuiscono all'interno dei sestieri veneziani.

```

dati_cluster$gruppo_competitor <- as.factor(clusters(modello_scelto))

analisi_competitor <- dati_cluster %>%
  group_by(gruppo_competitor) %>%
  summarise(
    n_annunci = n(),
    prezzo_medio_reale = mean(price_num),
    prezzo_mediano = median(price_num),
    ospiti_medi = mean(accommodates)
  )

```



```
print(analisi_competitor)
```

```
## # A tibble: 3 x 5
##   gruppo_competitor n_annunci prezzo_medio_reale prezzo_mediano ospiti_medi
##   <fct>             <int>           <dbl>           <dbl>           <dbl>
## 1 1                 3630           162.            156            3.05
## 2 2                 781           154.            141            3.89
## 3 3                1365           324.            304            5.40
```

```
dati_completi <- airbnb_pulito %>%
  inner_join(dati_cluster %>% select(id, gruppo_competitor), by = "id")

tab_quartieri <- table(dati_completi$gruppo_competitor,
  ↪ dati_completi$neighbourhood_cleansed)

tab_quartieri_df <- as.data.frame.matrix(tab_quartieri)
rownames(tab_quartieri_df) <- paste("Segmento", rownames(tab_quartieri_df))

tabella_segmenti_sestiere <- knitr::kable(
  tab_quartieri_df, booktabs = TRUE,
  caption = "Distribuzione degli annunci per segmento (righe) e sestiere (colonne).")
  ↪ |>
  kableExtra::kable_styling(
    latex_options = c("scale_down", "HOLD_position"),
    font_size = 8 # Abbassa questo numero finché non entra nella pagina (es. 7 o 6)
  )
```

```
## Warning in styling_latex_scale(out, table_info, "down"): Longtable cannot be
## resized.
```

```
tabella_segmenti_sestiere
```

Table 6: Distribuzione degli annunci per segmento (righe) e sestiere (colonne).

	Cannaregio	Castello	Dorsoduro	Giudecca	Lido	Murano	San Marco	San Polo	Santa Croce	Isole Minori
Segmento 1	1121	698	374	130	0	74	472	378	379	4
Segmento 2	0	367	0	0	269	0	0	0	0	145
Segmento 3	294	263	182	28	8	10	288	161	124	7

```
# =====
# TABELLA ANALISI DEI PREZZI (MEDIA E MEDIANA) PER GRUPPO COMPETITOR
# =====

# 1. Aggregazione dei dati per cluster (gruppo_competitor)
tabella_prezzi_cluster_df <- dati_completi %>%
  group_by(Segmento = gruppo_competitor) %>%
  summarise(
    `Numero Annunci` = n(),
    `Prezzo Medio` = mean(price_num, na.rm = TRUE),
```

```

`Prezzo Mediano` = median(price_num, na.rm = TRUE),
`Ospiti Medi`    = mean(accommodates, na.rm = TRUE)
) %>%
# Ordiniamo i segmenti dal più caro al meno caro
arrange(desc(`Prezzo Medio`))

# Rinominiamo i valori della colonna per una resa più elegante in tabella
tabella_prezzi_cluster_df$Segmento <- paste("Segmento",
  ↪ tabella_prezzi_cluster_df$Segmento)

# 2. Formattazione elegante con kable per il report PDF
tabella_prezzi_cluster_output <- knitr::kable(
  tabella_prezzi_cluster_df,
  booktabs = TRUE,
  digits = 1, # Arrotonda a un decimale
  caption = "Caratterizzazione economica dei segmenti: analisi comparativa di prezzo
  ↪ (Media e Mediana) e dimensione per Cluster."
) %>%
kableExtra::kable_styling(
  latex_options = c("striped", "HOLD_position"),
  font_size = 10
)

# Visualizza la tabella nel report
tabella_prezzi_cluster_output

```

Table 7: Caratterizzazione economica dei segmenti: analisi comparativa di prezzo (Media e Mediana) e dimensione per Cluster.

Segmento	Numero Annunci	Prezzo Medio	Prezzo Mediano	Ospiti Medi
Segmento 3	1365	324.2	304	5.4
Segmento 1	3630	162.0	156	3.1
Segmento 2	781	153.6	141	3.9

```

# Segmento di riferimento dell'Hotel = quello con la massima concentrazione di
# annunci al Lido (area della Mostra). Identificato in modo ROBUSTO (non per
# etichetta fissa, che dipende dall'inizializzazione casuale del K-means).
ref_cluster <- names(which.max(tab_quartieri[, "Lido"]))
stats_ref <- analisi_competitor %>% filter(gruppo_competitor == ref_cluster)
n_ref <- stats_ref$n_annunci
prezzo_med_ref <- round(stats_ref$prezzo_medio_reale)
prezzo_mediano_ref <- round(stats_ref$prezzo_mediano)
ospiti_ref <- round(stats_ref$ospiti_medi, 1)
lido_ref <- tab_quartieri[ref_cluster, "Lido"]
cast_ref <- tab_quartieri[ref_cluster, "Castello"]
isole_ref <- if ("Isole Minori" %in% colnames(tab_quartieri))
  ↪ tab_quartieri[ref_cluster, "Isole Minori"] else 0L

# I due segmenti NON di riferimento, ordinati per prezzo: il piu' caro e' la
# fascia alta/grande capacita', l'altro il mercato economico di base.
altri_cl <- analisi_competitor %>% filter(gruppo_competitor != ref_cluster) %>%
  ↪ arrange(desc(prezzo_medio_reale))

```

```

id_premium      <- as.character(altri_cl$gruppo_competitor[1])
n_premium       <- altri_cl$n_annunci[1]
prezzo_premium  <- round(altri_cl$prezzo_medio_reale[1])
ospiti_premium  <- round(altri_cl$ospiti_medi[1], 1)
id_base         <- as.character(altri_cl$gruppo_competitor[2])
n_base          <- altri_cl$n_annunci[2]
prezzo_base_seg <- round(altri_cl$prezzo_medio_reale[2])
ospiti_base     <- round(altri_cl$ospiti_medi[2], 1)

```

```

suppressPackageStartupMessages({ library(sf); library(ggspatial) })
# =====
# 1. IMPOSTAZIONE DIMENSIONI IN BASE ALL'OUTPUT (REPORT VS SLIDE)
# =====
if (modalita_output == "slide") {
  dim_punti_centro <- 1.5
  dim_punti_lido   <- 1.8
  dim_legenda_tit  <- 14
  dim_legenda_txt  <- 12
  opacita_punti    <- 0.95
} else {
  dim_punti_centro <- 1.0
  dim_punti_lido   <- 1.2
  dim_titolo       <- 14
  dim_sottotitolo  <- 11
  dim_legenda_tit  <- 11
  dim_legenda_txt  <- 10
  opacita_punti    <- 0.95
}

# =====
# 2. CREAZIONE OGGETTO SPAZIALE E RITAGLI
# =====
airbnb_mappa_dati <- dati_completi %>% mutate(Segmento = as.factor(gruppo_competitor))
airbnb_sf <- st_as_sf(airbnb_mappa_dati, coords = c("longitude", "latitude"), crs = 4326)

box_venezia <- st_bbox(c(xmin = 12.300, ymin = 45.415, xmax = 12.365, ymax = 45.470),
  ↪ crs = st_crs(4326))
box_lido_nord <- st_bbox(c(xmin = 12.350, ymin = 45.400, xmax = 12.390, ymax = 45.425),
  ↪ crs = st_crs(4326))
box_lido_sud <- st_bbox(c(xmin = 12.300, ymin = 45.320, xmax = 12.350, ymax = 45.380),
  ↪ crs = st_crs(4326))

airbnb_venezia <- st_crop(airbnb_sf, box_venezia)
airbnb_lido_nord <- st_crop(airbnb_sf, box_lido_nord)
airbnb_lido_sud <- st_crop(airbnb_sf, box_lido_sud)

colori_cluster <- c("1" = "#F36F21", "2" = "#2EC27E", "3" = "blue")

# =====
# 3. COSTRUZIONE MAPPE (usando le variabili dinamiche "dim_punti")
# =====
g_venezia <- ggplot() +
  annotation_map_tile(type = "osm", zoomin = 1) +
  geom_sf(data = airbnb_venezia, aes(color = Segmento), size = dim_punti_centro, alpha =
    ↪ opacita_punti) +

```

```

    scale_color_manual(values = colori_cluster, name = "Segmento")

g_lido_nord <- ggplot() +
  annotation_map_tile(type = "osm", zoomin = 1) +
  geom_sf(data = airbnb_lido_nord, aes(color = Segmento), size = dim_punti_lido, alpha =
    ↪ opacita_punti) +
  scale_color_manual(values = colori_cluster)

g_lido_sud <- ggplot() +
  annotation_map_tile(type = "osm", zoomin = 1) +
  geom_sf(data = airbnb_lido_sud, aes(color = Segmento), size = dim_punti_lido, alpha =
    ↪ opacita_punti) +
  scale_color_manual(values = colori_cluster)

# =====
# 4. APPLICAZIONE DELLO STILE (usando le variabili dinamiche "dim_titolo", ecc.)
# =====
if (modalita_output == "report") {

  tema_base <- theme_void() + theme(
    plot.title = element_text(face = "bold", size = dim_titolo),
    plot.subtitle = element_text(size = dim_sottotitolo)
  )

  print(g_venezia + tema_base + theme(legend.position = "right"))
  print(g_lido_nord + tema_base + theme(legend.position = "none"))
  print(g_lido_sud + tema_base + theme(legend.position = "none"))

} else if (modalita_output == "slide") {

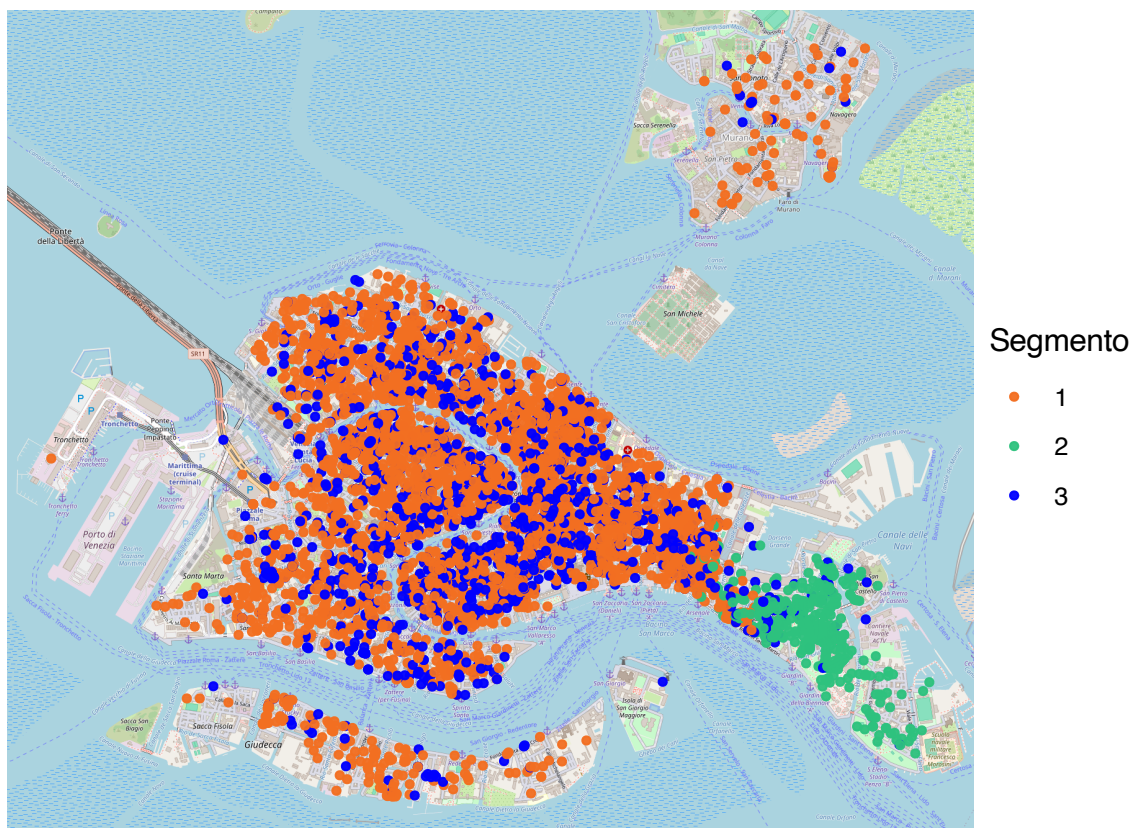
  tema_mappa_slide <- theme_void() + theme(
    plot.background = element_rect(fill = "black", color = NA),
    panel.background = element_rect(fill = "black", color = NA),
    plot.margin = margin(t = 10, r = 15, b = 10, l = 15, unit = "pt")
  )

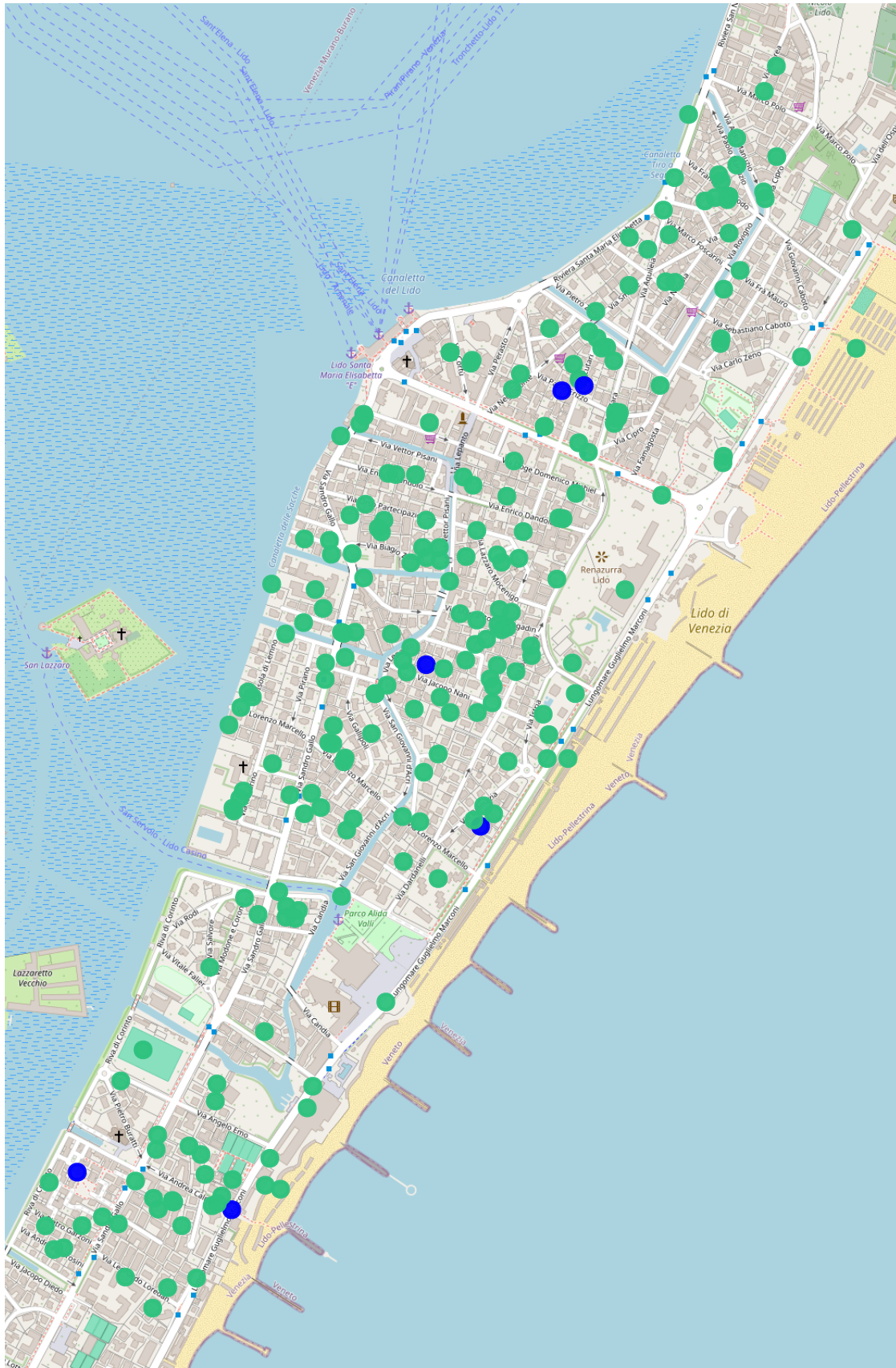
  mappa_1_slide <- g_venezia + tema_mappa_slide + theme(
    legend.position = "right",
    legend.text = element_text(color = "white", size = dim_legenda_txt),
    legend.title = element_text(color = "white", face = "bold", size = dim_legenda_tit)
  )

  mappa_2_slide <- g_lido_nord + tema_mappa_slide + theme(legend.position = "none")
  mappa_3_slide <- g_lido_sud + tema_mappa_slide + theme(legend.position = "none")

  print(mappa_1_slide)
  print(mappa_2_slide)
  print(mappa_3_slide)
}

```





Validazione della Robustezza: Clustering Gerarchico

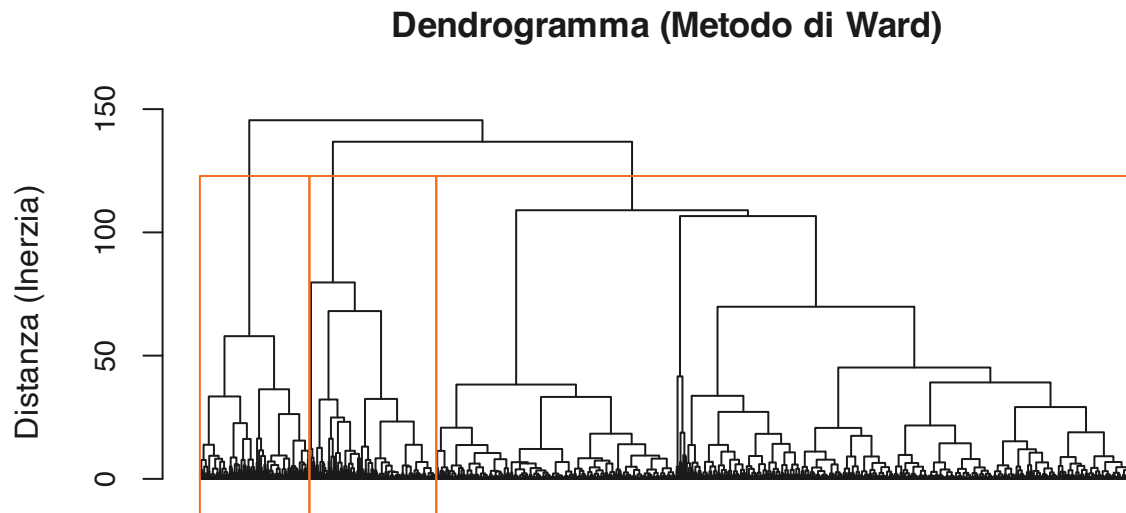
Per confermare che i segmenti individuati rappresentino reali dinamiche di mercato e non una forzatura matematica del K-means, si procede con una validazione incrociata utilizzando il **Clustering Gerarchico**.

La matrice delle distanze viene calcolata utilizzando la **Distanza manhattan** sulle variabili standardizzate. Per l'aggregazione si opta per il **Metodo di Ward**, il cui obiettivo è minimizzare la varianza interna (devianza) ad ogni fusione: una logica affine a quella del K-means, che rende il confronto tra i due output metodologicamente rigoroso.

```
# 1. Calcolo della matrice delle distanze (manhattan sui dati standardizzati,
#     coerente con il criterio di Ward che minimizza la varianza)
matrice_distanze <- dist(dati_scalati, method = "manhattan")

# 2. Applicazione del clustering gerarchico (Metodo di Ward)
modello_hc <- hclust(matrice_distanze, method = "ward.D2")

# 3. Visualizzazione del Dendrogramma (stampato nel report)
if (exists("modalita_output") && modalita_output == "report") {
  par_slide()
  plot(modello_hc, labels = FALSE, hang = -1,
       main = "Dendrogramma (Metodo di Ward)",
       sub = "Il salto di varianza conferma visivamente il taglio a 3 gruppi",
       xlab = "Strutture", ylab = "Distanza (Inerzia)")
  # Rettangoli attorno ai 3 gruppi
  rect.hclust(modello_hc, k = 3, border = "#F36F21")
}
```



Strutture
Il salto di varianza conferma visivamente il taglio a 3 gruppi

```
# 4. Taglio dell'albero a 3 gruppi
dati_cluster$gruppo_hc <- as.factor(cutree(modello_hc, k = 3))
```



```
# 5. Controllo di consistenza (matrice di confronto tra algoritmi).
# Se ogni gruppo K-means corrisponde a un distinto gruppo gerarchico dominante,
# i due metodi hanno individuato gli stessi macro-pattern.
tabella_confronto <- table(K_Means = dati_cluster$gruppo_competitor,
                           Gerarchico = dati_cluster$gruppo_hc)

cat("\n--- Controllo di Consistenza: K-Means vs Gerarchico ---\n")
```

```
##
## --- Controllo di Consistenza: K-Means vs Gerarchico ---
```

```
print(tabella_confronto)
```

```
##           Gerarchico
## K_Means    1      2      3
##          1      0 3495  135
##          2      2  143  636
##          3     676  675   14
```

La matrice di consistenza mostra una **forte corrispondenza** tra i due algoritmi: ogni segmento individuato dal K-means trova un corrispettivo *dominante* nel clustering gerarchico (le celle fuori dal blocco principale, contenute, sono casi di confine geografico). La convergenza di due tecniche con criteri diversi conferma che la partizione a 3 gruppi coglie dinamiche di mercato reali e non un artefatto metodologico del K-means.

Il segmento di riferimento: l'arena Lido–Castello–Isole (Cluster 2)

Per la struttura oggetto di studio — collocata al Lido — il segmento di riferimento non è definito dalla tipologia di alloggio ma dalla **posizione**. Il Cluster 2 raggruppa su base puramente geografica gli annunci di **Lido, della porzione orientale di Castello e delle Isole Minori**: esattamente l'area che ospita la Mostra del Cinema. Conta 781 strutture, con una capacità media di 3.9 ospiti e un prezzo medio di 154 € a notte (mediana 141 €).

- **Composizione territoriale.** Il segmento è di fatto assente dai sestieri centrali e occidentali e si concentra al Lido (269 strutture), a Castello (367) e nelle Isole Minori (145). È la firma del “modello gravitazionale” del mercato: la prossimità all'epicentro dell'evento prevale su ogni altra caratteristica.
- **Rilevanza per l'analisi.** È questo il bacino concorrenziale rilevante per la Mostra: la tariffa di base e il moltiplicatore di rincaro stimati nelle fasi successive poggiano su questo segmento, e la sua mediana di 141 € fornisce il parametro di partenza per il posizionamento.

Gli altri due segmenti

Gli altri due raggruppamenti insistono sul **nucleo storico centrale** (coordinate vicine alla media cittadina) e si distinguono fra loro per **prezzo e capacità ricettiva**, non per posizione. La struttura del Lido non vi compete direttamente:

- **Fascia alta / grande capacità (Cluster 3).** 1365 strutture posizionate sopra la media cittadina sia per tariffa (324 € a notte) sia per posti letto (5.4 ospiti in media): è l'offerta rivolta a gruppi e famiglie numerose. Nel profilo dei centri standardizzati è il segmento con i valori più alti su *prezzo* e *ospiti*.

- **Mercato economico di base (Cluster 1).** È il raggruppamento più numeroso (3630 strutture) e costituisce il “mercato generale” della città: alloggi di taglia contenuta (3.1 ospiti) e tariffa inferiore alla media (162 €), diffusi nell’intero centro storico. Nel profilo è il segmento che si colloca sotto la media su *prezzo* e *ospiti*.

La distinzione fra questi due blocchi — fascia alta contro mercato di base — emerge con nettezza dai profili dei centri standardizzati (grafico sopra) e non incide sull’analisi dell’evento, che dipende dal solo segmento geografico del Lido.

PREVISIONI PER PERIODO DELLA MOSTRA DEL CINEMA

```
# =====
#                               PROCEDURA COMPLETA
# Architettura integrata: modelli euristici, Gradient Boosting con perdita L1 e
# combinazione dei modelli
# =====

# =====
# FASE 1: ACQUISIZIONE, INTEGRAZIONE E PRE-ELABORAZIONE DEI DATI
# =====

# 1.1 Caricamento dei prezzi rilevati dal mercato e uniformazione degli indirizzi
scraped_airbnb <- read_csv("../dati/prezzi_airbnb_lido_pulito.csv") %>%
  mutate(URL_Annuncio = str_replace(URL_Annuncio, "airbnb\\.it", "airbnb\\.com"))

# 1.2 Unione con il campione locale, preservando tutte le osservazioni di base
lido_mostra <- dati_completi %>%
  filter(neighbourhood_cleansed %in% c("Lido", "Castello")) %>%
  left_join(
    scraped_airbnb %>% select(URL_Annuncio, Prezzo_Mostra_Notte),
    by = c("listing_url" = "URL_Annuncio")
  ) %>%
  rename(prezzo_notte_mostra = Prezzo_Mostra_Notte)

# 1.3 Calcolo della distanza geodetica (Formula di Haversine) dall'epicentro dell'evento
coord_palazzo <- c(12.367691248426379, 45.40592783111293)

lido_castello_mostra <- lido_mostra %>%
  rowwise() %>%
  mutate(
    distanza_cinema_km = distHaversine(c(longitude, latitude), coord_palazzo) / 1000
  ) %>%
  ungroup()

# 1.4 Risoluzione delle dipendenze strutturali per l'inferenza del modello base
lido_castello_mostra <- lido_castello_mostra %>% mutate(id = as.character(id))
dati_modelli <- dati_modelli %>% mutate(listing_id = as.character(listing_id))

colonne_da_aggiungere <- setdiff(names(dati_modelli), names(lido_castello_mostra))

lido_castello_mostra <- lido_castello_mostra %>%
  left_join(
```

```

    dati_modelli %>% select(listing_id, all_of(colonne_da_aggiungere)),
    by = c("id" = "listing_id")
  )

# -----
# 1.5 Preparazione delle variabili categoriche (eseguita prima della previsione)
# La codifica come fattore è necessaria perché il modello GBM richiede lo stesso
# tipo di dato impiegato in fase di addestramento
# -----
col_fattori <- c("room_type", "neighbourhood", "host_is_superhost",
               "host_is_local", "instant_bookable", "host_type_classe",
               "host_acceptance_classe", "host_response_classe",
               "has_reviews", "bathrooms_clean")

lido_castello_mostra <- lido_castello_mostra %>%
  mutate(across(any_of(col_fattori), as.factor))

lido_castello_mostra$neighbourhood_cleansed <-
  ↪ as.factor(lido_castello_mostra$neighbourhood_cleansed)
lido_castello_mostra$gruppo_competitor <-
  ↪ as.factor(lido_castello_mostra$gruppo_competitor)

# -----
# 1.6 Generazione della stima base tramite il GBM pre-addestrato
# Le variabili categoriche sono ora coerenti e non producono conversioni fallite
# -----
# La stima base usa il GBM già calibrato in 4.2 (oggetto mod_gbm_final in memoria),
# con il numero di alberi selezionato dalla regola 1-SE (opt_trees_final).
# Il GBM predice il LOG del prezzo: si ri-trasforma con exp() per riportare la
# stima base in scala € (necessario per il moltiplicatore di rincaro).
lido_castello_mostra$stima_modello_base <- exp(predict(
  mod_gbm_final,
  newdata = lido_castello_mostra,
  n.trees = opt_trees_final
))

cat("Fase 1 completata: insieme di dati costruito correttamente, senza avvisi.\n\n")

## Fase 1 completata: insieme di dati costruito correttamente, senza avvisi.

# =====
# FASE 2: DEFINIZIONE DELLA VARIABILE RISPOSTA E TRATTAMENTO DEI VALORI ANOMALI
# =====

# 2.1 Suddivisione tra osservazioni note (prezzo rilevato) e da stimare
dati_training_sani <- lido_castello_mostra %>% filter(!is.na(prezzo_notte_mostra))
dati_da_prevedere <- lido_castello_mostra %>% filter(is.na(prezzo_notte_mostra))

cat("Osservazioni note (valide):", nrow(dati_training_sani), "\n")

## Osservazioni note (valide): 132

```

```

cat("Osservazioni da stimare:", nrow(dati_da_prevedere), "\n\n")

## Osservazioni da stimare: 1474

# 2.2 Calcolo del moltiplicatore di rincaro relativo all'evento
dati_training_sani <- dati_training_sani %>%
  mutate(moltiplicatore = prezzo_notte_mostra / stima_modello_base)

# 2.3 Vettore delle variabili esplicative per i modelli
var_gbm <- c(var_alberi, "stima_modello_base", "gruppo_competitor")

# =====
# FASE 2.5: SCELTA DI K PER IL KNN SPAZIALE - CONVALIDA LEAVE-ONE-OUT (LOO)
# La convalida LOO seleziona il K che minimizza l'errore quadratico medio (MSE)
# fuori campione sulle osservazioni note. Si adotta l'MSE - anziché il MAE -
# perché penalizza maggiormente gli errori grandi, che in questo mercato si
# concentrano sugli annunci costosi prossimi all'epicentro: il KNN viene così
# specializzato sul segmento del rincaro (surge), la dinamica spaziale di
# maggiore interesse.
# =====

# Funzione ausiliaria: mediana ponderata con pesi pari a 1/(distanza + 1).
# I vicini più prossimi pesano di più nella stima; il termine +1 (in metri) evita
# la divisione per zero nel caso di alloggi quasi coincidenti.
weighted_median_knn <- function(x, w) {
  if (length(x) == 1) return(x)
  ord <- order(x)
  x_s <- x[ord]
  w_s <- w[ord] / sum(w[ord])
  x_s[which(cumsum(w_s) >= 0.5)[1]]
}

k_grid <- c(3, 5, 7, 10, 15, 20, 25, 30)
mse_k <- numeric(length(k_grid))

cat("Scelta di K per il KNN spaziale (LOO, criterio MSE, su", nrow(dati_training_sani),
  ↪ "osservazioni):\n")

```

```

## Scelta di K per il KNN spaziale (LOO, criterio MSE, su 132 osservazioni):

```

```

for (ki in seq_along(k_grid)) {
  K_try <- k_grid[ki]
  errs2 <- numeric(nrow(dati_training_sani))

  for (i in 1:nrow(dati_training_sani)) {
    coord_i <- c(dati_training_sani$longitude[i], dati_training_sani$latitude[i])
    ref <- dati_training_sani[-i, ]
    dist_i <- distHaversine(matrix(c(ref$longitude, ref$latitude), ncol = 2), coord_i)
    idx_nn <- order(dist_i)[1:min(K_try, nrow(ref))]
    w_nn <- 1 / (dist_i[idx_nn] + 1)
    mol_i <- weighted_median_knn(ref$moltiplicatore[idx_nn], w_nn)
    errs2[i] <- (dati_training_sani$prezzo_notte_mostra[i] -

```

```

        dati_training_sani$stima_modello_base[i] * mol_i)^2
    }

    mse_k[ki] <- mean(errs2)
    cat(sprintf(" K = %2d | MSE L00: %.0f | RMSE L00: %.2f €\n",
               K_try, mse_k[ki], sqrt(mse_k[ki])))
}

## K = 3 | MSE L00: 92047 | RMSE L00: 303.39 €
## K = 5 | MSE L00: 86357 | RMSE L00: 293.87 €
## K = 7 | MSE L00: 84784 | RMSE L00: 291.18 €
## K = 10 | MSE L00: 88265 | RMSE L00: 297.09 €
## K = 15 | MSE L00: 124427 | RMSE L00: 352.74 €
## K = 20 | MSE L00: 142445 | RMSE L00: 377.42 €
## K = 25 | MSE L00: 142383 | RMSE L00: 377.34 €
## K = 30 | MSE L00: 144493 | RMSE L00: 380.12 €

best_K <- k_grid[which.min(mse_k)]
cat(sprintf("\nK ottimale selezionato (min MSE): %d (RMSE L00: %.2f €)\n\n",
           best_K, sqrt(min(mse_k))))

##
## K ottimale selezionato (min MSE): 7 (RMSE L00: 291.18 €)

# =====
# FASE 3: MODELLAZIONE EURISTICA (DETERMINISTICA E SPAZIALE)
# =====

# -----
# METODO 1: mediana per segmento (ipotesi di omogeneità interna al segmento)
# -----
# Mediana complessiva dei moltiplicatori osservati

moltiplicatore_globale <- median(dati_training_sani$moltiplicatore, na.rm = TRUE)

tabelle_moltiplicatori <- dati_training_sani %>%
  group_by(gruppo_competitor) %>%
  summarise(
    n_annunci_usati = n(),
    moltiplicatore_cluster = median(moltiplicatore, na.rm = TRUE),
    .groups = "drop"
  )

dati_incogniti_cluster <- dati_da_prevedere %>%
  left_join(
    tabelle_moltiplicatori %>% select(gruppo_competitor, moltiplicatore_cluster),
    by = "gruppo_competitor"
  ) %>%
  mutate(
    moltiplicatore_cluster = ifelse(is.na(moltiplicatore_cluster),
      ↪ moltiplicatore_globale, moltiplicatore_cluster),
    prezzo_notte_mostra = stima_modello_base * moltiplicatore_cluster
  )

```

```

)

# -----
# METODO 2: interpolazione spaziale dei moltiplicatori tramite KNN
# K selezionato via LOO (Fase 2.5); pesi inversamente proporzionali alla distanza.
# -----
K_vicini <- best_K
dati_incogniti_knn <- dati_da_prevedere %>% mutate(prezzo_notte_mostra = NA_real_)

for (i in 1:nrow(dati_incogniti_knn)) {
  coord_target <- c(dati_incogniti_knn$longitude[i], dati_incogniti_knn$latitude[i])
  distanze <- distHaversine(
    matrix(c(dati_training_sani$longitude, dati_training_sani$latitude), ncol = 2),
    coord_target
  )
  indici_vicini <- order(distanze)[1:K_vicini]
  pesi_vicini <- 1 / (distanze[indici_vicini] + 1)
  moltiplicatore_locale <- weighted_median_knn(
    dati_training_sani$moltiplicatore[indici_vicini],
    pesi_vicini
  )
  dati_incogniti_knn$prezzo_notte_mostra[i] <- dati_incogniti_knn$stima_modello_base[i]
  ↪ * moltiplicatore_locale
}

cat("Fase 3 completata: applicata la stima euristica spaziale e per segmento.\n\n")

```

Fase 3 completata: applicata la stima euristica spaziale e per segmento.

```

# =====
# FASE 4-5: SELEZIONE IPERPARAMETRI (CONVALIDA A 5 BLOCCHI) E GBM DEFINITIVO
# Ottimizzazione del Gradient Boosting minimizzando la perdita L1 (Laplace).
# Questo blocco (selezione + addestramento finale) è la parte computazionalmente
# più onerosa: una volta eseguito, il modello e i suoi iperparametri vengono salvati
# su disco e ricaricati automaticamente alle esecuzioni successive, azzerando i tempi
# di calcolo.
# =====

# v6: segmentazione a K=3 (gruppo_competitor a 3 livelli). Selezione: per il GBM
# mostra si usa l'OTTIMO (min MAE-CV) perche' col campione ridotto la banda 1-SE
# e' troppo ampia. La regola 1-SE resta in uso solo per il GBM base.
file_cache_gbm <- "../Modelli/cache_gbm_mostra_v7.rds"

if (file.exists(file_cache_gbm)) {
  cat("GBM definitivo caricato dalla cache (nessun ricalcolo).\n")
  cache_gbm <- readRDS(file_cache_gbm)
  best_shrinkage <- cache_gbm$best_shrinkage
  best_depth <- cache_gbm$best_depth
  best_trees <- cache_gbm$best_trees
  best_trees_completo <- cache_gbm$best_trees_completo
  modello_gbm_completo <- cache_gbm$modello
  mae_cv_sel <- cache_gbm$mae_cv_sel
  opt_shrinkage_m <- cache_gbm$opt_shrinkage_m

```

```

    opt_depth_m      <- cache_gbm$opt_depth_m
    opt_trees_m      <- cache_gbm$opt_trees_m
    mae_cv_min_m     <- cache_gbm$mae_cv_min_m
    se_cv_min_m      <- cache_gbm$se_cv_min_m

  } else {

    set.seed(123)
    K <- 5
    folds <- sample(rep(1:K, length.out = nrow(dati_training_sani)))

    nus <- c(0.01, 0.02, 0.05, 0.1, 0.2, 0.5)
    dps <- c(1, 2, 3, 5)

    tuning_par <- expand.grid(shrinkage = nus, interaction.depth = dps)
    max_trees <- 3000
    step_trees <- 50
    tree_seq <- seq(step_trees, max_trees, by = step_trees)

    err_cv_gbm <- array(NA, dim = c(nrow(tuning_par), length(tree_seq), K))

    cat("Avvio della selezione del GBM Laplace (convalida incrociata a 5 blocchi, regola
    ↪ 1-SE)...\\n")

    for (j in 1:nrow(tuning_par)) {

      for (k in 1:K) {
        indici_val <- which(folds == k)
        train_cv <- dati_training_sani[-indici_val, ]
        val_cv <- dati_training_sani[indici_val, ]

        train_gbm_cv <- train_cv %>% select(all_of(var_gbm), prezzo_notte_mostra) %>%
        ↪ rename(y = prezzo_notte_mostra) %>% mutate(y = log(y))
        val_gbm_cv <- val_cv %>% select(all_of(var_gbm), prezzo_notte_mostra) %>%
        ↪ rename(y = prezzo_notte_mostra) %>% mutate(y = log(y))

        tmp <- gbm(
          y ~ .,
          data = train_gbm_cv,
          distribution = "laplace", # Perdita ottimizzata per il MAE
          n.trees = max_trees,
          shrinkage = tuning_par$shrinkage[j],
          interaction.depth = tuning_par$interaction.depth[j],
          bag.fraction = 0.5,
          n.minobsinnode = 10,
          keep.data = FALSE,
          verbose = FALSE
        )

        p.tmp <- predict(tmp, newdata = val_gbm_cv, n.trees = tree_seq)
        err_cv_gbm[j, , k] <- apply(p.tmp, 2, function(pred) mean(abs(val_gbm_cv$y - pred)))
      }
    }
  }
}

```

```

# Selezione iperparametri (calcola sia l'ottimo sia la 1-SE).
# L'insieme noto della Mostra e' piccolo, quindi l'errore standard TRA i fold
# e' ampio e la banda 1-SE risulterebbe larghissima, ammettendo modelli quasi
# degeneri. Per il GBM mostra si adotta perciò l'OTTIMO (minimo
# MAE-CV). La regola 1-SE resta in uso per il GBM base (n grande).
sel_m <- seleziona_gbm_1se(err_cv_gbm, tuning_par$interaction.depth, tree_seq)

opt_shrinkage_m <- tuning_par$shrinkage[sel_m$opt_row]
opt_depth_m <- tuning_par$interaction.depth[sel_m$opt_row]
opt_trees_m <- sel_m$opt_trees
mae_cv_min_m <- sel_m$mae_min
se_cv_min_m <- sel_m$se_min

# Ottimo (min MAE-CV)
best_shrinkage <- opt_shrinkage_m
best_depth <- opt_depth_m
best_trees <- opt_trees_m
mae_cv_sel <- mae_cv_min_m

# Correzione del numero di iterazioni per la maggiore numerosità del campione
# finale (in CV ogni fold vede (K-1)/K dei dati): riscalo di K/(K-1).
best_trees_completo <- round(best_trees * (K / (K - 1)))

cat(sprintf("\nGBM mostra - OTTIMO scelto (min MAE-CV): shrinkage=%.3f | depth=%d |
  ↳ alberi=%d (->%d sul completo) | MAE-CV=%.3f (log) (SE %.3f)\n",
    best_shrinkage, best_depth, best_trees, best_trees_completo, mae_cv_sel,
    se_cv_min_m))
cat(sprintf(" (la regola 1-SE non si applica qui: banda %.2f troppo ampia per il
  ↳ campione ridotto)\n\n",
    mae_cv_min_m + se_cv_min_m))

# ---- Addestramento del GBM definitivo sull'intero insieme di osservazioni note ----
set.seed(123)
modello_gbm_completo <- gbm(
  y ~ .,
  data = dati_training_sani %>% select(all_of(var_gbm), prezzo_notte_mostra)
  ↳ %>% rename(y = prezzo_notte_mostra) %>% mutate(y = log(y)),
  distribution = "laplace",
  n.trees = best_trees_completo,
  shrinkage = best_shrinkage,
  interaction.depth = best_depth,
  bag.fraction = 0.5,
  n.minobsinnode = 10,
  verbose = FALSE
)

# Salvataggio in cache per azzerare i tempi di calcolo nelle esecuzioni future
saveRDS(list(best_shrinkage = best_shrinkage,
  best_depth = best_depth,
  best_trees = best_trees,
  best_trees_completo = best_trees_completo,
  modello = modello_gbm_completo,
  mae_cv_sel = mae_cv_sel,
  opt_shrinkage_m = opt_shrinkage_m,

```



```

        opt_depth_m      = opt_depth_m,
        opt_trees_m      = opt_trees_m,
        mae_cv_min_m     = mae_cv_min_m,
        se_cv_min_m      = se_cv_min_m),
    file_cache_gbm)
cat("GBM definitivo calibrato e salvato in cache.\n")

} # fine ramo di calcolo del GBM (else)

```

```
## GBM definitivo caricato dalla cache (nessun ricalcolo).
```

```

# Stime per le osservazioni ignote
x_prev <- dati_da_prevedere %>% select(all_of(var_gbm))
pred_gbm_finali <- exp(predict(modello_gbm_completo, newdata = x_prev, n.trees =
  ↪ best_trees_completo))

# =====
# FASE 6: CALIBRAZIONE DEI PESI - CONVALIDA INCROCIATA ANNIDATA 5×5 (NON DISTORTA)
# Ciascun blocco esterno rieleziona, sul solo insieme di addestramento interno,
# tutti gli iperparametri che dipendono dai dati: gli iperparametri del GBM (per
# convalida interna a 5 blocchi) e il numero di vicini K del KNN (per LOO interna,
# criterio MSE). Nessuna scelta "vede" il blocco di verifica, eliminando la
# contaminazione del precedente protocollo a parametri fissi. Le stime di errore
# (MAE) sono quindi non distorte e direttamente confrontabili tra i tre modelli.
# =====

file_cache_pesi <- "../Modelli/cache_pesi_ensemble_v6.rds"

if (file.exists(file_cache_pesi)) {
  cat("Pesi della combinazione caricati dalla cache (convalida annidata non
    ↪ rieseguita).\n")
  cache_pesi <- readRDS(file_cache_pesi)
  mae_cluster_loo <- cache_pesi$mae_cluster
  mae_knn_loo <- cache_pesi$mae_knn
  mae_gbm_loo <- cache_pesi$mae_gbm
  pesi_norm <- cache_pesi$pesi_norm
  oof_pred_cluster <- cache_pesi$oof_cluster
  oof_pred_knn <- cache_pesi$oof_knn
  oof_pred_gbm <- cache_pesi$oof_gbm
} else {

cat("Avvio della convalida annidata (5 esterni × 5 interni) per pesi non distorti...\n")

set.seed(123)
K_outer <- 5
K_inner <- 5

folds_outer <- sample(rep(1:K_outer, length.out = nrow(dati_training_sani)))

# Griglia degli iperparametri (identica alla Fase 4)
max_trees_ncv <- 3000
step_trees_ncv <- 50

```

```

tree_seq_ncv <- seq(step_trees_ncv, max_trees_ncv, by = step_trees_ncv)

nus_ncv <- c(0.01, 0.02, 0.05, 0.1, 0.2, 0.5)
dps_ncv <- c(1, 2, 3, 5, 7)
tuning_par_ncv <- expand.grid(shrinkage = nus_ncv, interaction.depth = dps_ncv)

# Vettori per le previsioni fuori blocco (una per osservazione, non distorte)
oof_pred_gbm <- numeric(nrow(dati_training_sani))
oof_pred_cluster <- numeric(nrow(dati_training_sani))
oof_pred_knn <- numeric(nrow(dati_training_sani))

file_ncv_backup <- "../Modelli/ncv_oof_backup.rds"

for (k_out in 1:K_outer) {

  cat(sprintf("\n--- Blocco esterno %d/%d ---\n", k_out, K_outer))

  idx_test <- which(folds_outer == k_out)
  idx_train <- which(folds_outer != k_out)

  train_out <- dati_training_sani[idx_train, ]
  test_out <- dati_training_sani[idx_test, ]

  # ----- MEDIANA PER SEGMENTO (fuori blocco, nessun iperparametro da stimare) -----
  tab_mol_oof <- train_out %>%
    group_by(gruppo_competitor) %>%
    summarise(mol = median(moltiplicatore, na.rm = TRUE), .groups = "drop")

  mol_test <- test_out %>%
    left_join(tab_mol_oof, by = "gruppo_competitor") %>%
    mutate(mol = ifelse(is.na(mol), median(train_out$moltiplicatore, na.rm = TRUE), mol))
    ↪ %>%
    pull(mol)

  oof_pred_cluster[idx_test] <- test_out$stima_modello_base * mol_test

  # ----- KNN SPAZIALE (fuori blocco, K riselezionato per MSE nel solo train) -----
  # Per non distorcere la stima, K viene scelto tramite una LOO interna sul solo
  # train esterno (criterio MSE, coerente con la Fase 2.5), senza mai usare il
  # blocco di verifica. Solo dopo si predice il blocco esterno con il K selezionato.
  mse_k_in <- numeric(length(k_grid))
  for (ik in seq_along(k_grid)) {
    Kt <- k_grid[ik]
    e2_in <- numeric(nrow(train_out))
    for (a in seq_len(nrow(train_out))) {
      coord_a <- c(train_out$longitude[a], train_out$latitude[a])
      rif_a <- train_out[-a, ]
      d_a <- distHaversine(matrix(c(rif_a$longitude, rif_a$latitude), ncol = 2),
    ↪ coord_a)
      nn_a <- order(d_a)[1:min(Kt, nrow(rif_a))]
      w_a <- 1 / (d_a[nn_a] + 1)
      mol_a <- weighted_median_knn(rif_a$moltiplicatore[nn_a], w_a)
      e2_in[a] <- (train_out$prezzo_notte_mostra[a] - train_out$stima_modello_base[a] *
    ↪ mol_a)^2
    }
  }
}

```

```

}
mse_k_in[ik] <- mean(e2_in)
}
K_fold <- k_grid[which.min(mse_k_in)]

# Previsione del blocco esterno con il K selezionato internamente
for (i in idx_test) {
  coord_i <- c(dati_training_sani$longitude[i], dati_training_sani$latitude[i])
  distanze_i <- distHaversine(
    matrix(c(train_out$longitude, train_out$latitude), ncol = 2),
    coord_i
  )
  vicini_i <- order(distanze_i)[1:min(K_fold, nrow(train_out))]
  pesi_i <- 1 / (distanze_i[vicini_i] + 1)
  mol_locale_i <- weighted_median_knn(train_out$moltiplicatore[vicini_i], pesi_i)
  oof_pred_knn[i] <- dati_training_sani$stima_modello_base[i] * mol_locale_i
}
cat(sprintf(" K interno selezionato (KNN, min MSE): %d\n", K_fold))

# ----- GBM: convalida interna per gli iperparametri specifici di questo blocco -----
folds_inner <- sample(rep(1:K_inner, length.out = nrow(train_out)))
err_inner_arr <- array(NA, dim = c(nrow(tuning_par_ncv), length(tree_seq_ncv),
  K_inner))

for (j in 1:nrow(tuning_par_ncv)) {
  for (k_in in 1:K_inner) {
    tr_in <- train_out[folds_inner != k_in, ]
    val_in <- train_out[folds_inner == k_in, ]

    tr_gbm <- tr_in %>% select(all_of(var_gbm), prezzo_notte_mostra) %>% rename(y =
  prezzo_notte_mostra) %>% mutate(y = log(y))
    val_gbm <- val_in %>% select(all_of(var_gbm), prezzo_notte_mostra) %>% rename(y =
    prezzo_notte_mostra) %>% mutate(y = log(y))

    tmp_in <- gbm(
      y ~ .,
      data = tr_gbm,
      distribution = "laplace",
      n.trees = max_trees_ncv,
      shrinkage = tuning_par_ncv$shrinkage[j],
      interaction.depth = tuning_par_ncv$interaction.depth[j],
      bag.fraction = 0.5,
      n.minobsinnode = 10,
      keep.data = FALSE,
      verbose = FALSE
    )

    p_in <- predict(tmp_in, newdata = val_gbm, n.trees = tree_seq_ncv)
    err_inner_arr[j, , k_in] <- apply(p_in, 2, function(p) mean(abs(val_gbm$y - p)))
  }
}

# Iperparametri ottimali per questo blocco esterno (mai esposti al suo insieme
# di verifica). Si usa l'OTTIMO (min MAE-CV), COERENTE col criterio del modello

```

```

# finale del GBM mostra: cosi' la stima OOF rispecchia il modello deployato.
sel_in      <- seleziona_gbm_1se(err_inner_arr, tuning_par_ncv$interaction.depth,
↪ tree_seq_ncv)
shr_opt     <- tuning_par_ncv$shrinkage[sel_in$opt_row]
dep_opt     <- tuning_par_ncv$interaction.depth[sel_in$opt_row]
trees_opt   <- sel_in$opt_trees
trees_opt_fin <- round(trees_opt * (K_inner / (K_inner - 1)))

cat(sprintf("  Ottimo interno: shrinkage=%.3f | profondita'=%d | alberi=%d\n",
            shr_opt, dep_opt, trees_opt_fin))

# GBM addestrato sull'intero insieme esterno con gli iperparametri ottimali interni
tr_gbm_out  <- train_out %>% select(all_of(var_gbm), prezzo_notte_mostra) %>% rename(y
↪ = prezzo_notte_mostra) %>% mutate(y = log(y))
test_gbm_out <- test_out  %>% select(all_of(var_gbm))

mod_gbm_out <- gbm(
  y ~ .,
  data      = tr_gbm_out,
  distribution = "laplace",
  n.trees    = trees_opt_fin,
  shrinkage  = shr_opt,
  interaction.depth = dep_opt,
  bag.fraction = 0.5,
  n.minobsinnode = 10,
  verbose     = FALSE
)

oof_pred_gbm[idx_test] <- exp(predict(mod_gbm_out, newdata = test_gbm_out, n.trees =
↪ trees_opt_fin))

saveRDS(list(oof_gbm = oof_pred_gbm, oof_cluster = oof_pred_cluster,
            oof_knn = oof_pred_knn, fold_completato = k_out),
        file_ncv_backup)

cat(sprintf("  Blocco %d completato e salvato.\n", k_out))
}

# MAE non distorto per ciascun metodo
mae_cluster_loo <- mean(abs(dati_training_sani$prezzo_notte_mostra - oof_pred_cluster))
mae_knn_loo     <- mean(abs(dati_training_sani$prezzo_notte_mostra - oof_pred_knn))
mae_gbm_loo     <- mean(abs(dati_training_sani$prezzo_notte_mostra - oof_pred_gbm))

cat(sprintf("\nMAE (convalida annidata, non distorto) - Mediana segmento: %.2f € | KNN:
↪ %.2f € | GBM: %.2f €\n\n", mae_cluster_loo, mae_knn_loo, mae_gbm_loo))

# Pesi della combinazione: inversamente proporzionali al MAE, ora con stime non distorte
mae_metodi <- c(Cluster = mae_cluster_loo, KNN = mae_knn_loo, GBM = mae_gbm_loo)
pesi_raw   <- 1 / mae_metodi
pesi_norm  <- pesi_raw / sum(pesi_raw)

# Salvataggio dei pesi e dei MAE non distorti per le esecuzioni successive
saveRDS(list(mae_cluster = mae_cluster_loo, mae_knn = mae_knn_loo,
            mae_gbm = mae_gbm_loo, pesi_norm = pesi_norm,

```

```

        oof_cluster = oof_pred_cluster, oof_knn = oof_pred_knn,
        oof_gbm = oof_pred_gbm),
        file_cache_pesi)

} # fine ramo di calcolo dei pesi (else)

## Pesi della combinazione caricati dalla cache (convalida annidata non rieseguita).

cat("Pesi normalizzati della combinazione (convalida annidata):\n")

## Pesi normalizzati della combinazione (convalida annidata):

print(round(pesi_norm, 4))

## Cluster      KNN      GBM
## 0.3366 0.3246 0.3388

# MAE del SISTEMA COMBINATO sugli annunci NOTI (stima OOF, non distorta):
# si combinano le previsioni fuori-blocco dei tre metodi con i pesi appresi e si
# confronta col prezzo Mostra osservato. (Lieve ottimismo: i pesi sono tarati
# sugli stessi errori OOF, ma le previsioni di base non hanno mai visto il proprio
  ↪ blocco.)
oof_pred_ensemble <- pesi_norm["Cluster"] * oof_pred_cluster +
                     pesi_norm["KNN"]      * oof_pred_knn +
                     pesi_norm["GBM"]      * oof_pred_gbm
mae_ensemble_loo <- mean(abs(dati_training_sani$prezzo_notte_mostra -
  ↪ oof_pred_ensemble))
cat(sprintf("MAE del sistema combinato sugli annunci noti (OOF): %.2f EUR\n",
  ↪ mae_ensemble_loo))

## MAE del sistema combinato sugli annunci noti (OOF): 166.97 EUR

# --- Diagnostica dei residui OOF e MAE sui competitor diretti (2 posti letto) ---
diag_oof <- dati_training_sani %>%
  mutate(stima_oof = oof_pred_ensemble,
         residuo   = prezzo_notte_mostra - stima_oof,
         abs_res   = abs(residuo))

# Alloggi con residuo sproporzionato (|residuo| > 500 EUR): outlier che gonfiano il MAE
soglia_residuo <- 500
residui_anomali <- diag_oof %>%
  filter(abs_res > soglia_residuo) %>%
  arrange(desc(abs_res)) %>%
  transmute(id, neighbourhood = neighbourhood_cleansed, accommodates,
            `Prezzo osservato` = round(prezzo_notte_mostra),
            `Stima (OOF)`      = round(stima_oof),
            Residuo             = round(residuo))
n_anomali <- nrow(residui_anomali)

# MAE complessivo, senza outlier, e sui SOLI competitor diretti (2 posti letto)

```

```

mae_tot_oof      <- mean(diag_oof$abs_res)
mae_no_out_oof   <- mean(diag_oof$abs_res[diag_oof$abs_res <= soglia_residuo])
diretti          <- diag_oof %>% filter(accommodates == 2)
n_diretti        <- nrow(diretti)
mae_diretti_oof  <- mean(diretti$abs_res)
mae_diretti_noout<- mean(diretti$abs_res[diretti$abs_res <= soglia_residuo])
cat(sprintf("MAE OOF - competitor diretti (2 posti, n=%d): %.2f EUR | senza outlier: %.2f
  ↪ EUR\n",
            n_diretti, mae_diretti_oof, mae_diretti_noout))

```

```
## MAE OOF - competitor diretti (2 posti, n=32): 85.32 EUR | senza outlier: 85.32 EUR
```

```

# =====
# FASE 7: COMBINAZIONE FINALE DEI MODELLI (ENSEMBLE) E DIAGNOSTICA NEL CAMPIONE
# =====

# Aggregazione lineare dei modelli deboli e del modello forte
dati_da_prevedere <- dati_da_prevedere %>%
  mutate(
    stima_cluster = dati_incogniti_cluster$prezzo_notte_mostra,
    stima_knn      = dati_incogniti_knn$prezzo_notte_mostra,
    stima_gbm      = pred_gbm_finali,
    stima_ensemble = pesi_norm["Cluster"] * stima_cluster +
                      pesi_norm["KNN"]    * stima_knn      +
                      pesi_norm["GBM"]    * stima_gbm
  )

lido_castello_completo <- bind_rows(dati_training_sani, dati_da_prevedere)

cat(sprintf("\nCombinazione applicata con successo su %d osservazioni da stimare.\n",
  ↪ nrow(dati_da_prevedere)))

```

```
##
## Combinazione applicata con successo su 1474 osservazioni da stimare.
```

```

# Diagnostica nel campione (distorsione ottimistica attesa, a solo scopo illustrativo)
pred_gbm_noti <- exp(predict(modello_gbm_completo, newdata = dati_training_sani %>%
  ↪ select(all_of(var_gbm)), n.trees = best_trees_completo))
dati_training_sani <- dati_training_sani %>% mutate(pred_gbm = pred_gbm_noti, residuo_gbm
  ↪ = prezzo_notte_mostra - pred_gbm)

# =====
# FASE 8: VISUALIZZAZIONE DEI DATI E INDICATORI COMPLESSIVI
# =====

# Tema unico per le slide (sfondo trasparente, testo/assi bianchi, definito nel setup)
tema_base <- tema_slide

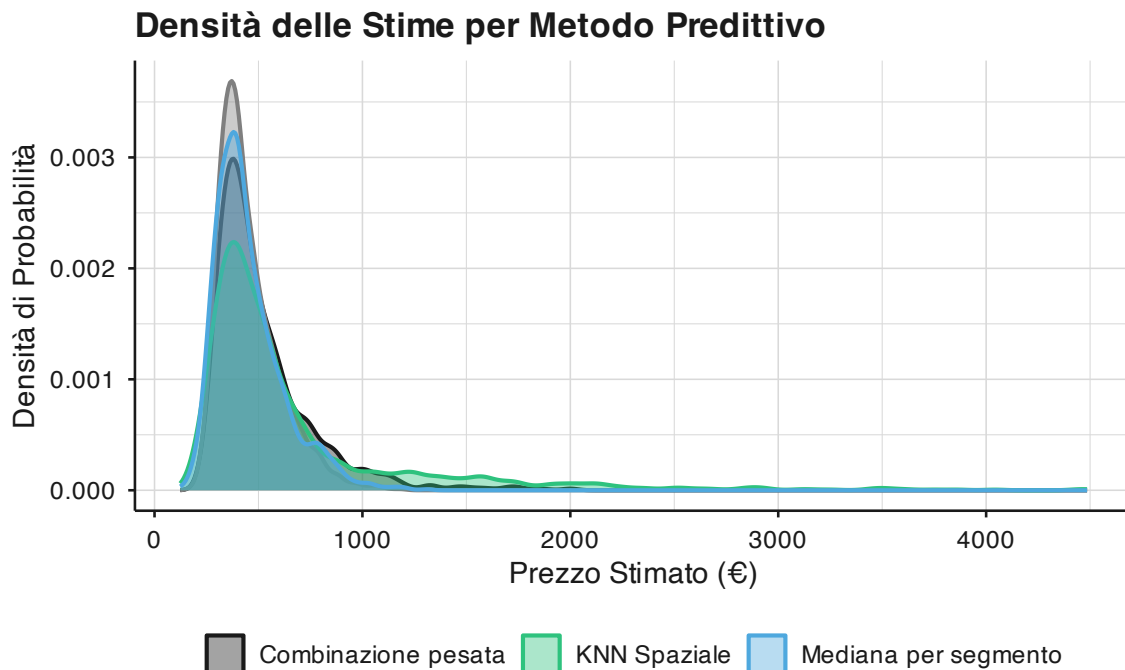
# G5: confronto delle distribuzioni delle stime prodotte dai diversi metodi
g5 <- dati_da_prevedere %>%
  select(stima_cluster, stima_knn, stima_gbm, stima_ensemble) %>%
  pivot_longer(everything(), names_to = "metodo", values_to = "stima") %>%

```

```
mutate(metodo = dplyr::case_match(metodo,
                                   "stima_cluster" ~ "Mediana per segmento",
                                   "stima_knn"      ~ "KNN Spaziale",
                                   "stima_gbm"      ~ "GBM (Laplace)",
                                   "stima_ensemble" ~ "Combinazione pesata")) %>%

ggplot(aes(x = stima, fill = metodo, colour = metodo)) +
  geom_density(alpha = 0.4, linewidth = 0.8) +
  scale_fill_manual(values = pal_metodi) +
  scale_colour_manual(values = pal_metodi) +
  labs(title = "Densità delle Stime per Metodo Predittivo",
       x = "Prezzo Stimato (€)", y = "Densità di Probabilità", fill = NULL, colour =
       ↪ NULL) +
  tema_base

print(g5)
```



```
# cat("\n=====\\n")
# cat("SINTESI DELLE PRESTAZIONI PREDITTIVE (convalida annidata, non distorta)\\n")
# cat("=====\\n")
cat(sprintf("%-22s %12s %12s %10s\\n", "Modello Base", "MAE (€)", "Convalida", "Peso
↪ Rel."))
```

```
## Modello Base          MAE (€)    Convalida  Peso Rel.
```

```
# cat("-----\\n")
cat(sprintf("%-22s %12.2f %12s %10.4f\\n", "Mediana segmento", mae_cluster_loo,
↪ "Annidata", pesi_norm["Cluster"])))
```

```
## Mediana segmento          192.36    Annidata    0.3366
```

```
cat(sprintf("%-22s %12.2f %12s %10.4f\n", "KNN Spaziale", mae_knn_loo,
  ↪ "Annidata", pesi_norm["KNN"])))
```

```
## KNN Spaziale                199.45    Annidata    0.3246
```

```
cat(sprintf("%-22s %12.2f %12s %10.4f\n", "GBM (Laplace)", mae_gbm_loo,
  ↪ "Annidata", pesi_norm["GBM"])))
```

```
## GBM (Laplace)              191.08    Annidata    0.3388
```

```
cat(sprintf("%-22s %12.2f %12s %10s\n", "Combinazione (OOF)", mae_ensemble_loo,
  ↪ "Annidata", "--"))
```

```
## Combinazione (OOF)        166.97    Annidata    --
```

```
# cat("=====\n\n")

# =====
# FASE 9: ANALISI STRATEGICA PER LA STRUTTURA OBIETTIVO (Camere per 2 Ospiti)
# Analisi segmentata per l'individuazione dei tetti di mercato (price cap)
# =====

# =====
# ANALISI STRATEGICA: STRUTTURA OBIETTIVO (Alloggi per 2 Ospiti / Coppie)
# Distribuzione tariffaria stimata dalla combinazione per fasce di prossimità
# =====

dati_target_hotel <- lido_castello_completo %>%
  filter(accommodates == 2) %>%
  mutate(
    # Suddivisione della distanza in fasce operative utili al Revenue Management
    fascia_distanza = cut(
      distanza_cinema_km,
      breaks = c(0, 0.5, 1.5, 20),
      labels = c("Zona Rossa (< 500m)", "Zona Gialla (500m-1.5km)", "Zona Verde (>
        ↪ 1.5km)")
    ),
    # Si usa il prezzo rilevato se disponibile, altrimenti la stima della combinazione
    prezzo_finale = ifelse(is.na(prezzo_notte_mostra), stima_ensemble,
      ↪ prezzo_notte_mostra)
  )

# Soglia di riferimento: scelta prudenziale, consolidata nel Revenue Management.
# Il 15% più alto del mercato (oltre l'85° percentile) è occupato da strutture con
# caratteristiche di fascia alta difficilmente replicabili (vista diretta sull'evento,
# suite con servizi di portineria dedicati, ecc.). Collocarsi oltre questa soglia
# senza elementi distintivi espliciti aumenta il rischio di restare invenduti.
# La tabella di sensibilità sottostante consente di calibrare la soglia in base
# alla propensione al rischio del cliente.
p_cap <- 0.85
```



```

report_hotel <- dati_target_hotel %>%
  group_by(fascia_distanza) %>%
  summarise(
    n_competitor      = n(),
    prezzo_mediano    = median(prezzo_finale, na.rm = TRUE),
    prezzo_medio      = mean(prezzo_finale, na.rm = TRUE),
    price_cap_85       = quantile(prezzo_finale, p_cap, na.rm = TRUE),
    rincaro_medio_px  = mean(prezzo_finale / stima_modello_base, na.rm = TRUE)
  ) %>%
  arrange(fascia_distanza)

print(report_hotel)

```

```

## # A tibble: 3 x 6
##   fascia_distanza      n_competitor prezzo_mediano prezzo_medio price_cap_85
##   <fct>              <int>         <dbl>         <dbl>         <dbl>
## 1 Zona Rossa (< 500m)           9          694.          757.         1003.
## 2 Zona Gialla (500m-1.5km)       31          525.          519.          680.
## 3 Zona Verde (> 1.5km)         431          359.          373.          462.
## # i 1 more variable: rincaro_medio_px <dbl>

```

*# Tabella di sensibilità: tetto di prezzo a percentili alternativi per l'analisi del
↳ rischio*

```

sensitivity_cap <- dati_target_hotel %>%
  group_by(fascia_distanza) %>%
  summarise(
    n_competitor = n(),
    cap_80        = quantile(prezzo_finale, 0.80, na.rm = TRUE),
    cap_85        = quantile(prezzo_finale, 0.85, na.rm = TRUE),
    cap_90        = quantile(prezzo_finale, 0.90, na.rm = TRUE),
    .groups = "drop"
  ) %>%
  arrange(fascia_distanza)

print(sensitivity_cap)

```

```

## # A tibble: 3 x 5
##   fascia_distanza      n_competitor cap_80 cap_85 cap_90
##   <fct>              <int>    <dbl> <dbl> <dbl>
## 1 Zona Rossa (< 500m)           9   981. 1003. 1016.
## 2 Zona Gialla (500m-1.5km)       31   659.  680.  749.
## 3 Zona Verde (> 1.5km)         431   438.  462.  484.

```

```

# -----
# Variante: TUTTI gli alloggi con almeno 2 posti letto (accommodates >= 2)
# Stessa logica della tabella precedente, ma sull'intero mercato (non solo coppie).
# -----
dati_target_2plus <- lido_castello_completo %>%
  filter(accommodates >= 2) %>%
  mutate(
    fascia_distanza = cut(
      distanza_cinema_km,

```

```

    breaks = c(0, 0.5, 1.5, 20),
    labels = c("Zona Rossa (< 500m)", "Zona Gialla (500m-1.5km)", "Zona Verde (>
    ↪ 1.5km)")
  ),
  prezzo_finale = ifelse(is.na(prezzo_notte_mostra), stima_ensemble,
    ↪ prezzo_notte_mostra)
)

report_2plus <- dati_target_2plus %>%
  group_by(fascia_distanza) %>%
  summarise(
    n_competitor      = n(),
    prezzo_mediano    = median(prezzo_finale, na.rm = TRUE),
    prezzo_medio      = mean(prezzo_finale, na.rm = TRUE),
    price_cap_85      = quantile(prezzo_finale, p_cap, na.rm = TRUE),
    rincaro_medio_px  = mean(prezzo_finale / stima_modello_base, na.rm = TRUE),
    .groups = "drop"
  ) %>%
  arrange(fascia_distanza)

sensitivity_2plus <- dati_target_2plus %>%
  group_by(fascia_distanza) %>%
  summarise(
    n_competitor = n(),
    cap_80 = quantile(prezzo_finale, 0.80, na.rm = TRUE),
    cap_85 = quantile(prezzo_finale, 0.85, na.rm = TRUE),
    cap_90 = quantile(prezzo_finale, 0.90, na.rm = TRUE),
    .groups = "drop"
  ) %>%
  arrange(fascia_distanza)

cat("\n--- Distribuzione tariffaria: TUTTI gli alloggi con almeno 2 posti ---\n")

```

```

##
## --- Distribuzione tariffaria: TUTTI gli alloggi con almeno 2 posti ---

```

```
print(report_2plus)
```

```

## # A tibble: 3 x 6
##   fascia_distanza      n_competitor prezzo_mediano prezzo_medio price_cap_85
##   <fct>              <int>          <dbl>          <dbl>          <dbl>
## 1 Zona Rossa (< 500m)           52          1014.          1159.          1650.
## 2 Zona Gialla (500m-1.5km)       168           751.           785.          1114.
## 3 Zona Verde (> 1.5km)        1373           426.           468.           618.
## # i 1 more variable: rincaro_medio_px <dbl>

```

```
print(sensitivity_2plus)
```

```

## # A tibble: 3 x 5
##   fascia_distanza      n_competitor cap_80 cap_85 cap_90
##   <fct>              <int>    <dbl> <dbl> <dbl>

```

## 1 Zona Rossa (< 500m)	52	1524.	1650.	1733.
## 2 Zona Gialla (500m-1.5km)	168	1033.	1114.	1175.
## 3 Zona Verde (> 1.5km)	1373	574.	618.	697.

```
# =====
# VALORI DINAMICI PER LA RELAZIONE
# Tutti i numeri citati nel testo sottostante sono ricavati da queste variabili:
# aggiornando i dati o la calibrazione, la relazione si aggiorna da sola al knit.
# =====

# Formattazione all'italiana (virgola decimale, punto per le migliaia)
fmt <- function(x, d = 2) formatC(x, format = "f", digits = d, decimal.mark = ",",
  ↪ big.mark = ".")
fmt0 <- function(x)      formatC(round(x), format = "d", big.mark = ".")

# Numerosità dei due insiemi
n_noti    <- fmt0(nrow(dati_training_sani))
n_stimati <- fmt0(nrow(dati_da_prevedere))

# Pesi della combinazione (in %), peso GBM in forma decimale e MAE non distorti
# (estrazione con [[ ]] per ottenere il valore senza nome)
peso_gbm <- fmt(100 * pesi_norm[["GBM"]], 2)
peso_knn <- fmt(100 * pesi_norm[["KNN"]], 2)
peso_clu <- fmt(100 * pesi_norm[["Cluster"]], 2)
w_gbm_dec <- fmt(pesi_norm[["GBM"]], 4)
mae_gbm_s <- fmt(mae_gbm_loo)
mae_knn_s <- fmt(mae_knn_loo)
mae_clu_s <- fmt(mae_cluster_loo)

# MAE di selezione del GBM (Fase 4, distorto) ed entità della distorsione,
# usati per illustrare nel testo il confronto con la stima annidata non distorta
mae_gbm_sel <- fmt(mae_cv_sel)
gap_bias    <- fmt(mae_gbm_loo - mae_cv_sel)

# K ottimale del KNN spaziale (produzione, Fase 2.5)
k_opt <- best_K

# Accesso per fascia (robusto all'ordinamento del fattore)
.zv <- function(df, col, key) df[[col]][grepl(key, df$fascia_distanza)][1]
med_rossa <- fmt0(.zv(report_hotel, "prezzo_mediano", "Rossa"))
med_gialla <- fmt0(.zv(report_hotel, "prezzo_mediano", "Gialla"))
med_verde <- fmt0(.zv(report_hotel, "prezzo_mediano", "Verde"))
cap_rossa <- fmt0(.zv(report_hotel, "price_cap_85", "Rossa"))
cap_gialla <- fmt0(.zv(report_hotel, "price_cap_85", "Gialla"))
cap_verde <- fmt0(.zv(report_hotel, "price_cap_85", "Verde"))
cap80_rossa <- fmt0(.zv(sensitivity_cap, "cap_80", "Rossa"))
cap90_rossa <- fmt0(.zv(sensitivity_cap, "cap_90", "Rossa"))
cap90_gialla <- fmt0(.zv(sensitivity_cap, "cap_90", "Gialla"))
mol_rossa <- fmt(.zv(report_hotel, "rincaro_medio_px", "Rossa"), 2)
mol_gialla <- fmt(.zv(report_hotel, "rincaro_medio_px", "Gialla"), 2)
mol_verde <- fmt(.zv(report_hotel, "rincaro_medio_px", "Verde"), 2)
n_rossa <- .zv(report_hotel, "n_competitor", "Rossa")
mult_max <- fmt(max(report_hotel$rincaro_medio_px), 1)

# Funzione per costruire la tabella delle fasce (da report_* + sensitivity_*)
```

```

costruisci_tabella_zone <- function(report, sensitivity) {
  report %>%
    select(fascia_distanza, n_competitor, prezzo_mediano, prezzo_medio, rincaro_medio_px)
    ↪ %>%
    left_join(sensitivity %>% select(fascia_distanza, cap_80, cap_85, cap_90),
              by = "fascia_distanza") %>%
    transmute(
      `Fascia Geografica`      = fascia_distanza,
      `N. Concorrenti`        = n_competitor,
      `Prezzo Mediano (€)`     = round(prezzo_mediano),
      `Prezzo Medio (€)`      = round(prezzo_medio),
      `Tetto 80° (€)`         = round(cap_80),
      `Tetto 85° (€)`         = round(cap_85),
      `Tetto 90° (€)`         = round(cap_90),
      `Moltiplicatore di Rincaro` = paste0(fmt(rincaro_medio_px, 2), "x")
    )
}

# Tabella alloggi per 2 ospiti (struttura obiettivo) e tabella su tutti gli alloggi >= 2
  ↪ posti
tabella_zone      <- costruisci_tabella_zone(report_hotel, sensitivity_cap)
tabella_zone_2plus <- costruisci_tabella_zone(report_2plus, sensitivity_2plus)

```

Affidabilità sui competitor diretti e residui anomali

Il MAE complessivo del sistema combinato sugli annunci noti (167 €, stima OOF) è influenzato da pochi alloggi con **errore sproporzionato**. Si tratta in genere di strutture atipiche (prezzi-Mostra estremi o anomalie di rilevazione) i cui residui, superiori a 500 €, gonfiano la media senza essere rappresentativi del mercato di riferimento.

Table 8: Alloggi noti con residuo OOF sproporzionato ($|\text{residuo}| > 500$ €): 8 casi che falsano il MAE complessivo.

id	neighbourhood	accommodates	Prezzo osservato	Stima (OOF)	Residuo
1360322103669864704	Lido	5	4232	1125	3107
1075299871768616192	Lido	8	1768	833	935
50078067	Castello	9	2015	1132	883
934511716348321280	Lido	3	1227	529	698
1242528494463824128	Castello	4	1076	448	627
49727247	Castello	4	967	396	570
34014328	Castello	6	1233	684	549
938978623479288192	Lido	5	675	1200	-525

Per valutare l'accuratezza sul segmento che conta davvero — i **concorrenti diretti dell'Hotel, gli alloggi per 2 ospiti** — si ricalcola il MAE solo su quel sottoinsieme:

Table 9: MAE OOF del sistema combinato: complessivo vs competitor diretti (2 posti letto).

Insieme	N annunci	MAE (EUR)
Tutti gli annunci noti	132	166.97
Solo competitor diretti (2 posti)	32	85.32
Competitor diretti, senza outlier	32	85.32

Sul segmento dei competitor diretti il MAE scende a **85 €** (contro 167 € sull'intero campione noto). Gli alloggi per 2 ospiti di Lido e Castello sono più omogenei e quindi più prevedibili. È su questo sottoinsieme che la stima del prezzo-evento risulta più affidabile per le decisioni di posizionamento.

Interpretabilità del GBM evento (prezzo Mostra)

Lo stesso strumento di lettura si applica al GBM che stima il **prezzo durante la Mostra**: la dipendenza parziale mostra come il modello traduce prezzo base e segmento di mercato nella tariffa dell'evento.

```
pred_gbm_mostra <- function(d) predict(modello_gbm_completo, newdata = d, n.trees =
  ↪ best_trees_completo)

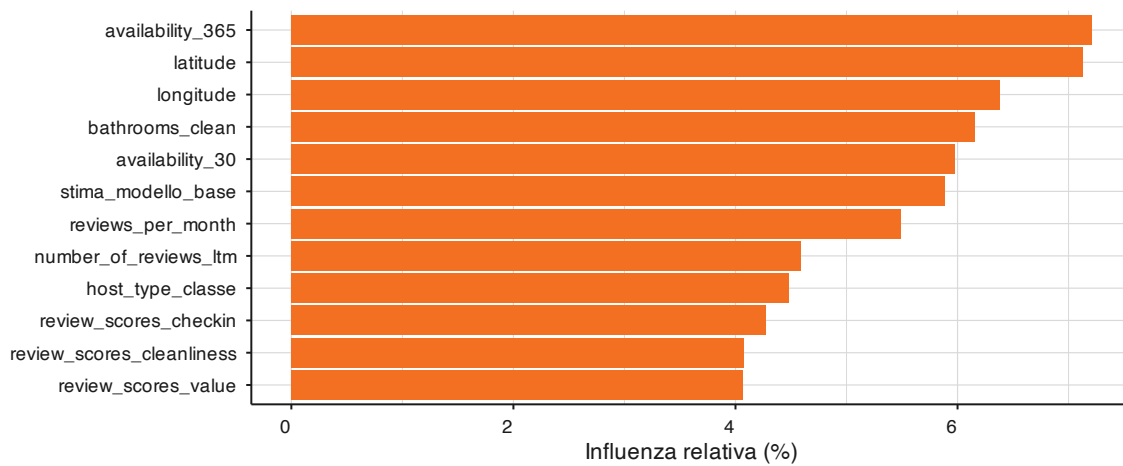
# (a) Influenza relativa: quanto pesa la posizione nel GBM base
ri_gbm_mostra <- summary(modello_gbm_completo, n.trees = best_trees_completo, plotit =
  ↪ FALSE)
ri_gbm_mostra$var <- as.character(ri_gbm_mostra$var)
peso_posizione_gbm_mostra <- sum(ri_gbm_mostra$rel.inf[ri_gbm_mostra$var %in%
  ↪ c("latitude", "longitude")])
ri_top <- head(ri_gbm_mostra[order(-ri_gbm_mostra$rel.inf), ], 12)

# =====
# 1. Variabili più importanti
# =====
g_ri <- ggplot(ri_top, aes(reorder(var, rel.inf), rel.inf)) +
  geom_col(fill = col_arancio) +
  coord_flip() +
  labs(title = "GBM: variabili più importanti (influenza relativa)",
       subtitle = sprintf("Latitudine + longitudine pesano insieme circa %.0f%% del
  ↪ totale",
                           peso_posizione_gbm),
       x = NULL, y = "Influenza relativa (%)") +
  tema_slide +
  theme(
    axis.text.y = element_text(size = 10, hjust = 1),
    axis.text.x = element_text(size = 10, hjust = 1),
    plot.margin = margin(t = 10, r = 15, b = 10, l = 30, unit = "pt"),
    plot.title = element_text(size = 14),
    plot.subtitle = element_text(size = 11)
  )

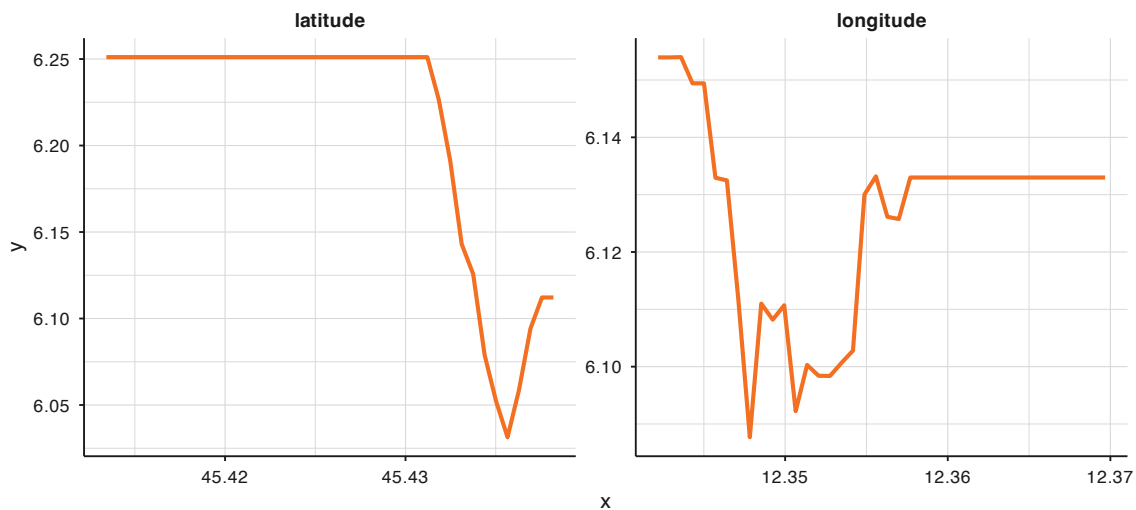
print(g_ri)
```

GBM: variabili più importanti (influenza relativa)

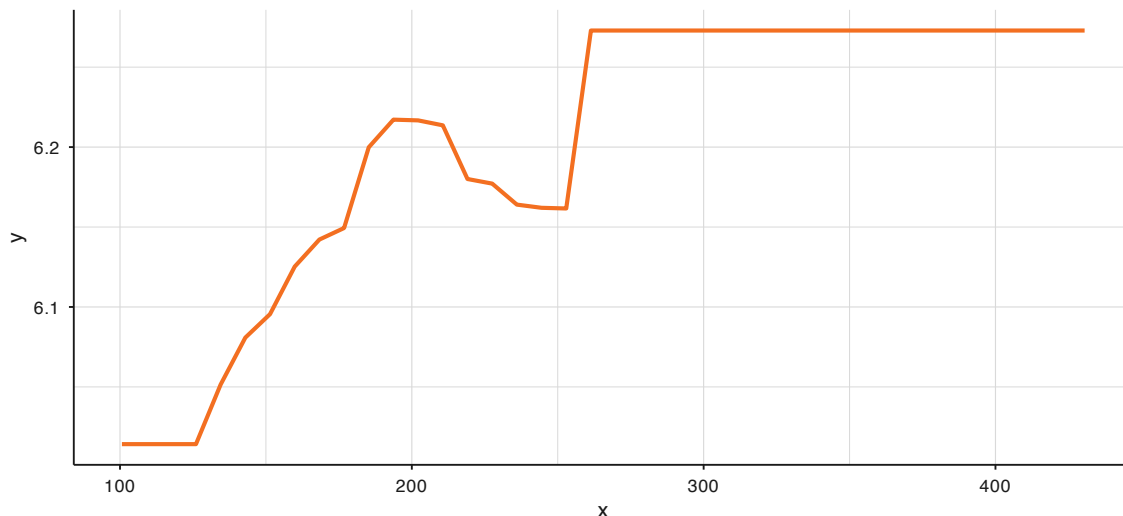
Latitudine + longitudine pesano insieme circa 18% del totale



```
# PDP
pdp_m_cont <- rbind(
  pdp_modello(pred_gbm_mostra, dati_training_sani, "latitude"),
  pdp_modello(pred_gbm_mostra, dati_training_sani, "longitude")
)
g_pdp_m <- ggplot(pdp_m_cont, aes(x, y)) +
  geom_line(colour = col_arancio, linewidth = 1.1) +
  facet_wrap(~ variabile, scales = "free") +
  tema_slide
print(g_pdp_m)
```



```
# PDP sul segmento (effetto del cluster di concorrenti)
pdp_m_seg <- pdp_modello(pred_gbm_mostra, dati_training_sani, "stima_modello_base")
g_pdp_seg <- ggplot(pdp_m_seg, aes(x, y)) +
  geom_line(colour = col_arancio, linewidth = 1.1) +
  tema_slide
print(g_pdp_seg)
```



La curva su `stima_modello_base` è, di fatto, il **moltiplicatore di rincaro** appreso dal modello, mentre il segmento sposta il livello in funzione del posizionamento competitivo.

Revenue Management Predittivo: Dinamiche Tariffarie e Posizionamento Strategico durante la Mostra del Cinema di Venezia

1. Architettura metodologica: dal modello di base alla combinazione

Per ottenere stime il quanto più affidabili possibili abbiamo applicato una procedura articolata in modo da separare il valore strutturale dell'immobile dalla componente speculativa legata all'evento.

1.1 Isolamento del moltiplicatore di rincaro

Il procedimento ha previsto la stima di un **prezzo di base** (mercato ordinario) per l'intero campione tramite un algoritmo Gradient Boosting Machine (GBM) pre-addestrato. Per i 132 alloggi di cui è stato rilevato il prezzo durante la Mostra (dati osservati), è stato calcolato il **moltiplicatore di rincaro** (rapporto tra prezzo durante la Mostra e prezzo di base). Questa separazione è essenziale: consente di quantificare la spinta speculativa dell'evento al netto delle caratteristiche intrinseche dell'alloggio (numero di bagni, recensioni, qualifica dell'inserzionista).

1.2 I tre modelli deboli

Per stimare le tariffe dei 1.474 appartamenti privi di prezzo rilevato, sono state formulate tre logiche predittive complementari:

- **Mediana per segmento (approccio economico):** assume omogeneità tariffaria all'interno dei segmenti concorrenziali e applica il moltiplicatore mediano del segmento all'alloggio da stimare.
- **KNN spaziale (approccio geografico):** ricorre all'interpolazione spaziale (K vicini più prossimi, *K-Nearest Neighbors*), calcolando la tariffa sulla base dei rincari applicati dai vicini geografici più prossimi, individuati tramite la distanza di Haversine, con pesi inversamente proporzionali alla distanza per

privilegiare i concorrenti immediatamente adiacenti. Poiché l'MSE penalizza maggiormente gli errori grandi — che ricorrono sugli annunci costosi prossimi all'epicentro — si è deciso di utilizzare questa metrica per questo metodo.

- **GBM Laplace (approccio non lineare):** un algoritmo di Gradient Boosting ottimizzato sulla perdita L1 (Laplace). Tale scelta minimizza l'errore assoluto medio (MAE) e rende il modello robusto ai valori anomali speculativi tipici dei grandi eventi, senza richiedere trasformazioni logaritmiche della variabile risposta.

2. Analisi strategica: alloggi per 2 ospiti (struttura obiettivo)

Il risultato di maggior valore operativo è la segmentazione del mercato ristretta agli **alloggi per 2 ospiti** (i diretti concorrenti delle camere matrimoniali della struttura considerata), suddivisi in tre fasce concentriche di prossimità all'evento. I risultati evidenziano dinamiche di posizionamento rilevanti:

Table 10: Distribuzione tariffaria per fascia di prossimità all'evento (alloggi per 2 ospiti). Tetto 80°/85°/90° = percentili della distribuzione stimata; il moltiplicatore è il rincaro medio rispetto al prezzo di base.

Fascia Geografica	N. Concorrenti	Prezzo Mediano (€)	Prezzo Medio (€)	Tetto 80° (€)	Tetto 85° (€)	Tetto 90° (€)	Moltiplicatore di Rincaro
Zona Rossa (< 500m)	9	694	757	981	1003	1016	5,78×
Zona Gialla (500m-1.5km)	31	525	519	659	680	749	4,74×
Zona Verde (> 1.5km)	431	359	373	438	462	484	2,57×

Estesa a **tutti gli alloggi con almeno 2 posti letto** (intero mercato, non solo le camere doppie):

Table 11: Distribuzione tariffaria per fascia di prossimità all'evento (tutti gli alloggi con almeno 2 posti letto). Stesse definizioni della tabella precedente.

Fascia Geografica	N. Concorrenti	Prezzo Mediano (€)	Prezzo Medio (€)	Tetto 80° (€)	Tetto 85° (€)	Tetto 90° (€)	Moltiplicatore di Rincaro
Zona Rossa (< 500m)	52	1014	1159	1524	1650	1733	5,76×
Zona Gialla (500m-1.5km)	168	751	785	1033	1114	1175	4,98×
Zona Verde (> 1.5km)	1373	426	468	574	618	697	2,65×

3.1 Il gradiente gravitazionale del rincaro

Il moltiplicatore di rincaro decresce in modo monotono con la distanza dall'epicentro: si passa da circa 5,78× in **Zona Rossa** (entro 500 m dal Palazzo del Cinema) a 4,74× in **Zona Gialla** (500 m - 1,5 km) e a 2,57× in **Zona Verde** (oltre 1,5 km). La prossimità all'evento si conferma quindi il fattore dominante

nella formazione del prezzo: nel cuore dell'evento le tariffe arrivano a moltiplicarsi fino a 5,78 volte rispetto al mercato ordinario, un premio di posizione che si attenua rapidamente allontanandosi.

Dal punto di vista economico è il comportamento atteso di un mercato a forte **rendita di posizione** (*location rent*): l'offerta immediatamente adiacente all'evento è rigidamente limitata, mentre la disponibilità a pagare degli spettatori e degli operatori del settore è massima e poco elastica. La scarsità localizzata genera così un premio che cresce in modo più che proporzionale all'avvicinarsi dell'epicentro, anziché distribuirsi uniformemente sul territorio.

I tre profili mostrano un comportamento marcatamente diverso:

- In **Zona Rossa** si concentra la massima rendita di posizione: mediana di 694 € e tetto che sale fino a circa 1.003 €. L'ampio divario tra mediana e tetto segnala che le poche strutture immediatamente adiacenti all'evento comandano un premio fortemente crescente. Questa fascia poggia però su un campione molto piccolo (9 strutture) e su un'interpolazione spaziale volutamente reattiva ($K = 7$): le cifre vanno lette come **ordine di grandezza** del potenziale tariffario, non come valori puntuali affidabili.
- In **Zona Gialla** mediana 525 € e tetto 680 € (fino a 749 € al 90° percentile): le strutture dotate di elementi distintivi dispongono di un margine apprezzabile per collocarsi al di sopra della mediana di zona.
- In **Zona Verde** il livello è più contenuto (mediana 359 €, tetto 462 €) ma la stima è la più robusta, poggiando su un mercato ampio (432 strutture).

3.3 Raccomandazioni operative per la struttura cliente

Alla luce delle stime prodotte dalla combinazione, si delineano le seguenti indicazioni:

1. **Definizione del prezzo minimo.** La mediana stimata della propria zona di appartenenza va adottata come tariffa di partenza, da non ridurre durante i giorni di picco del Festival (694 € in Zona Rossa, 525 € in Zona Gialla, 359 € in Zona Verde).
2. **Sfruttamento del tetto di prezzo.** È in **Zona Rossa** che si concentra il maggiore potenziale al rialzo: una struttura con elementi distintivi può spingersi dalla mediana (694 €) fino al tetto stimato (~1.003 €), beneficiando della rendita di posizione del cuore dell'evento. Vista la fragilità del campione (9 strutture), conviene avvicinarsi al tetto in modo graduale e monitorato. In **Zona Gialla** il margine resta apprezzabile (fino a ~749 € al 90° percentile), mentre in **Zona Verde** il posizionamento va mantenuto più prudente, attorno al tetto di 462 €.
3. **Gestione dinamica della disponibilità.** Data la ripidità del premio di posizione vicino all'epicentro, si raccomanda di non immettere sul mercato l'intera disponibilità in fase di prenotazione anticipata, ma di trattenere una quota di camere (15-20%) da collocare sotto data, a tariffe prossime al tetto di mercato. In questo modo si massimizza il ricavo medio giornaliero intercettando la domanda dei ritardatari, particolarmente intensa nelle fasce più prossime all'evento.

```
# =====  
# FASE 10: ANALISI DELLE DISTRIBUZIONI TARIFFARIE (ORDINARIO VS EVENTO)  
# Confronto dello spostamento dei prezzi sui tre sottoinsiemi campionari  
# =====  
  
# 10.1 Preparazione dell'insieme unificato per le distribuzioni  
# Il "prezzo evento" è il prezzo rilevato (se noto) o la stima combinata (se ignoto)  
lido_castello_completo <- lido_castello_completo %>%  
  mutate(prezzo_evento_finale = ifelse(is.na(prezzo_notte_mostra), stima_ensemble,  
    ↪ prezzo_notte_mostra))  
  
# 10.2 Costruzione delle tabelle in formato esteso per ggplot2
```

```

# Sottoinsieme A: solo prezzi noti (rilevati)
df_noti <- dati_training_sani %>%
  select(Ordinario = price_num, Evento = prezzo_notte_mostra) %>%
  pivot_longer(everything(), names_to = "Scenario", values_to = "Prezzo") %>%
  mutate(Campione = "1. Prezzi noti (rilevati)")

# Sottoinsieme B: solo prezzi stimati
df_stimati <- dati_da_prevedere %>%
  select(Ordinario = price_num, Evento = stima_ensemble) %>%
  pivot_longer(everything(), names_to = "Scenario", values_to = "Prezzo") %>%
  mutate(Campione = "2. Prezzi stimati")

# Sottoinsieme C: mercato complessivo (noti + stimati)
df_generale <- lido_castello_completo %>%
  select(Ordinario = price_num, Evento = prezzo_evento_finale) %>%
  pivot_longer(everything(), names_to = "Scenario", values_to = "Prezzo") %>%
  mutate(Campione = "3. Mercato complessivo (noti + stimati)")

# Unione in un'unica tabella per il grafico combinato
df_densita <- bind_rows(df_noti, df_stimati, df_generale)

# Tavolozza cromatica per coerenza visiva tra i grafici
colori_scenario <- pal_scenario # Ordinario = azzurro, Evento = arancio (palette slide)

# =====
# 10.3 GRAFICI SEPARATI
# =====

g_noti <- ggplot(df_noti, aes(x = Prezzo, fill = Scenario)) +
  geom_density(alpha = 0.7, trim = TRUE) +
  scale_fill_manual(values = colori_scenario) +
  coord_cartesian(xlim = c(0, 2000)) +
  labs(title = "Sottoinsieme noto",
       subtitle = paste0("Distribuzione per i ", fmt0(nrow(dati_training_sani)),
                        " appartamenti con prezzo Mostra accertato"),
       x = "Prezzo per Notte (€)", y = "Densità di Probabilità", fill = "Contesto di
       ↳ Mercato") +
  tema_base

```

```

## Warning in prettyNum(r, big.mark = big.mark, big.interval = big.interval, :
## 'big.mark' and 'decimal.mark' are both '.', which could be confusing

```

```

g_stimati <- ggplot(df_stimati, aes(x = Prezzo, fill = Scenario)) +
  geom_density(alpha = 0.7, trim = TRUE) +
  scale_fill_manual(values = colori_scenario) +
  coord_cartesian(xlim = c(0, 2000)) +
  labs(title = "Sottoinsieme stimato",
       subtitle = paste0("Distribuzione per i ", fmt0(nrow(dati_da_prevedere)),
                        " appartamenti stimati dalla combinazione"),
       x = "Prezzo per Notte (€)", y = "Densità di Probabilità", fill = "Contesto di
       ↳ Mercato") +
  tema_base

```

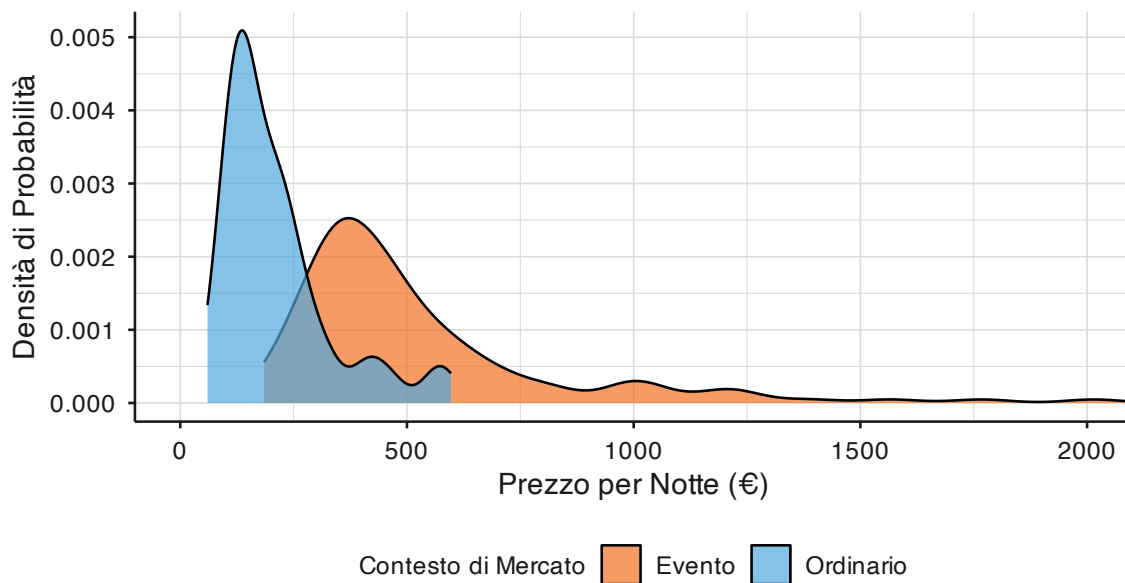
```
## Warning in prettyNum(r, big.mark = big.mark, big.interval = big.interval, :
## 'big.mark' and 'decimal.mark' are both '.', which could be confusing
```

```
g_generale <- ggplot(df_generale, aes(x = Prezzo, fill = Scenario)) +
  geom_density(alpha = 0.7, trim = TRUE) +
  scale_fill_manual(values = colori_scenario) +
  coord_cartesian(xlim = c(0, 2000)) +
  labs(title = "Mercato complessivo Lido-Castello",
       subtitle = "Sintesi complessiva del mercato delle locazioni brevi durante il
        ↳ Festival",
       x = "Prezzo per Notte (€)", y = "Densità di Probabilità", fill = "Contesto di
        ↳ Mercato") +
  tema_base

# Produzione dei grafici singoli
print(g_noti)
```

Sottoinsieme noto

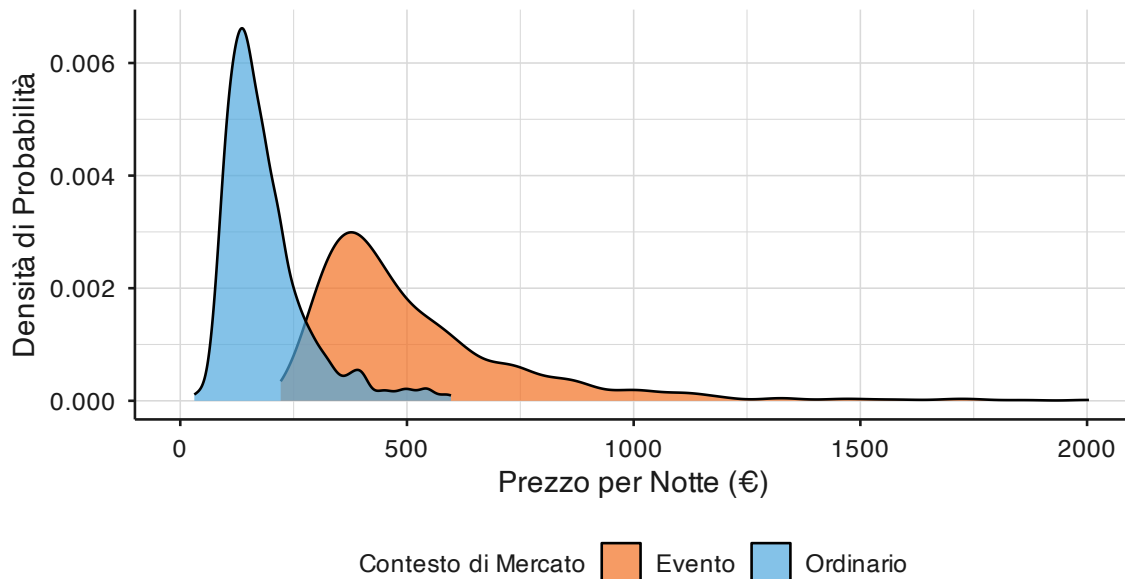
Distribuzione per i 132 appartamenti con prezzo Mostra accertato



```
print(g_stimati)
```

Sottoinsieme stimato

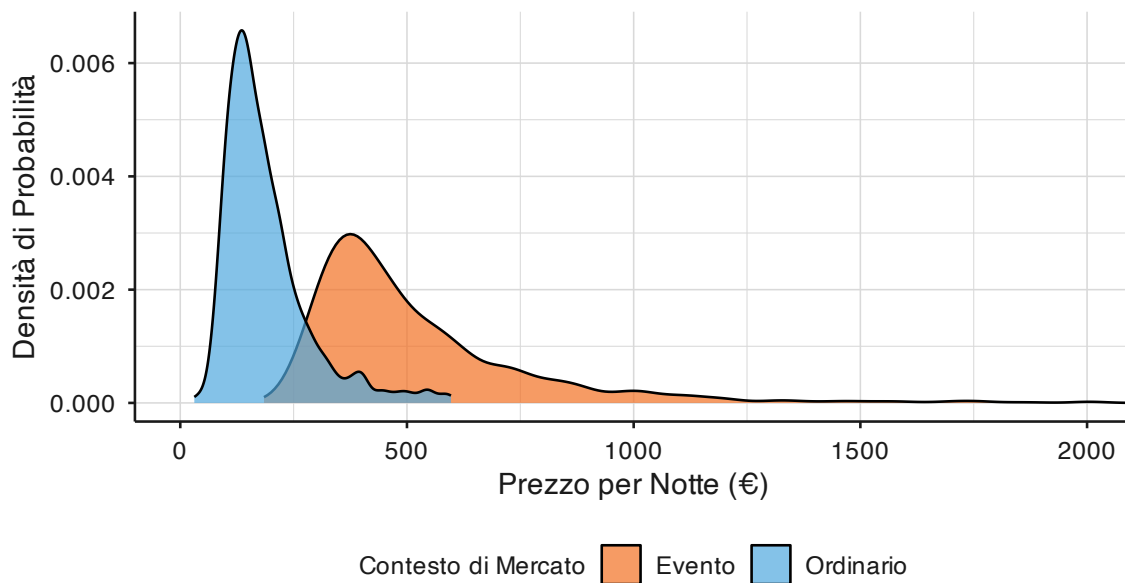
Distribuzione per i 1.474 appartamenti stimati dalla combinazione



```
print(g_generale)
```

Mercato complessivo Lido-Castello

Sintesi complessiva del mercato delle locazioni brevi durante il Festival



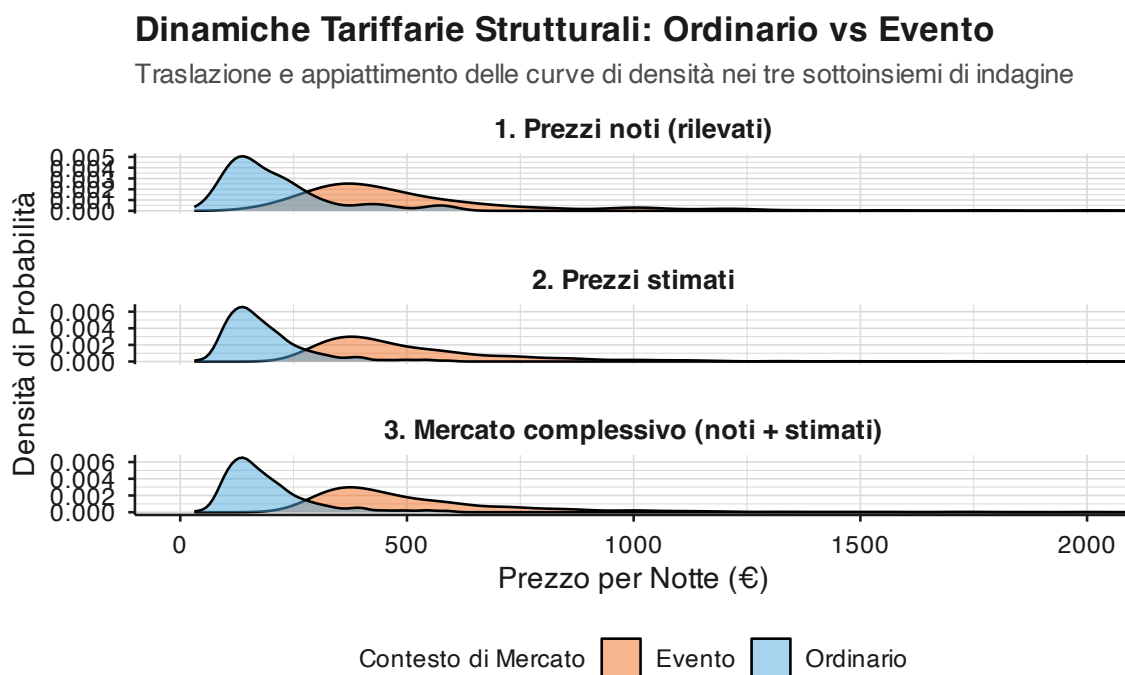
```
# =====  
# 10.4 GRAFICO UNIFICATO (GRIGLIA DI PANNELLI)  
# =====  
  
g_combinato <- ggplot(df_densita, aes(x = Prezzo, fill = Scenario)) +  
  geom_density(alpha = 0.5) +  
  scale_fill_manual(values = colori_scenario) +
```

```

facet_wrap(~ Campione, ncol = 1, scales = "free_y") +
coord_cartesian(xlim = c(0, 2000)) +
labs(title = "Dinamiche Tariffarie Strutturali: Ordinario vs Evento",
      subtitle = "Traslazione e appiattimento delle curve di densità nei tre
        ↳ sottoinsiemi di indagine",
      x = "Prezzo per Notte (€)", y = "Densità di Probabilità", fill = "Contesto di
        ↳ Mercato") +
tema_base +
theme(strip.text = element_text(face = "bold", size = sz_strip, colour = col_testo),
      panel.spacing = unit(1.2, "lines"))

# Produzione del grafico riepilogativo
print(g_combinato)

```



```

# Esporta i PNG per le slide SOLO in modalita' "slide": evita di sovrascrivere
# gli asset scuri di File_slide con versioni a sfondo bianco knittando il report.
if (!is_report) {
  ggsave("../File_slide/Densità_comb.png", g_combinato, width = 12, height = 6.5, dpi =
    ↳ 300, bg = col_sfondo)
  ggsave("../File_slide/Densità_noti.png", g_noti, width = 12, height = 6.5, dpi = 300,
    ↳ bg = col_sfondo)
  ggsave("../File_slide/Densità_valori_stimati.png", g_stimati, width = 12, height = 6.5,
    ↳ dpi = 300, bg = col_sfondo)
}

```

L’impennata dei prezzi Durante la Mostra del Cinema il mercato si trasforma radicalmente. In condizioni ordinarie gli alloggi presentano prezzi tra loro simili, con una distribuzione concentrata e appuntita. Durante il Festival questa regolarità viene meno: i prezzi si distribuiscono su un intervallo molto più ampio — la curva si appiattisce e si estende verso destra — facendo emergere una fascia di tariffe elevate che nel resto dell’anno non trova riscontro.

Voce dei clienti: analisi delle recensioni dei competitor

Principio. Il posizionamento non parte da *cosa si offre*, ma da *cosa lamentano* i clienti della concorrenza: ogni debolezza ricorrente degli affitti brevi è un potenziale messaggio di vendita per l'hotel.

Perimetro. Annunci competitor di **Lido e Castello**.

Metodo. Punteggi numerici delle recensioni e analisi del testo (*sentiment*) in lingua inglese, con gestione di negazioni e amplificatori.

```
.req <-
  ↪ c("data.table", "tidytext", "stopwords", "wordcloud", "cld2", "RColorBrewer", "sentimentr")
.miss <- .req[!.req %in% rownames(installed.packages())]
if (length(.miss)) install.packages(.miss, repos = "https://cloud.r-project.org")
suppressMessages({library(data.table); library(tidytext)})

dir_rec <- "output_recensioni"; dir.create(dir_rec, showWarnings = FALSE)
f_punteggi <- file.path(dir_rec, "punteggi_competitor.csv")
f_profilo <- file.path(dir_rec, "profilo_dimensioni.csv")
f_servizi <- file.path(dir_rec, "servizi_mancanti.csv")
f_freq <- file.path(dir_rec, "frequenza_parole.csv")
f_bigr <- file.path(dir_rec, "bigrammi_top.csv")
f_lowrated <- file.path(dir_rec, "servizi_lowrated.csv")
f_festival <- file.path(dir_rec, "servizi_festival.csv")
f_meta <- file.path(dir_rec, "meta_recensioni.csv")
files_attesi <- c(f_punteggi, f_profilo, f_servizi, f_freq, f_bigr, f_lowrated,
  ↪ f_festival, f_meta)

dims_rec <-
  ↪ c("review_scores_rating", "review_scores_accuracy", "review_scores_cleanliness",
      "review_scores_checkin", "review_scores_communication",
      ↪ "review_scores_location",
      "review_scores_value")

# Universo competitor (sempre disponibile, anche quando si carica dalla cache)
comp <- airbnb_pulito %>% filter(neighbourhood_cleansed %in% c("Lido", "Castello"))
n_comp <- nrow(comp)
n_con_rec <- sum(comp$number_of_reviews > 0, na.rm = TRUE)
n_ge5 <- sum(comp$number_of_reviews >= 5, na.rm = TRUE)
n_ge10 <- sum(comp$number_of_reviews >= 10, na.rm = TRUE)
# Soglia di recensioni per il "peggiore robusto": si abbassa se troppo pochi annunci la
  ↪ superano
min_rec <- if (n_ge10 >= 30) 10L else 5L

# Stopword per la wordcloud: oltre alle stopwords inglesi, si rimuovono luoghi,
# riferimenti di prossimità e giudizi generici, per far emergere i servizi.
stop_dom_recensioni <- c(
  "apartment", "apartments", "flat", "flats", "place", "places", "stay", "stayed", "staying",
  ↪ "room", "rooms",
  "host", "hosts", "airbnb", "accommodation", "property", "listing",
  "venice", "venezia", "lido", "castello", "location", "located", "area", "areas", "position",
  "central", "centrally", "centre", "center", "close", "near", "nearby", "far", "away", "around",
  ↪ "right", "next",
```

```

"distance", "walk", "walking", "walkable", "minute", "minutes", "reach", "step", "steps",
"vaporetto", "bus", "boat", "taxi", "water", "station", "stop", "stops", "marco", "rialto",
  ↪ "piazza", "square",
"bridge", "street", "streets", "canal", "canals", "island", "islands", "main", "city",
  ↪ "district", "train",
"time", "times", "day", "days", "night", "nights", "trip", "week", "weekend", "arrival",
  ↪ "arrived", "arrive",
"check", "checkin", "checkout",
"really", "great", "good", "nice", "lovely", "perfect", "beautiful", "wonderful", "amazing",
  ↪ "fantastic",
"excellent", "loved", "love", "comfortable", "comfy", "easy", "definitely", "recommend",
  ↪ "recommended",
"highly", "made", "make", "back", "get", "got", "one", "two", "three", "also", "well", "just",
  ↪ "very", "much",
"lot", "lots", "bit", "enough", "everything", "anything", "something", "nothing", "spacious",
  ↪ "quiet", "quietly",
"thank", "thanks", "home", "feel", "felt", "even", "need", "needed", "want", "wanted", "sure",
  ↪ "able", "always",
"every", "little", "big", "large", "small", "friendly", "kind", "helpful", "help", "gave",
  ↪ "give", "took", "take",
"come", "came", "went", "going", "see", "saw", "look", "looking", "find", "found", "way", "ways",
  ↪ "us", "will", "can",
"super", "like", "liked", "enjoy", "enjoyed", "experience", "overall", "especially",
  ↪ "biennale", "festival", "film", "cinema", "mostra",
"san", "restaurants", "restaurant", "local", "best", "met", "many", "short", "visit", "visited",
  ↪ "house", "tip", "tips", "recommendation", "recommendations")

if (all(file.exists(files_attesi))) {
  message("Recensioni: risultati caricati dalla cache.")
  punteggi      <- fread(f_punteggi)
  profilo_dim   <- fread(f_profilo)
  servizi_mancanti <- fread(f_servizi)
  freq          <- fread(f_freq)
  bigrammi      <- fread(f_bigr)
  servizi_lowrated <- fread(f_lowrated)
  servizi_festival <- fread(f_festival)
  n_festival     <- fread(f_meta)$n_festival[1]
} else {
  message("Recensioni: calcolo in corso (puo' richiedere qualche minuto)...")

  # A) Punteggi per competitor (media delle 7 dimensioni, scala 0-5)
  punteggi <- comp %>%
    filter(number_of_reviews > 0, !is.na(review_scores_rating)) %>%
    # per non creare "peggiori" artificiali; si tengono gli annunci con >= 4 dimensioni
    ↪ valide.
    mutate(across(all_of(dims_rec), ~ifelse(.x == 0, NA_real_, .x))) %>%
    filter(rowSums(!is.na(across(all_of(dims_rec)))) >= 4) %>%
    mutate(punteggio_medio = rowMeans(across(all_of(dims_rec)), na.rm = TRUE)) %>%
    select(id, neighbourhood_cleaned, number_of_reviews, all_of(dims_rec),
      ↪ punteggio_medio) %>%
    arrange(punteggio_medio)
  fwrite(punteggi, f_punteggi)

  profilo_dim <- punteggi %>%

```

```

summarise(across(all_of(dims_rec), ~mean(.x, na.rm = TRUE))) %>%
pivot_longer(everything(), names_to = "dimensione", values_to = "media") %>%
mutate(dimensione = sub("review_scores_", "", dimensione)) %>%
arrange(media)
fwrite(profilo_dim, f_profilo)

# B/C) Text mining (solo inglese)
rece <- fread("../dati/reviews.csv", encoding = "UTF-8")
rece <- rece[listing_id %in% comp$id][!is.na(comments) & nchar(comments) > 20]
rece <- rece[!str_detect(tolower(comments),
  "automated posting|host cancel|cancel(l)?ed (this|the) reservation")]
rece[, testo := str_replace_all(comments, "<[~>]+>", " ")]
rece[, testo := str_replace_all(testo, "[\\r\\n]+", " ")]
rece[, lingua := cld2::detect_language(testo)]
rece_en <- rece[lingua == "en"]

stop_dom <- stop_dom_recensioni
stopE <- unique(c(stopwords::stopwords("en"), stop_dom))

freq <- as.data.frame(rece_en) %>% select(testo) %>%
  unnest_tokens(parola, testo) %>%
  filter(str_detect(parola, "[a-z]+$"), !parola %in% stopE, nchar(parola) >= 3) %>%
  count(parola, sort = TRUE)
fwrite(freq, f_freq)

servizi <- list(
  "Aria condizionata" = "air[ -]?condition|aircon|\\ba/?c\\b|cooling",
  "Ascensore" = "elevator|\\blift\\b",
  "Wi-Fi / internet" = "wifi|wi-?fi|internet|connection",
  "Riscaldamento" = "heating|heater|radiator|cold (room|apartment|flat)",
  "Acqua calda" = "hot water",
  "Rumore / insonorizzazione" = "noise|noisy|soundproof|loud|quiet",
  "Pulizia" = "clean|cleanliness|dirty|dust|stain",
  "Cucina / attrezzatura" = "kitchen|stove|oven|microwave|fridge|cooking",
  "Lavatrice / lavanderia" = "washing machine|washer|laundry|dryer",
  "Asciugamani / biancheria" = "towel|linen|sheet|bedding",
  "Colazione" = "breakfast",
  "Parcheggio" = "parking|\\bpark\\b|garage",
  "Reception / check-in" = "reception|front desk|check[ -]?in|self check",
  "TV" = "\\btv\\b|television|netflix",
  "Doccia / pressione acqua" = "shower|water pressure",
  "Asciugacapelli" = "hair ?dryer|hairdryer",
  "Caffè / bollitore" = "coffee|kettle|espresso",
  "Scale (ripide)" = "stairs|flights of stairs|steep stairs"
)
neg_cue <- paste0("\\b(no|not|without|lack|lacking|lacked|missing|broken|didn't|did
  ↪ not|",
  "wasn't|was not|isn't|is not|doesn't|does not|couldn't|could
  ↪ not|need|needed|",
  "needs|would have|should have|absence|there was no|there is no|no
  ↪ working|",
  "not working|too (hot|cold|small|noisy))\\b")
# Flag "Festival del Cinema" a livello di recensione (per isolare chi soggiorna per
  ↪ l'evento)

```



```

festival_regex <- paste0("film festival|cinema festival|movie festival|venice film
↪ festival|venice festival|",
                        "film fest|red carpet|premiere|during the festival|for the
↪ festival|festival period|film screening")
rece_en[, festival := str_detect(tolower(testo), festival_regex)]
frasi <- as.data.frame(rece_en) %>% transmute(listing_id, festival, frase = testo) %>%
  mutate(frase = str_split(tolower(frase), "(?<=[.!?])\\s+")) %>%
  unnest(frase) %>% filter(nchar(frase) >= 10)
# Sentiment con valence-shifting (sentimentr) sulle sole frasi che citano un servizio.
amen_regex <- paste(unlist(servizi), collapse = "|")
frasi_amen <- frasi[str_detect(frase$frase, amen_regex), , drop = FALSE]
frasi_amen$sentiment <- sentimentr::sentiment_by(frase_amen$frase)$ave_sentiment
frasi_amen$ha_neg <- str_detect(frase_amen$frase, neg_cue)
# tabella servizi (sentiment) da un sottoinsieme di frasi_amen
agg_servizi <- function(fr) lapply(names(servizi), function(s) {
  idx <- str_detect(fr$frase, servizi[[s]])
  data.frame(servizio = s, menzioni = sum(idx),
             menzioni_negative = sum(idx & fr$ha_neg),
             sentiment_medio = round(mean(fr$sentiment[idx], na.rm = TRUE), 3),
             quota_negativa = round(mean(fr$sentiment[idx] < 0, na.rm = TRUE), 3))
}) %>% bind_rows()
# (1) tutti i competitor; (2) alloggi peggio recensiti (decile inf.); (3) soggiorni
↪ Festival
servizi_mancanti <- agg_servizi(frase_amen) %>% arrange(sentiment_medio)
fwrite(servizi_mancanti, f_servizi)
soglia_low <- as.numeric(quantile(punteggi$punteggio_medio, 0.10, na.rm = TRUE))
id_low <- punteggi %>% filter(punteggio_medio <= soglia_low, number_of_reviews >=
↪ 3) %>% pull(id)
servizi_lowrated <- agg_servizi(frase_amen %>% filter(listing_id %in% id_low)) %>%
↪ arrange(sentiment_medio)
fwrite(servizi_lowrated, f_lowrated)
servizi_festival <- agg_servizi(frase_amen %>% filter(festival)) %>%
↪ arrange(sentiment_medio)
fwrite(servizi_festival, f_festival)
n_festival <- sum(rece_en$festival)
fwrite(data.frame(n_festival = n_festival), f_meta)

bigrammi <- as.data.frame(rece_en) %>% select(testo) %>%
  unnest_tokens(bg, testo, token = "ngrams", n = 2) %>%
  separate(bg, c("w1", "w2"), sep = " ") %>%
  filter(!w1 %in% stopE, !w2 %in% stopE,
         str_detect(w1, "^[a-z]+$"), str_detect(w2, "^[a-z]+$")) %>%
  unite(bigramma, w1, w2, sep = " ") %>% count(bigramma, sort = TRUE) %>% slice(1:40)
fwrite(bigrammi, f_bigr)
}

# ---- Valori derivati per il report (dinamici) ----
sm <- servizi_mancanti %>% as.data.frame() %>% arrange(sentiment_medio) # sentiment
↪ piu' negativo in alto
peggiore <- punteggi %>% as.data.frame() %>% arrange(punteggio_medio) %>% slice(1)
↪ # peggiore assoluto (post-filtro 0)
peggiore_rob <- punteggi %>% as.data.frame() %>% filter(number_of_reviews >= min_rec) %>%
  arrange(punteggio_medio) %>% slice(1) # peggiore con >= min_rec recensioni
# Dimensioni piu' deboli del peggiore

```

```

pegg_dims <- peggiore_rob %>% select(all_of(dims_rec)) %>%
  pivot_longer(everything(), names_to = "dimensione", values_to = "voto") %>%
  mutate(dimensione = sub("review_scores_", "", dimensione)) %>% arrange(voto)
pegg_dims_str <- paste0(head(pegg_dims$dimensione, 3), " (", round(head(pegg_dims$voto,
  ↪ 3), 1), ") ", collapse = ", ")
dim_debole <- profilo_dim$dimensione[1]
dim_debole_val <- round(profilo_dim$media[1], 2)
serv_critici <- paste(head(sm$servizio, 6), collapse = ", ") # ranking per sentiment
  ↪ (peso fine)
serv_buoni <- sm %>% arrange(desc(sentiment_medio)) %>% slice(1:5) %>% pull(servizio)
  ↪ %>% paste(collapse = ", ")
serv_volume <- sm %>% arrange(desc(menzioni_negative)) %>% slice(1:5) %>% pull(servizio)
  ↪ %>% paste(collapse = ", ")
serv_elogio <- paste(tail(sm$servizio, 2), collapse = " e ") # sentiment piu' alto =
  ↪ bene
# Pool degli alloggi peggio recensiti (decile inferiore del punteggio) e soggiorni
  ↪ Festival
soglia_low_rep <- as.numeric(quantile(punteggi$punteggio_medio, 0.10, na.rm = TRUE))
n_lowrated <- punteggi %>% as.data.frame() %>%
  filter(punteggio_medio <= soglia_low_rep, number_of_reviews >= 3) %>% nrow()
sl <- servizi_lowrated %>% as.data.frame() %>% arrange(sentiment_medio)
serv_low <- paste(head(sl$servizio, 6), collapse = ", ")
# Festival: campione piccolo -> soglia minima di menzioni abbassata a 3 (riduce comunque
  ↪ il rumore da 1-2 frasi)
sf <- servizi_festival %>% as.data.frame() %>% filter(menzioni >= 3) %>%
  ↪ arrange(sentiment_medio)
serv_festival <- paste(head(sf$servizio, 5), collapse = ", ")

```

L'ultimo passaggio strategico guarda al mercato **dal punto di vista del cliente**. Sono state analizzate le recensioni dei competitor di **Lido e Castello** (1501 annunci con punteggi disponibili) sotto due profili: i **punteggi numerici** lasciati dagli ospiti e il **testo** delle recensioni in lingua inglese, da cui far emergere i servizi percepiti come carenti. È esattamente l'informazione che orienta il **posizionamento di marketing** dell'hotel: non *cosa pensiamo* di offrire, ma *cosa lamentano* i clienti della concorrenza.

1. Dove sono deboli i competitor (punteggi)

Table 12: Punteggio medio dei competitor (Lido + Castello) per dimensione, dal piu' debole al piu' forte.

Dimensione	Punteggio medio (0-5)
value	4.644
rating	4.752
cleanliness	4.776
accuracy	4.787
checkin	4.826
communication	4.827
location	4.881

La base dati è solida: dei **1605** annunci competitor (Lido + Castello), **1502** hanno recensioni, di cui **1331** con almeno 5 e **1186** con almeno 10. La soglia per il “peggiore robusto” è fissata a **10** recensioni (si abbassa automaticamente se troppo pochi annunci la superano).

La dimensione strutturalmente più debole dell'intero mercato competitor è **value** (media 4.64/5): è il punto su cui l'hotel ha il maggior margine per **differenziarsi**. Il competitor con la valutazione media più bassa si ferma a **2.71/5**; tra quelli con almeno 10 recensioni (dato più affidabile) il minimo è **3.97/5**, debole soprattutto su **cleanliness (3.5)**, **rating (3.8)**, **value (3.8)** — proprio le aree di servizio in cui riceve le lamentele e su cui l'hotel può fare la differenza.

2. I servizi che i clienti segnalano come carenti

Per pesare correttamente le lamentele si analizza il **sentiment** delle frasi (con gestione di negazioni e amplificatori: “*not clean*” viene letto come negativo, “*very clean*” come molto positivo). Il sentiment medio di ciascun servizio, è un segnale molto più affidabile del semplice conteggio di parole di negazione.

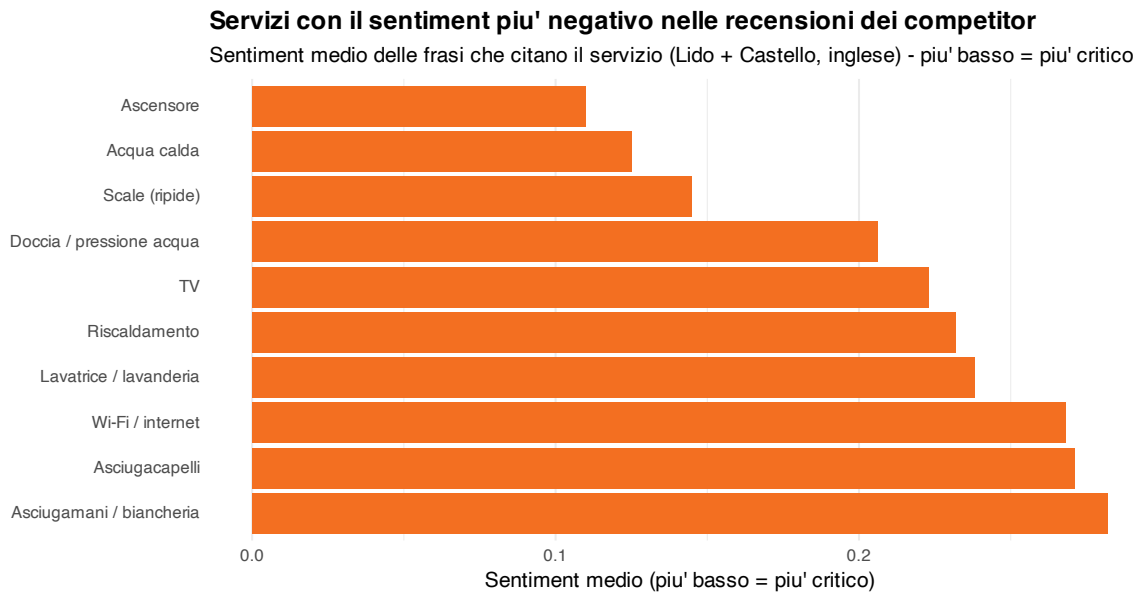


Table 13: Servizi ordinati dal sentiment piu' negativo. 'Menz. negative' = conteggio con parole di negazione; 'Sentiment medio' = tono medio (gestione negazioni), piu' basso = piu' critico.

Servizio	Menzioni	Menz. negative	Sentiment medio	Quota frasi neg.
Ascensore	659	461	0.110	22%
Acqua calda	575	191	0.125	35%
Scale (ripide)	2865	780	0.145	25%
Doccia / pressione acqua	2616	667	0.206	26%
TV	496	168	0.223	17%
Riscaldamento	651	179	0.232	22%
Lavatrice / lavanderia	2171	630	0.238	12%
Wi-Fi / internet	2357	731	0.268	23%
Asciugacapelli	230	84	0.271	10%
Asciugamani / biancheria	1578	428	0.282	17%
Parcheggio	708	73	0.291	6%
Caffe / bollitore	2853	468	0.295	8%

L'analisi del *tono* affina nettamente la lettura:

- **Servizi già gestiti bene** (sentiment più positivo): Pulizia, Colazione, Reception / check-in, Rumore / insonorizzazione, Cucina / attrezzatura. Qui i competitor partono forti: non sono leve di differenziazione, ma uno standard minimo da garantire per non scendere sotto le aspettative.

I servizi gestiti meglio (sentiment più positivo)

Lo specchio della tabella precedente: i servizi che i clienti dei competitor citano in tono più favorevole. Non sono leve di differenziazione (la concorrenza è già forte), ma rappresentano lo **standard minimo** che l'hotel deve garantire per non deludere le aspettative.

Table 14: Servizi ordinati dal sentiment più positivo (dove i competitor sono già forti).

Servizio	Menzioni	Sentiment medio	Quota frasi neg.
Pulizia	22632	0.683	3%
Colazione	2807	0.502	3%
Reception / check-in	6916	0.397	8%
Rumore / insonorizzazione	16223	0.356	9%
Cucina / attrezzatura	6966	0.335	8%
Aria condizionata	4367	0.310	14%
Caffè / bollitore	2853	0.295	8%
Parcheggio	708	0.291	6%

Cosa lamentano gli ospiti degli alloggi peggio recensiti

Analizzare un *singolo* competitor peggiore non basta: poche recensioni producono pochissime frasi sui servizi, troppo poche per concludere. È invece molto più informativo il **pool degli alloggi peggio recensiti** (decile inferiore del punteggio, 134 annunci con punteggio 4.55): qui le recensioni negative si concentrano e i problemi emergono in modo robusto. È la risposta operativa alla domanda “su quali alloggi conviene concentrare l'analisi”: **non tutti, ma i peggio valutati**, dove le criticità sono reali e ricorrenti.

Table 15: Servizi nelle recensioni degli alloggi peggio recensiti (decile inferiore). Il sentiment qui è nettamente più basso: i problemi reali si concentrano in questi annunci.

Servizio	Menzioni	Menz. negative	Sentiment medio	Quota frasi neg.
Acqua calda	32	16	-0.041	53%
Scale (ripide)	53	21	0.010	38%
Ascensore	14	9	0.014	29%
Doccia / pressione acqua	93	32	0.049	45%
Riscaldamento	22	9	0.067	41%
TV	20	6	0.088	30%
Wi-Fi / internet	97	40	0.122	37%
Asciugacapelli	5	2	0.153	20%
Cucina / attrezzatura	148	50	0.158	24%
Lavatrice / lavanderia	32	12	0.170	12%

Rispetto al mercato complessivo, qui il sentiment è sistematicamente più basso, **risultando più consistente**, e i nodi critici risaltano: Acqua calda, Scale (ripide), Ascensore, Doccia / pressione acqua, Riscaldamento, TV. Sono le carenze concrete che fanno scivolare un alloggio in fondo alla classifica — e quindi gli aspetti su cui l'hotel deve ottenere il vantaggio competitivo più immediato.

Le opinioni di chi ha soggiornato durante il Festival

Sarebbe ideale isolare il giudizio di chi ha alloggiato *per la Mostra del Cinema*. In pratica **solo 92 recensioni in inglese** citano esplicitamente il festival: i recensori raramente dichiarano il motivo del soggiorno. Il campione è quindi piccolo e i numeri sottostanti vanno letti come **indicazione qualitativa**, non come stime robuste; il quadro affidabile resta quello dell'intero mercato Lido–Castello (l'area che ospita l'evento).

Riportiamo comunque i servizi citati da chi nomina il festival. Data l'esiguità del campione, il rilevamento è stato **allargato** (oltre a “film festival”: *red carpet*, *premiere*, “during/for the festival”...) e la soglia minima di menzioni abbassata a **3** per mostrare più servizi, restando consapevoli che con poche frasi il sentiment è instabile:

Table 16: Servizi nelle 92 recensioni che citano il Festival del Cinema (≥ 3 menzioni). Campione limitato: lettura solo indicativa.

Servizio	Menzioni	Sentiment medio	Quota frasi neg.
Lavatrice / lavanderia	7	0.122	29%
Wi-Fi / internet	9	0.123	44%
Doccia / pressione acqua	6	0.129	17%
Cucina / attrezzatura	15	0.283	20%
Aria condizionata	9	0.288	11%
Caffè / bollitore	4	0.310	25%
Reception / check-in	11	0.342	18%
Rumore / insonorizzazione	26	0.430	12%
Pulizia	29	0.624	7%

Pur con tutte le cautele, l'indicazione è **coerente** con l'analisi complessiva.

3. Le parole dei clienti

```
## Loading required package: RColorBrewer
```



4. Implicazioni di marketing per l'Hotel

Il filo conduttore è netto: le lamentele ricorrenti sugli affitti brevi sono esattamente i punti in cui un hotel è strutturalmente più forte. La strategia di comunicazione può quindi essere la chiave.

Dove puntare:

1. **“Zero scale, zero stress.”** L’ascensore come antidoto assoluto al disagio numero uno degli affitti brevi veneziani: l’incubo dei bagagli pesanti su scale ripide. *Ascensore* e *Scale* sono tra i servizi col tono più critico nelle recensioni dei competitor; l’hotel li trasforma in un vantaggio.
2. **“Il comfort idraulico come vero lusso.”** Docce ad alta pressione e acqua sempre calda contro i disservizi cronici dei vecchi appartamenti: l’impianto alberghiero diventa un elemento da valorizzare.
3. **“Standard professionali, zero imprevisti.”** Climatizzazione e Wi-Fi ad alta velocità garantiti: dove l’host privato fa “fai-da-te”, l’hotel assicura un’infrastruttura impeccabile e assistenza continua.

Attivazione operativa:

- **Espansione Multi-Canale e sito proprio (Booking, Expedia, sito diretto):** Portare in cima alla descrizione e nelle foto i servizi anti-lamentela (ascensore, aria condizionata, Wi-Fi, reception 24h, insonorizzazione), con un blocco del tipo “Cosa trovi sempre da noi — e spesso manca altrove”
- **Campagne a pagamento:** titoli costruiti sui gap dei competitor (es. “Ascensore, aria condizionata e reception 24h, a due passi dal vaporetto”).
- **Gestione delle recensioni:** Sollecitare il feedback degli ospiti proprio sui punti di forza, per costruire un vantaggio reputazionale misurabile sulle stesse dimensioni in cui i competitor sono deboli.
- **Periodo della Mostra:** Giustificare i prezzi maggiorati durante la Mostra comunicando affidabilità e servizi dedicati.